MiniMind 学习笔记

第一部分：Model构建

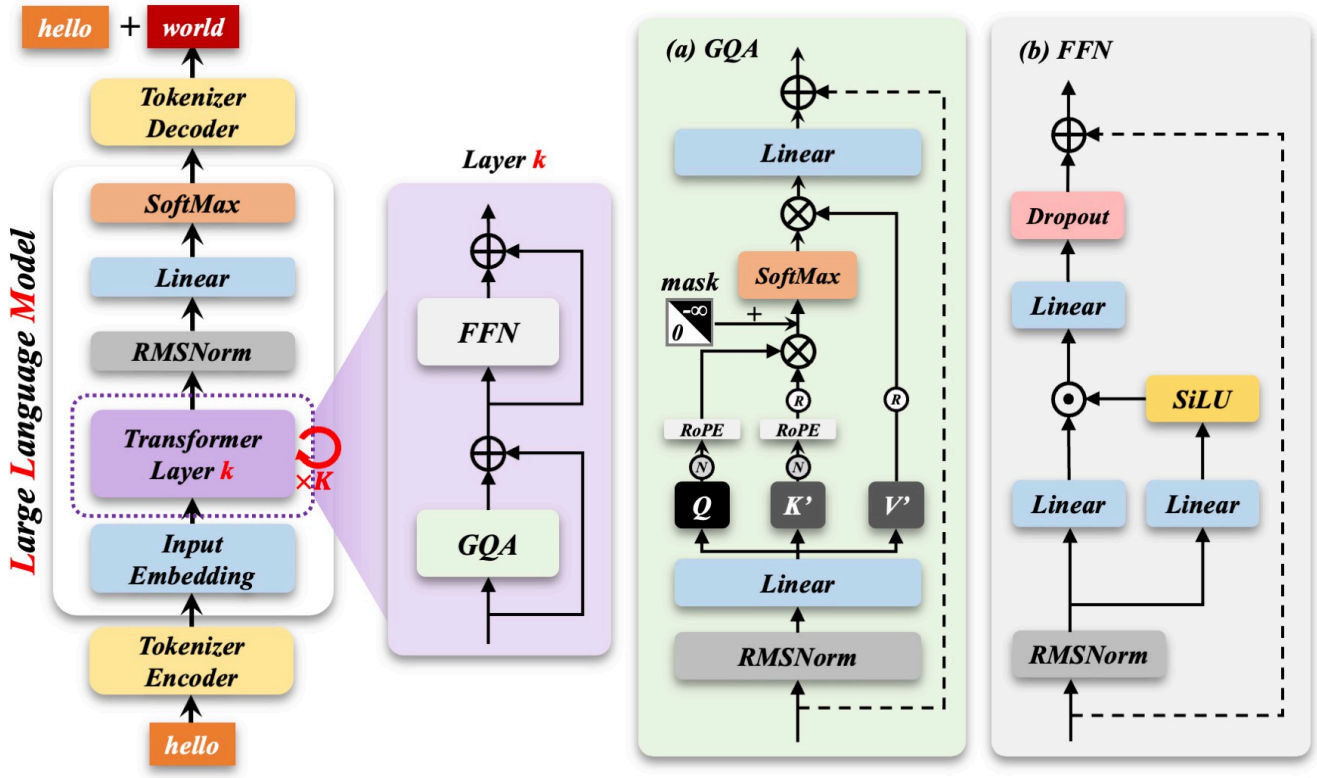
1. PyTorch 基础组件

在阅读 MiniMind 的模型代码前，需要先了解 PyTorch 中几个核心对象。它们共同构成了模型定义、梯度计算和参数更新的基础。

组件	作用	与 MiniMind 的关系
`Tensor`	张量，是 PyTorch 中存储和计算数据的基本单位。	输入文本转换后的 token、隐藏状态和模型输出，都会以张量形式参与计算。
`nn.Parameter`	可优化参数；在 `Module` 中注册后，优化器会自动更新它。	RMSNorm 中的 `weight` 就是可学习的缩放参数。
`nn.Module`	神经网络层或容器的基类，可组合多个层并管理参数。	`RMSNorm`、注意力层和整个 MiniMind 模型都继承自它。
`Autograd`	自动微分机制，能够根据计算过程自动计算梯度。	根据损失函数自动求出每个参数的梯度。
`Optimizer`	根据梯度更新可学习参数的优化器。	训练脚本使用优化器更新模型参数，使损失逐步降低。

这些概念的关系可以概括为：`Tensor` 参与前向计算，`Autograd` 计算梯度，`Optimizer` 更新 `Parameter`；而 `Module` 用来组织这些参数和计算层。

2. MiniMind 模型整体结构



The Structure of MiniMind (Dense Model)

MiniMind 是一个自回归语言模型：输入文本先经过 Tokenizer 转换为 token 编号，再经过 Input Embedding 映射为向量；向量依次通过多层 Transformer Layer，最后由 Linear 层和 Softmax 预测下一个 token，并由 Tokenizer Decoder 转回文本。

每个 Transformer Layer 包含两个核心子层，并都配有残差连接：

1. **GQA（Grouped Query Attention，分组查询注意力）**：通过 Q、K、V 计算 token 之间的关联；使用因果掩码（mask）保证当前位置只能关注自己及之前的 token，使用 RoPE（旋转位置编码）注入位置信息。
2. **FFN（前馈神经网络）**：对每个 token 的特征进行非线性变换。图中先经两条 Linear 分支，其中一条经过 SiLU 激活，再逐元素相乘，最后通过 Linear 投影回隐藏维度。

图中的 RMSNorm 位于注意力层和 FFN 的输入处，用于稳定各层特征的数值尺度。下面将对 RMSNorm 的原理和源码实现进行展开说明。

3. RMSNorm

3.1 为什么需要归一化

设一层中有计算：

$$y = w \times x$$

根据链式法则，参数 w 的梯度为：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w} = \frac{\partial L}{\partial y} \times x$$

可见梯度大小与输入 x 的数值有关。若各层激活值过大或过小，梯度在层间传递时就可能发生梯度爆炸或梯度消失，使训练不稳定。因此，需要通过归一化让特征的数值尺度保持在合理范围内。

从这个推导还可以看出：当输入的尺度被控制住时，梯度项中的 x 也不会出现过大的波动，训练过程会更稳定。传统归一化通常会将数据调整到均值接近 0、标准差接近 1；而 RMSNorm 采用更简化的方式，只控制均方根（RMS）尺度，并不强制均值为 0。

3.2 RMSNorm 的计算方式

对于最后一个维度长度为 d 的输入向量 x，RMSNorm 先计算均方根：

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}$$

再进行缩放：

$$\mathrm{RMSNorm}(x) = w \odot \frac{x}{\mathrm{RMS}(x)}$$

按单个元素写，计算过程可表示为：

$$y_i = \frac{x_i}{\sqrt{\frac{1}{n}\sum_{j=1}^{n}(x_j)^2 + \varepsilon}} \times \gamma_i$$

其中，ε 用于防止除零；w（或 γ）是可学习的缩放参数；$\odot$ 表示逐元素相乘。

与传统 LayerNorm 相比，RMSNorm 不计算均值，也不执行减均值操作，只依据均方根调整尺度。因此它的计算更简单、开销更低，同时在大语言模型中通常能保持良好的训练效果。

3.3 MiniMind 中的实现对应

```
class RMSNorm(torch.nn.Module):
    def __init__(self, dim: int, eps: float = 1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def norm(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)

    def forward(self, x):
        return (self.weight * self.norm(x.float())).type_as(x)
```

代码中的 `x.pow(2).mean(-1)` 对应“平方后取均值”；`torch.rsqrt(...)` 计算平方根的倒数；两者相乘即完成 $x/\mathrm{RMS}(x)$。最后再乘以可学习参数 `weight`，并转换回原始数据类型。

3.4 小结

RMSNorm 的作用是控制隐藏状态的数值尺度，使梯度传播更稳定。MiniMind 采用它替代传统 LayerNorm，减少了均值计算，也符合现代大语言模型常见的实现方式。

4. 正弦位置编码 PE

4.1 为什么需要位置编码

自注意力机制只计算 Q 和 K 的相似度，本身对输入顺序不敏感。比如“狗咬人”和“人咬狗”含有相同的词，却有完全不同的语义；模型必须获得词序信息，才能分辨谁是施事、谁是受事。

4.2 PE 是怎样计算的

Transformer 也可以直接把正弦、余弦组成的位置向量加到 token 表示上，这种方法称为正弦位置编码（sinusoidal positional encoding，简称 PE）。对模型维度 d_{model}，常见定义为：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

假设输入序列长度为 L，每个 token 的表示维度为 d_{model}。PE 会先为每个位置 `pos=0,1,...,L-1` 生成一个同样是 d_{model} 维的位置向量，再与 token 表示逐元素相加：

$$H_{pos} = X_{pos} + PE(pos)$$

其中 X_{pos} 是第 `pos` 个 token 的 embedding，H_{pos} 是加入位置信息后的表示。PE 本身不包含 token 内容，只由位置和维度索引计算得到。

例如 `d_model=4` 时只有两组频率：

```
维度 0: sin(pos)       维度 1: cos(pos)
维度 2: sin(pos / 100) 维度 3: cos(pos / 100)
```

所以：

```
pos=0: PE(0) = [0,      1,      0,      1]
pos=1: PE(1) = [0.8415, 0.5403, 0.0100, 0.99995]
```

每个位置都有一个独立的向量，但这些向量都由同一组正弦、余弦公式确定。然后把对应位置的 PE 加到 token embedding 上，送入 Transformer。

4.3 PE 的线性平移性质

令：

$$\omega_i = \frac{1}{10000^{2i/d_{model}}}, \qquad \alpha = pos\,\omega_i$$

则第 i 对维度可以写成二维向量：

$$p_i(pos) = \begin{bmatrix} \sin\alpha \\ \cos\alpha \end{bmatrix}$$

现在把位置向后移动 k，令 $\beta = k\omega_i$。根据三角函数的和角公式：

$$\sin(\alpha+\beta)=\sin\alpha\cos\beta+\cos\alpha\sin\beta,$$
$$\cos(\alpha+\beta)=\cos\alpha\cos\beta-\sin\alpha\sin\beta.$$

可以写成矩阵形式：

$$\begin{bmatrix}\cos\beta & \sin\beta\\ -\sin\beta & \cos\beta\end{bmatrix}\begin{bmatrix}PE(pos,2i)\\ PE(pos,2i+1)\end{bmatrix}=\begin{bmatrix}PE(pos+k,2i)\\ PE(pos+k,2i+1)\end{bmatrix}$$

因此：

> 对固定的维度对 `i`，`pos+k` 的位置编码可以由 `pos` 的位置编码经过一个只与偏移量 `k` 和该维度频率 `ω_i` 有关、而与 `pos` 无关的线性变换得到。

这个变换矩阵对每一对维度都成立；把所有维度对组合起来，就是一个分块对角的线性变换。举例来说，取 $i=0$、$pos=1$、$k=2$，此时 $\omega_0=1$：

```
PE(1) = [sin(1), cos(1)] = [0.8415, 0.5403]
β = 2
```

用

```
[[cos(2),  sin(2)],
 [-sin(2), cos(2)]]
```

相乘后得到：

```
[sin(3), cos(3)] = [0.1411, -0.9900] = PE(3)
```

这说明 PE 能用统一的线性变换表达位置平移关系。

4.4 PE 的主要缺点

1. **位置和内容在输入处相加。** token embedding 与 PE 混合后，后续注意力需要自己学习如何从混合结果中利用位置信息；位置并没有直接出现在 Q、K 的相对关系中。
2. **注意力分数不天然只依赖相对距离。** 即使 `PE(pos+k)` 可以由 `PE(pos)` 线性变换得到，加入 token 表示后，注意力点积仍会包含内容项、位置项以及内容和位置的交叉项，通常不能像 RoPE 那样直接化为只与 `n-m` 有关的形式。
3. **频率是固定的。** PE 使用预先设定的正弦、余弦频率；距离很大时，某些频率会周期性重复，可能出现位置混淆（aliasing）。
4. **长上下文外推不一定可靠。** 虽然公式可以计算超过训练长度的位置，但模型训练时没有充分见过这些角度分布，实际效果可能下降。

5. RoPE：旋转位置编码

5.1 为什么需要 RoPE

位置关系不仅是"第几个 token"，还包括"相距多远、谁在前谁在后"。RoPE 对 Q、K 施加与位置有关的旋转，使注意力分数体现相对距离。它既能保留词序，又比单纯的绝对位置编号更易适应不同长度的文本，并且不需要额外的可训练位置参数。

传统绝对位置编码直接为第 1、第 2 个等位置加入不同向量；RoPE 虽然也按绝对位置旋转，但 Q、K 的点积最终只与位置差有关，因而可以表达相对位置关系。

需要注意，`RoPE=PE*token` 不是准确的公式。两者的计算链不同：

```
PE：    token 表示 x  +  位置向量 PE(pos)  ->  送入 Transformer
RoPE：  token 表示 x  ->  Q=xW_Q、K=xW_K
        Q、K 再分别乘以位置旋转矩阵 R(pos)  ->  计算注意力
```

这里的"乘"是矩阵乘法，且 RoPE 旋转的是 Q、K 的二维维度组，不是把一个叫作 PE 的向量直接乘到 token 上。代码中的：

```
q_rot = q * cos + rotate_half(q) * sin
```

只是把矩阵旋转 $R(pos)q$ 展开成逐元素运算后的写法。

5.2 RoPE 推导过程

5.2.1 从注意力计算开始

设 Transformer 的输入为：

$$X \in \mathbb{R}^{B\times L\times D}$$

其中，B 为 batch size，L 为序列长度，D 为隐藏维度。输入经过线性映射得到：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

第 m、n 个 token 对应 q_m, k_m, v_m 和 q_n, k_n, v_n。加入位置编码前的注意力分数为：

$$\langle q_m, k_n \rangle = q_m^T k_n$$

目标是在此分数中加入位置信息，并使其主要与相对距离 $n-m$ 有关。

5.2.2 用二维旋转表示位置

对二维向量，角度为 α 的旋转矩阵为：

$$R(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}, \qquad v' = R(\alpha)v$$

令 θ 为该维度对的旋转频率。对第 m、n 个 token 的 Q、K 进行旋转：

$$q_m^{\mathrm{RoPE}} = R(m\theta)q_m, \qquad k_n^{\mathrm{RoPE}} = R(n\theta)k_n$$

RoPE 通常只作用于 Q 和 K，不作用于 V。

5.2.3 得到相对位置

旋转后的注意力分数为：

$$\begin{aligned}\langle q_m^{\mathrm{RoPE}}, k_n^{\mathrm{RoPE}} \rangle &= q_m^T R(m\theta)^T R(n\theta) k_n \\ &= q_m^T R(-m\theta) R(n\theta) k_n \\ &= q_m^T R((n-m)\theta) k_n\end{aligned}$$

这里利用了：

$$R(\alpha)^T = R(-\alpha), \qquad R(\alpha+\beta) = R(\alpha)R(\beta)$$

最终结果只依赖 $n-m$，所以 Q、K 的点积自然编码了两个 token 的相对位置。

5.2.4 扩展到高维向量与源码

实际模型中的 Q、K 不是二维向量，而是 d 维向量（通常 d 为偶数）。为了说明数学原理，通常先采用“相邻维度配对”的写法，把它拆成 $d/2$ 个二维向量：

$$q = \left[(q_0, q_1), (q_2, q_3), \ldots, (q_{d-2}, q_{d-1})\right]$$

在这种写法中，第一组是 `(q0,q1)`，第二组是 `(q2,q3)`。这只是维度排布约定，并不是所有代码都采用这种排布。

第 i 组使用自己的旋转频率 θ_i。对于位置为 m 的 token，第 i 组的旋转角度为 $m\theta_i$，因此：

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$

把所有二维旋转放在一起，就是一个分块对角矩阵：

$$R_m = \mathrm{diag}\left(R(m\theta_0), R(m\theta_1), \ldots, R(m\theta_{d/2-1})\right), \qquad q_m^{\mathrm{RoPE}} = R_m q_m$$

对 K 的处理完全相同：$k_m^{\mathrm{RoPE}} = R_m k_m$。在注意力计算中，Q 和 K 各自按所在位置旋转，V 不做旋转。

不同组的频率通常为：

$$\theta_i = \mathrm{rope_base}^{-2i/d}$$

因此，前面的组频率较高、旋转较快，后面的组频率较低、旋转较慢。用户给出的示意图中，第一组和最后一组使用不同的 θ_i，正是这种“每个二维平面使用不同频率”的含义。

6. YaRN：RoPE 的长度外推

6.1 为什么需要 YaRN

RoPE 在训练时只见过有限长度的序列。例如，若模型训练时的最大长度为 2048 个 token，而推理时输入 8192 个 token，那么后半部分位置对应的旋转角度没有在训练中充分出现过。位置角度分布发生变化后，Q、K 的相对位置关系可能失真，模型对长文本的理解能力会下降。

直接把最大长度调大并不能解决这个问题，因为模型参数仍是在较短上下文中训练得到的。YaRN（Yet another RoPE extensioN）是一种 RoPE 长度外推方法：它在推理时调整不同频率的位置旋转速度，让模型能以更平滑的方式处理超出原训练长度的文本。

6.2 YaRN 的基本原理

RoPE 的每一对维度都有对应频率 f_i，位置为 t 时的旋转角度可以理解为：

$$\text{angle}_{t,i} = t \cdot f_i$$

若希望将上下文长度扩展 s 倍，最简单的做法是将全部频率缩小为 f_i/s，从而放慢旋转速度；这相当于把更长的位置范围压缩回模型较熟悉的角度范围。但对所有频率一律缩放会损伤短距离的位置信息。

YaRN 采用按频率分段、平滑过渡的策略：

1. **高频维度**主要表达短距离、局部位置关系，尽量保持原频率，不做或少做缩放。
2. **低频维度**主要表达长距离、全局位置关系，将频率缩小约 s 倍，使其适应更长的上下文。
3. **中间频率维度**使用线性 ramp 平滑地从“不缩放”过渡到“缩放”，避免在频率边界处突然变化。

可以将调整后的频率写成：

$$f_i' = f_i \left[(1 - r_i) + \frac{r_i}{s} \right], \qquad r_i \in [0, 1]$$

其中，$r_i = 0$ 表示保持原频率，$r_i = 1$ 表示按 $1/s$ 缩放，中间值表示平滑过渡。

6.2.1 `r_i` 的计算过程：从旋转圈数确定频率区间

`r_i` 不是随意指定的，而是根据每个频率在原始训练长度内旋转的圈数计算出来的。

RoPE 第 `i` 个二维旋转平面的基础频率为：

$$f_i = \text{rope_base}^{-2i/\text{dim}}$$

在原始训练长度 $L = \text{original_max_position_embeddings}$ 内，它总共旋转的圈数为：

$$\text{rotations}_i = \frac{L f_i}{2\pi}$$

YaRN 用 `beta_fast` 和 `beta_slow` 作为两个旋转圈数边界。令某个频率恰好旋转 b 圈：

$$\frac{L}{2\pi} \text{rope_base}^{-2i/\text{dim}} = b$$

解出维度索引 i：

$$i = \frac{\text{dim} \cdot \log\left(\frac{L}{b \cdot 2\pi}\right)}{2 \log(\text{rope_base})}$$

这就是代码中的 `inv_dim(b)`：

```
inv_dim = lambda b: (
    dim * math.log(orig_max / (b * 2 * math.pi))
```

```
) / (2 * math.log(rope_base))
```

然后用两个边界确定过渡区间：

```
low = max(math.floor(inv_dim(beta_fast)), 0)
high = min(math.ceil(inv_dim(beta_slow)), dim // 2 - 1)
```

通常 `beta_fast > beta_slow`，所以 `low` 位于高频区域一侧，`high` 位于低频区域一侧。对第 `i` 个频率，线性过渡系数为：

$$r_i = \text{clip}\left(\frac{i - \text{low}}{\text{high} - \text{low}}, 0, 1\right)$$

对应代码：

```
ramp = torch.clamp(
    (torch.arange(dim // 2).float() - low)
    / max(high - low, 0.001),
    0, 1
)
```

例如 `dim=8` 时有 4 个频率。若 `low=0`、`high=2`，则：

```
i       = [0, 1, 2, 3]
r_i     = [0, 0.5, 1, 1]
```

下面用 MiniMind 配置中的数值走一遍计算。设：

```
dim = 8
orig_max = 2048
rope_base = 1_000_000
beta_fast = 32
beta_slow = 1
factor = 16
```

先计算两个边界对应的维度位置：

$$\text{inv_dim}(32) = \frac{8\log(2048/(32\cdot 2\pi))}{2\log(10^6)} \approx 0.672,$$
$$\text{inv_dim}(1) = \frac{8\log(2048/(1\cdot 2\pi))}{2\log(10^6)} \approx 1.675.$$

因此：

```
low = floor(0.672) = 0
high = ceil(1.675) = 2
```

dim=8 只有 4 个频率索引 i=[0,1,2,3]，代入 r_i 公式：

```
i=0: (0-0)/(2-0) = 0   -> r_0 = 0
i=1: (1-0)/(2-0) = 0.5 -> r_1 = 0.5
i=2: (2-0)/(2-0) = 1   -> r_2 = 1
i=3: (3-0)/(2-0) = 1.5 -> 截断为 r_3 = 1
```

所以：

```
r_i = [0, 0.5, 1, 1]
```

再计算每个频率的缩放系数：

$$1 - r_i + \frac{r_i}{16} = [1,\ 0.53125,\ 0.0625,\ 0.0625]$$

如果原始频率为：

```
f_i = [1.0, 0.0316228, 0.001, 0.0000316228]
```

调整后就是：

```
f_i' = [1.0, 0.0167971, 0.0000625, 0.0000019764]
```

这说明第 0 个高频维度完全不变，第 1 个维度缩小一半左右，第 2、3 个低频维度缩小到原来的 1/16。

最后将这个系数代入频率调整公式：

$$f_i' = f_i \left[(1 - r_i) + \frac{r_i}{s}\right]$$

其中 $s = \text{factor}$。因此，高频维度（r_i=0）保留原频率，低频维度（r_i=1）缩小到原来的 1/factor，中间维度平滑过渡。

freqs的计算

假设 old_freqs=100，factor=8，分三种情况验证：

情况 1：低频维度（ramp=0）

```
new_freqs=100×(1−0+0/8)=100×1=100
```

→ 完全保留原始频率，对应 YaRN “低频不缩放，保留长程依赖” 的核心设计。

情况 2：高频维度（`ramp=1`）

```
new_freqs=100×(1−1+1/8)=100×0.125=12.5
```

→ 频率被缩放到原始的 1/8，对应 NTK 缩放策略，适应更长的上下文。

情况 3：中间维度（`ramp=0.5`）

```
new_freqs=100×(1−0.5+0.5/8)=100×(0.5+0.0625)=56.25
```

→ 混合原始频率（50）和缩放频率（6.25），实现平滑过渡，避免频率突变。

6.3 MiniMind 中的实现对应

MiniMind 在 `MiniMindConfig` 中定义了 YaRN 相关参数：

```
"beta_fast": 32,
"beta_slow": 1,
"factor": 16,
"original_max_position_embeddings": 2048,
"type": "yarn"
```

其中 `original_max_position_embeddings=2048` 表示原始训练长度，`factor=16` 表示目标长度可按 16 倍进行外推。在 `precompute_freqs_cis` 函数中，代码先计算各维度的 `ramp`，再执行：

```
freqs = freqs * (1 - ramp + ramp / factor)
```

这正对应上面的 f_i'：高频部分 `ramp=0`，保持原频率；低频部分 `ramp=1`，频率缩小为原来的 $1/\text{factor}$；中间部分则平滑过渡。这样既尽量保留短距离建模能力，又提高了模型处理长上下文的能力。

6.4 `precompute_freqs_cis`：预先计算 RoPE 的旋转表

这个函数不直接计算注意力，也不直接处理文本。它的任务是提前生成两张表：每个 token 位置、每个维度所需要的 `cos` 值和 `sin` 值。注意力层在旋转 Q、K 时直接取用这两张表。

```
def precompute_freqs_cis(dim: int, end: int = int(32 * 1024),
                         rope_base: float = 1e6, rope_scaling: dict = None):
```

```
    freqs, attn_factor = (
        1.0 / (rope_base ** (torch.arange(0, dim, 2)[:(dim // 2)].float() / dim)),
        1.0
    )
    if rope_scaling is not None:  # YaRN
        orig_max, factor, beta_fast, beta_slow, attn_factor = (
            rope_scaling.get("original_max_position_embeddings", 2048),
            rope_scaling.get("factor", 16),
            rope_scaling.get("beta_fast", 32.0),
            rope_scaling.get("beta_slow", 1.0),
            rope_scaling.get("attention_factor", 1.0)
        )
        if end / orig_max > 1.0:
            inv_dim = lambda b: (
                dim * math.log(orig_max / (b * 2 * math.pi))
            ) / (2 * math.log(rope_base))
            low = max(math.floor(inv_dim(beta_fast)), 0)
            high = min(math.ceil(inv_dim(beta_slow)), dim // 2 - 1)
            ramp = torch.clamp(
                (torch.arange(dim // 2, device=freqs.device).float() - low)
                / max(high - low, 0.001), 0, 1
            )
            freqs = freqs * (1 - ramp + ramp / factor)

    t = torch.arange(end, device=freqs.device)
    freqs = torch.outer(t, freqs).float()
    freqs_cos = torch.cat([torch.cos(freqs), torch.cos(freqs)], dim=-1) *
attn_factor
    freqs_sin = torch.cat([torch.sin(freqs), torch.sin(freqs)], dim=-1) *
attn_factor
    return freqs_cos, freqs_sin
```

6.4.1 输入参数

参数	含义
`dim`	一个注意力头的维度，例如 64。
`end`	要预先准备的位置数量，即最大 token 长度。默认是 32768。
`rope_base`	计算不同维度旋转频率时使用的基数。
`rope_scaling`	YaRN 配置；若为 `None`，则不做长度外推。

例如 `dim=8` 时，需要 `dim // 2 = 4` 个旋转频率，对应四个二维旋转平面。MiniMind 的实现采用“前半部分与后半部分配对”的写法：`rotate_half` 实际将 `[x0,x1,x2,x3]` 与 `[x4,x5,x6,x7]` 分别配对。

6.4.2 计算每组维度的基础频率

```
torch.arange(0, dim, 2)
```

当 dim=8 时，结果是 [0, 2, 4, 6]，刚好对应四个二维维度组。代码计算：

$$f_i = \frac{1}{\text{rope_base}^{2i/\text{dim}}}$$

不同维度组的 f_i 不同：高频维度旋转更快，主要表达短距离关系；低频维度旋转更慢，更适合表达长距离关系。此时变量 freqs 表示“旋转速度”。

6.4.3 YaRN 如何调整频率

```
if end / orig_max > 1.0:
```

只有目标长度 end 超过原始训练长度 orig_max 时才启用 YaRN。例如 end=32768、orig_max=2048 时，模型要处理原长度 16 倍的文本。

low、high 用于确定频率调整的边界；ramp 则为每一个维度组生成 0 到 1 的平滑系数：

```
ramp = 0：保持原频率
0 < ramp < 1：平滑过渡
ramp = 1：频率缩小为原来的 1 / factor
```

核心代码为：

```
freqs = freqs * (1 - ramp + ramp / factor)
```

即：

$$f_i' = f_i \left[(1 - \gamma_i) + \frac{\gamma_i}{s}\right]$$

其中 γ_i 是 ramp，s 是 factor。这使高频维度保留短距离能力，低频维度放慢旋转以覆盖更长的上下文，中间维度则平滑衔接。

6.4.4 attn_factor：调节注意力强度

YaRN 不只调整旋转频率，还可以通过 attn_factor 调整旋转后 Q/K 的整体幅度。代码将它同时乘到 cos 和 sin 上：

```
freqs_cos = torch.cat([torch.cos(freqs), torch.cos(freqs)], dim=-1) * attn_factor
freqs_sin = torch.cat([torch.sin(freqs), torch.sin(freqs)], dim=-1) * attn_factor
```

如果 attn_factor = a，则旋转后的向量近似为

$$q' = aR(q), \qquad k' = aR(k)$$

因此注意力点积大约按 a^2 缩放：`a > 1` 时 softmax 更尖锐，`a < 1` 时更平滑，`a = 1` 时没有额外影响。

它调节的是注意力强度（相当于温度校正），不改变相对位置对应的旋转角度。`factor` 和 `ramp` 负责改变 `freqs`，即旋转速度；`attn_factor` 负责改变旋转后 Q/K 的幅度。标准 RoPE 的旋转满足 $\cos^2\theta + \sin^2\theta = 1$，向量长度基本不变；乘上它以后则有意放大或缩小这个长度。本项目默认 `attention_factor: 1.0`，因此实际只进行了频率缩放。

6.4.5 从“转速”得到“每个位置的旋转角度”

```
t = torch.arange(end)
freqs = torch.outer(t, freqs)
```

若 `end=4`，则 `t=[0,1,2,3]`，分别表示第 0 到第 3 个 token 的位置。`torch.outer` 会把每个位置与每个频率相乘：

$$\text{angle}_{m,i} = m \times f_i$$

因此，`freqs` 在这一行之后不再只是“频率”，而是“第 m 个 token 在第 i 个二维平面上应旋转的角度”。其形状为 `[end, dim // 2]`。

6.4.6 生成 `cos`、`sin` 表并返回

二维旋转需要 $\cos(\text{angle})$ 与 $\sin(\text{angle})$，所以代码最后计算：

```
torch.cos(freqs)
torch.sin(freqs)
```

每个二维平面的角度需要应用到两个维度上，因此代码用 `torch.cat(..., dim=-1)` 将结果复制拼接，使最后一维从 `dim // 2` 变为 `dim`，可以与 Q、K 的维度对齐。结合 MiniMind 的 `rotate_half`，前半部分的维度会与后半部分对应位置的维度共同完成旋转。最终返回的 `freqs_cos`、`freqs_sin` 会被 `apply_rotary_pos_emb` 用于旋转 Q 和 K。

6.4.7 具体数值示例

为了看清每一步，假设：

```
dim = 8
end = 4
rope_base = 1_000_000
orig_max = 2_048
factor = 16
```

首先，`torch.arange(0, dim, 2)` 的结果为：

```
[0, 2, 4, 6]
```

基础频率为：

$$\left[1,\ \frac{1}{(10^6)^{0.25}},\ \frac{1}{(10^6)^{0.5}},\ \frac{1}{(10^6)^{0.75}}\right]$$

近似数值是：

```
freqs = [1.0, 0.0316228, 0.001, 0.0000316228]
```

这四个数表示四个旋转平面的“转速”：第一个每前进一个 token 转 1 弧度，最后一个每前进一个 token 仅转约 `0.0000316` 弧度。

当 `end=4` 时没有超过 `orig_max=2048`，实际代码不会启用 YaRN，频率保持不变。为了展示 YaRN 的作用，假设模型正要处理 `32768` 个 token，此时 `32768 / 2048 = 16`，会进入 YaRN 分支。在上述简化维度下，`ramp` 可近似看作：

```
ramp = [0.0, 0.5, 1.0, 1.0]
```

频率缩放系数为：

```
1 - ramp + ramp / 16 = [1.0, 0.53125, 0.0625, 0.0625]
```

因此 YaRN 调整后的频率约为：

```
freqs = [1.0, 0.0167971, 0.0000625, 0.0000019764]
```

接着以这组调整后的频率演示 `torch.outer(t, freqs)`。当 `end=4` 时，`t=[0,1,2,3]`，得到的旋转角度表约为：

```
位置 0: [0.0, 0.0,       0.0,       0.0]
位置 1: [1.0, 0.0167971, 0.0000625, 0.0000019764]
位置 2: [2.0, 0.0335942, 0.0001250, 0.0000039528]
位置 3: [3.0, 0.0503913, 0.0001875, 0.0000059292]
```

例如位置 1 的四个角度分别求余弦、正弦：

```
cos = [0.540302, 0.999859, 1.000000, 1.000000]
sin = [0.841471, 0.016796, 0.000063, 0.000002]
```

最后，`torch.cat` 会复制拼接，使它们从 4 个数变成与 `dim=8` 对齐的 8 个数：

```
freqs_cos[1] = [0.540302, 0.999859, 1.0, 1.0,
                0.540302, 0.999859, 1.0, 1.0]
freqs_sin[1] = [0.841471, 0.016796, 0.000063, 0.000002,
                0.841471, 0.016796, 0.000063, 0.000002]
```

这就是位置为 1 的 token 在旋转 Q、K 时实际要使用的系数。位置改变时，角度、`cos` 和 `sin` 都会随之改变，模型便能借此区分不同的词序和相对距离。

6.5.`apply_rotary_pos_emb` 中的维度计算

```
def apply_rotary_pos_emb(q, k, cos, sin, unsqueeze_dim=1):
    def rotate_half(x): return torch.cat((-x[..., x.shape[-1] // 2:], x[..., :
x.shape[-1] // 2]), dim=-1)
    q_embed = ((q * cos.unsqueeze(unsqueeze_dim)) + (rotate_half(q) *
sin.unsqueeze(unsqueeze_dim))).to(q.dtype)
    k_embed = ((k * cos.unsqueeze(unsqueeze_dim)) + (rotate_half(k) *
sin.unsqueeze(unsqueeze_dim))).to(k.dtype)
    return q_embed, k_embed
```

MiniMind 的代码使用 `[c0,c1,c2,c3,c0,c1,c2,c3]`，它对应的是前后半部分配对。

```
def rotate_half(x):
    return torch.cat((-x[..., x.shape[-1] // 2:],
                      x[..., : x.shape[-1] // 2]), dim=-1)
```

当 `d=8` 时，`rotate_half` 会把后半段 `-[q4,q5,q6,q7]` 放到前面，再把前半段 `[q0,q1,q2,q3]` 放到后面。因此实际旋转组是：

```
(q0, q4) 使用 θ0
(q1, q5) 使用 θ1
(q2, q6) 使用 θ2
(q3, q7) 使用 θ3
```

MiniMind 的实现将上式写成向量运算：

```
q_rot = q * cos + rotate_half(q) * sin
```

其中 cos、sin 为每个位置和每个频率预先计算的系数，rotate_half 负责组织二维旋转所需的另一坐标和符号。由于这是前后半部分配对，cos 和 sin 也会按相同顺序复制，使 q0 和 q4 使用同一个频率系数。它与前面的相邻配对只是维度排列不同，数学本质相同。

如果令 $c_i = \cos(m\theta_i)$、$s_i = \sin(m\theta_i)$，则 d=8 时：

```
q              = [q0, q1, q2, q3, q4, q5, q6, q7]
rotate_half(q) = [-q4,-q5,-q6,-q7, q0, q1, q2, q3]
cos            = [c0, c1, c2, c3, c0, c1, c2, c3]
sin            = [s0, s1, s2, s3, s0, s1, s2, s3]
```

因此：

```
q_rot[0] = q0*c0 - q4*s0       q_rot[4] = q4*c0 + q0*s0
q_rot[1] = q1*c1 - q5*s1       q_rot[5] = q5*c1 + q1*s1
q_rot[2] = q2*c2 - q6*s2       q_rot[6] = q6*c2 + q2*s2
q_rot[3] = q3*c3 - q7*s3       q_rot[7] = q7*c3 + q3*s3
```

这就明确说明了 MiniMind 的一组是 (q0,q4)，而不是 (q0,q1)。图中的“组 1”到“组 2048”表示不同的二维维度组；它们的旋转速度不同，但每一组都使用同一个二维旋转矩阵形式。

在 MiniMind 的注意力层中，Q、K 和位置系数的形状为：

```
q   : [batch, seq_len, num_heads, head_dim]
k   : [batch, seq_len, num_kv_heads, head_dim]
cos : [seq_len, head_dim]
sin : [seq_len, head_dim]
```

其中 cos、sin 是根据序列中每个位置预先计算的旋转系数。代码中的：

```
cos.unsqueeze(1)
sin.unsqueeze(1)
```

会把形状从 [seq_len, head_dim] 变为 [seq_len, 1, head_dim]。结合广播机制，它们可以与 Q 的 [batch, seq_len, num_heads, head_dim] 对齐：同一个位置的系数会应用到所有 batch 和所有 attention head，但不同位置使用不同系数。

以 head_dim=8 为例：

```
q              = [q0, q1, q2, q3, q4, q5, q6, q7]
rotate_half(q) = [-q4,-q5,-q6,-q7, q0, q1, q2, q3]
cos            = [c0, c1, c2, c3, c0, c1, c2, c3]
sin            = [s0, s1, s2, s3, s0, s1, s2, s3]
```

所以：

```
q_embed = q * cos + rotate_half(q) * sin
```

展开后，第一组 `(q0,q4)` 的结果为：

```
q_embed[0] = q0*c0 - q4*s0
q_embed[4] = q4*c0 + q0*s0
```

这就是对二维向量 `(q0,q4)` 应用旋转矩阵；其他三组 `(q1,q5)`、`(q2,q6)`、`(q3,q7)` 完全相同。`k_embed` 使用同样的 `cos/sin` 和配对方式，只是输入向量换成了 K。

最后的：

```
).to(q.dtype)
```

表示将计算结果转换回 Q 原本的数据类型（例如 `float16` 或 `bfloat16`），以控制显存和计算开销；K 则转换回 `k.dtype`。

7. GQA

7.1 为什么需要 GQA

标准多头注意力（MHA）为每个注意力头都生成一份 Q、K、V。假设有 16 个头，在自回归生成时，每生成一个 token，都要把 16 份 K/V 保存到 KV cache 中：

```
MHA: 16 个 Q 头 + 16 个 K 头 + 16 个 V 头
```

序列越长，KV cache 越大，推理时的显存占用和访存开销越高。Q 决定“当前要问什么”，需要保留多个头来表达不同的关注方式；K/V 更像被查询的上下文数据库，不一定需要同样多的副本。因此可以让多个 Q 头共享一组 K/V，这就是 GQA（Grouped Query Attention）。

7.2 GQA 的结构和公式

设 Q 头数为 H_q，K/V 头数为 H_{kv}，并且 H_q 能被 H_{kv} 整除。每组包含：

$$r = \frac{H_q}{H_{kv}}$$

个 Q 头。第 g 组的 K/V 被组内的 r 个 Q 头共同使用：

```
Q:  q0  q1 | q2  q3 | q4  q5 | q6  q7
KV: k0,v0 | k1,v1 | k2,v2 | k3,v3
```

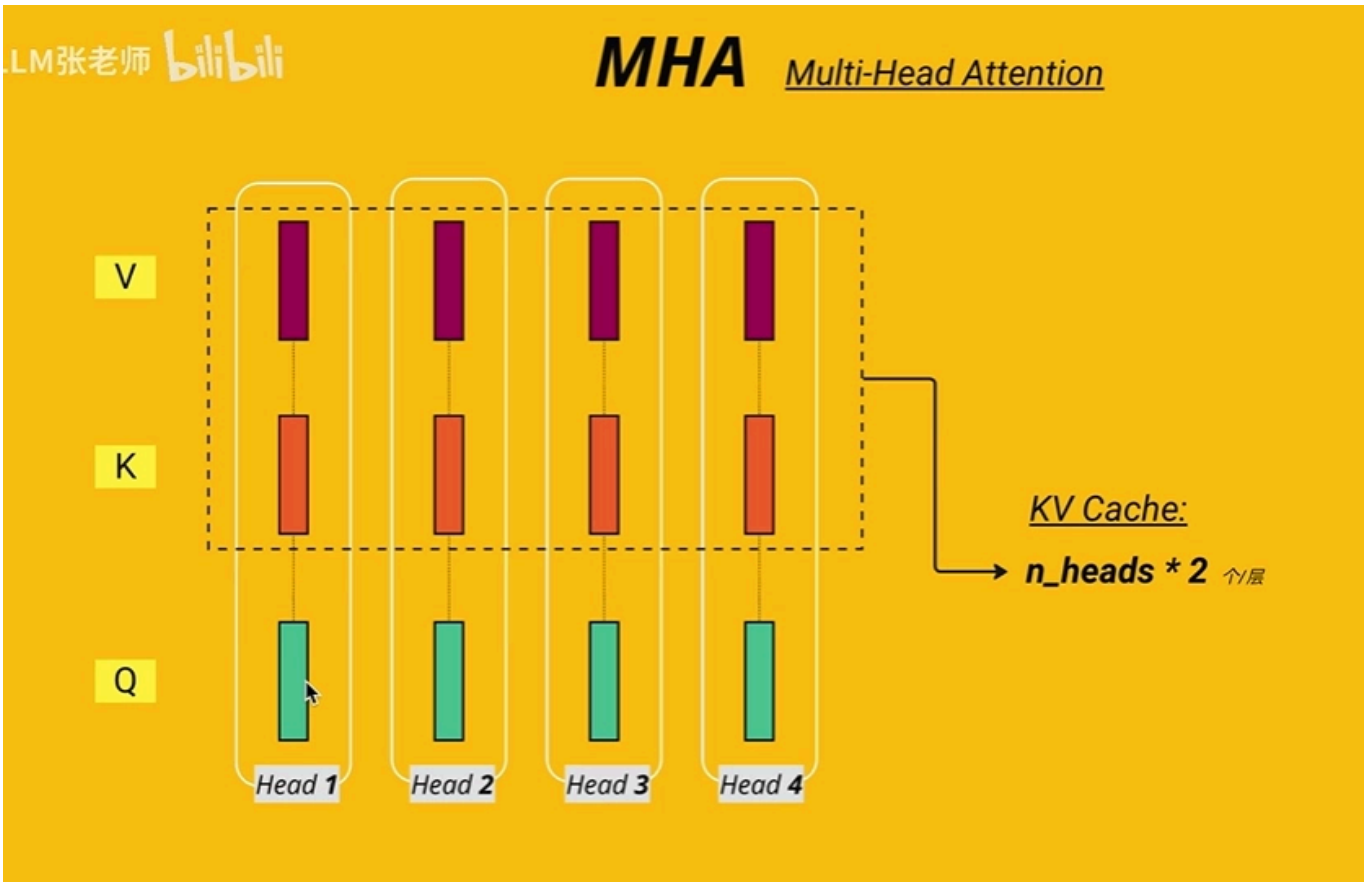
对属于第 g 组的第 j 个 Q 头，注意力计算仍然是标准公式：

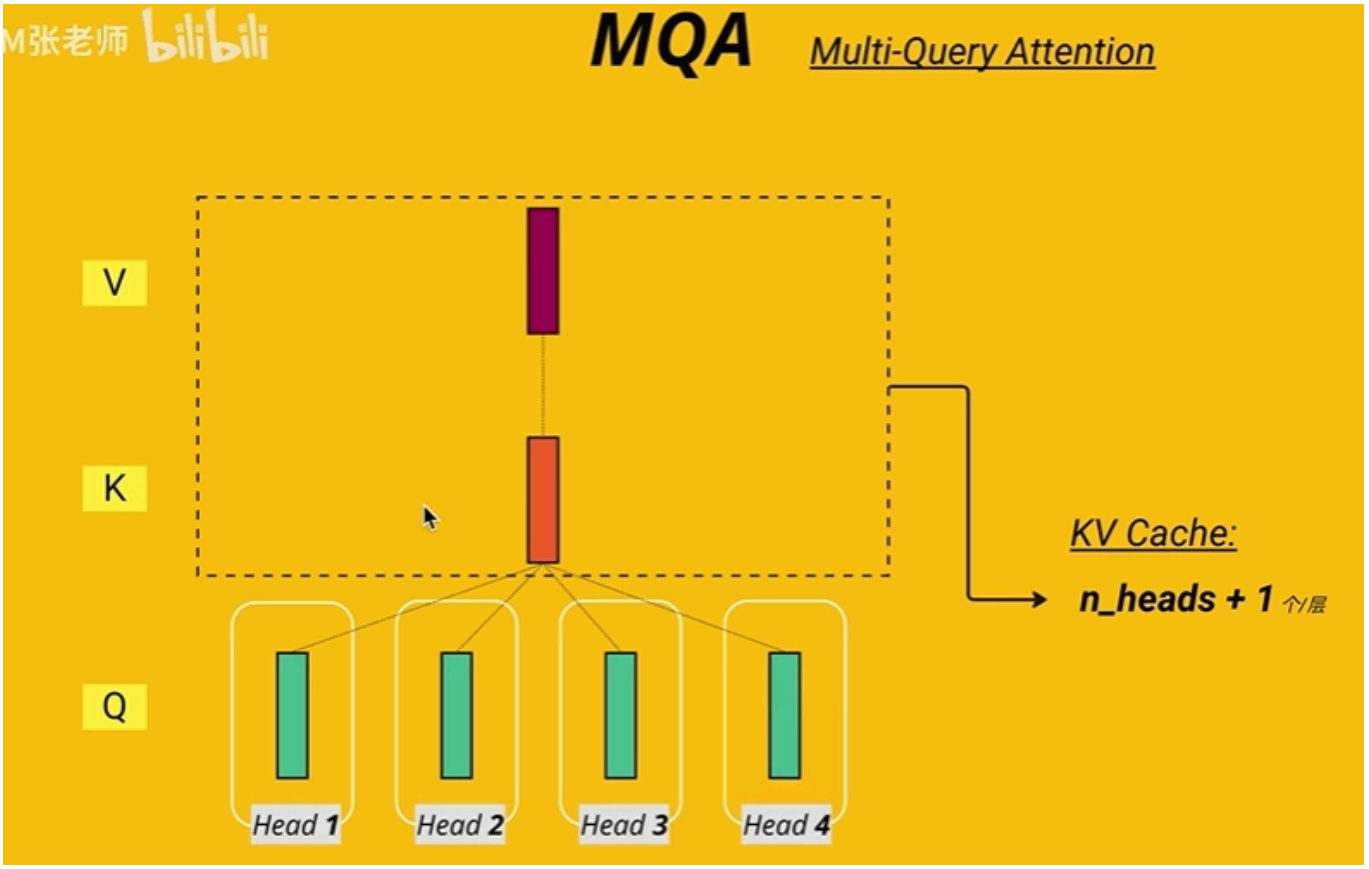
$$A_{g,j} = \mathrm{softmax}\left(\frac{Q_{g,j}K_g^{\mathsf{T}}}{\sqrt{d}} + M\right)$$

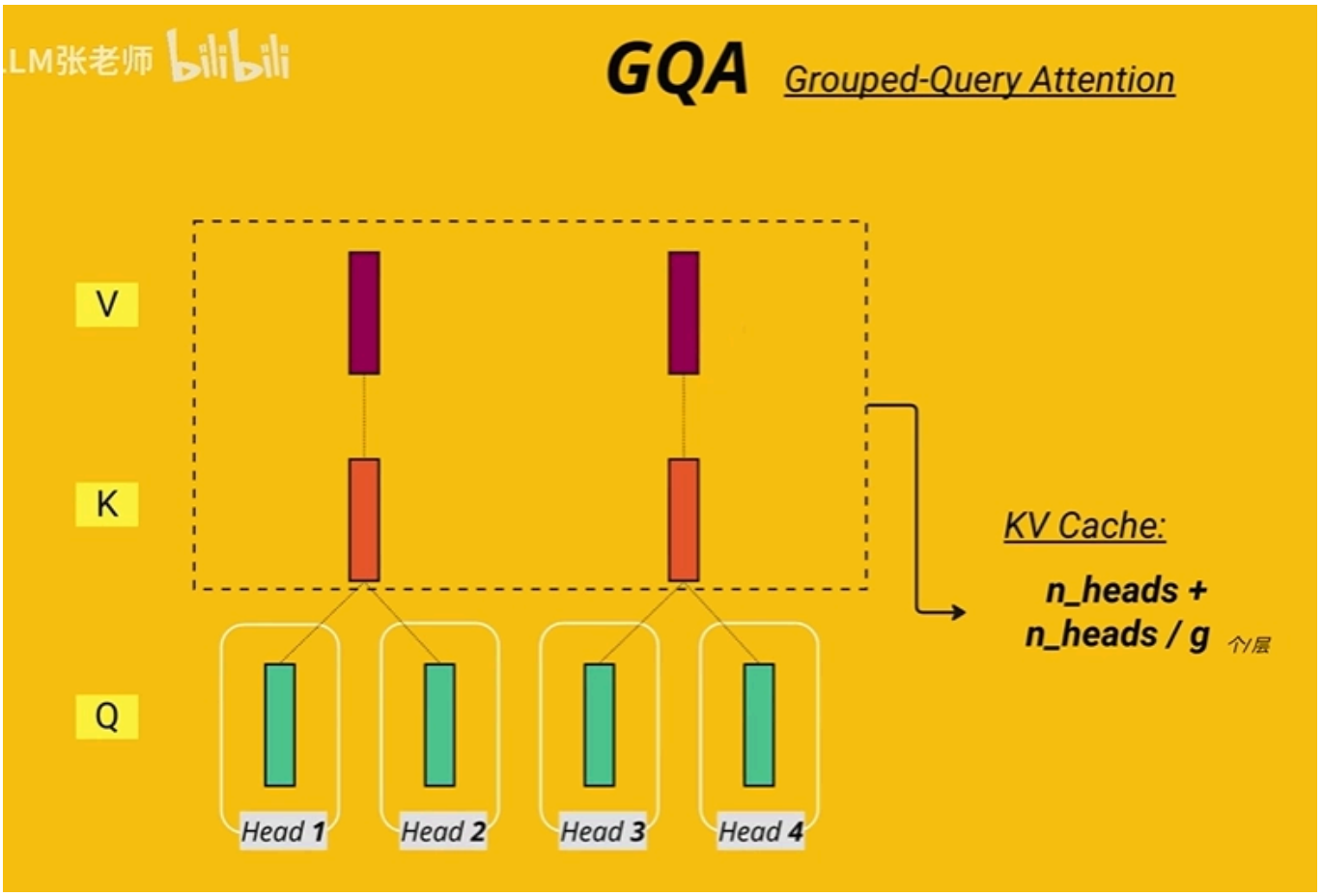
$$O_{g,j} = A_{g,j}V_g$$

其中 M 是因果 mask。不同 Q 头仍然有不同的 $Q_{g,j}$，所以它们可以关注不同内容；它们只是读取同一组 K/V 表示的上下文。

7.3 MHA、MQA 与 GQA 的关系


LM张老师 bilibili
MHA Multi-Head Attention
V
K
Q
Head 1
Head 2
Head 3
Head 4
KV Cache:
n_heads * 2 个/层


M张老师 bilibili
MQA Multi-Query Attention
V
K
Q
Head 1
Head 2
Head 3
Head 4
KV Cache:
n_heads + 1 个/层




```
MHA：Hq = Hkv，每个 Q 都有独立的 K/V
MQA：Hkv = 1，所有 Q 共享一组 K/V
GQA：1 < Hkv < Hq，多个 Q 共享一组 K/V
```

GQA 是表达能力和推理效率之间的折中：相较 MHA，它减少 KV cache；相较 MQA，它保留了多组 K/V，通常能减少精度损失。工程上常用"每组 2～8 个 Q 头"，具体比例需要结合模型规模和训练结果确定。

KV cache 的大小近似与 H_{kv} 成正比：

$$\text{显存} \propto 2 \times L \times H_{kv} \times d$$

其中 L 是缓存序列长度，d 是 head dimension，前面的 2 对应 K 和 V。因此从 MHA 的 16 个 KV 头减少到 GQA 的 4 个 KV 头，KV cache 理论上约缩小为原来的 $4/16 = 1/4$。

7.4 MiniMind 中的实现

MiniMind 设置 `num_attention_heads=8`、`num_key_value_heads=4`，所以每组有：

$$r = 8/4 = 2$$

个 Q 头。代码先分别投影出 8 组 Q 和 4 组 K/V，并以 4 组 K/V 形成 KV cache；随后 `repeat_kv(x, n_rep=2)` 将每组 K/V 对应到两个 Q 头，再进行点积注意力。这里的复制只是为了对齐计算形状，真正节省的是投影参数和生成阶段的 KV cache。

7.5 `Attention` 初始化：确定头数和投影维度

```
self.num_key_value_heads = (
    config.num_attention_heads
    if config.num_key_value_heads is None
    else config.num_key_value_heads
)
self.n_local_heads = config.num_attention_heads
self.n_local_kv_heads = self.num_key_value_heads
self.n_rep = self.n_local_heads // self.n_local_kv_heads
self.head_dim = config.head_dim
```

以 MiniMind 的配置为例：

```
num_attention_heads     = 8
num_key_value_heads     = 4
head_dim                = 96
n_rep                   = 8 / 4 = 2
```

因此每个 token 会产生 8 个 Q 头，但只产生 4 个 K 头和 4 个 V 头；每一组 K/V 被 2 个 Q 头共享。

三个输入投影层把 hidden states 映射成 Q、K、V：

```
self.q_proj = nn.Linear(hidden_size, num_attention_heads * head_dim, bias=False)
self.k_proj = nn.Linear(hidden_size, num_key_value_heads * head_dim, bias=False)
self.v_proj = nn.Linear(hidden_size, num_key_value_heads * head_dim, bias=False)
```

默认配置下输出维度是：

```
Q: 8 × 96 = 768
K: 4 × 96 = 384
V: 4 × 96 = 384
```

`o_proj` 则把 8 个 Q 头最终拼接出的 `8 × 96=768` 维向量映射回 `hidden_size=768`：

```
self.o_proj = nn.Linear(num_attention_heads * head_dim, hidden_size, bias=False)
```

7.6 `forward`：一次注意力计算的完整流程

第一步：从 hidden states 生成 Q、K、V

```
bsz, seq_len, _ = x.shape
```

```
xq, xk, xv = self.q_proj(x), self.k_proj(x), self.v_proj(x)
```

如果输入 `x` 的形状是 `[batch, seq_len, hidden_size]`，投影后仍是三维张量，但最后一维分别是 `8×head_dim`、`4×head_dim`、`4×head_dim`。接着 `view` 把最后一维拆成“头数 × 每头维度”：

```
xq = xq.view(bsz, seq_len, n_local_heads, head_dim)
xk = xk.view(bsz, seq_len, n_local_kv_heads, head_dim)
xv = xv.view(bsz, seq_len, n_local_kv_heads, head_dim)
```

形状变为：

```
xq: [batch, seq_len, 8, 96]
xk: [batch, seq_len, 4, 96]
xv: [batch, seq_len, 4, 96]
```

第二步：对每个头做归一化并应用 RoPE

```
xq, xk = self.q_norm(xq), self.k_norm(xk)
cos, sin = position_embeddings
xq, xk = apply_rotary_pos_emb(xq, xk, cos, sin)
```

Q/K 的最后一维 `head_dim` 被 RMSNorm 归一化；随后 RoPE 根据 token 的位置旋转 Q 和 K。V 不参与 RoPE，因为 V 负责携带内容，位置信息通过 Q 与 K 的匹配注入。

第三步：拼接历史 KV cache

```
if past_key_value is not None:
    xk = torch.cat([past_key_value[0], xk], dim=1)
    xv = torch.cat([past_key_value[1], xv], dim=1)
past_kv = (xk, xv) if use_cache else None
```

`past_key_value` 是这一层之前已经计算过的 `(K,V)`。`dim=1` 对应序列长度维度，所以这里只追加新 token 的 K/V，不重新计算历史 token。拼接后：

```
历史 K: [batch, old_len, 4, 96]
新 K:   [batch, new_len, 4, 96]
结果 K: [batch, old_len + new_len, 4, 96]
```

`use_cache=True` 时，拼接后的 K/V 会作为下一轮的缓存返回。

第四步：转置并扩展 GQA 的 K/V

```
xq = xq.transpose(1, 2)
xk = repeat_kv(xk, n_rep).transpose(1, 2)
xv = repeat_kv(xv, n_rep).transpose(1, 2)
```

转置前，Q/K/V 的形状来自前面的 `view` 和 KV cache 拼接：

```
Q: [batch, query_len, 8, head_dim]
K: [batch, key_len,   4, head_dim]
V: [batch, key_len,   4, head_dim]
```

这里 Q 的 `query_len` 是本轮输入的 token 数；K/V 的 `key_len` 是历史缓存加上本轮 token 数。在首次处理完整 prompt 时，二者通常相等；增量生成时，`query_len=1`，而 `key_len` 会不断增长。

`repeat_kv(x, n_rep=2)` 仍然在转置前执行。它把 K/V 的第三维从 4 个头复制成 8 个头，形状变化为：

```
K/V（扩展前）: [batch, key_len, 4, head_dim]
K/V（扩展后）: [batch, key_len, 8, head_dim]
```

然后 `transpose(1, 2)` 交换第 1、2 维，也就是交换“序列长度”和“头数”：

```
Q 转置前: [batch, query_len, 8, head_dim]
Q 转置后: [batch, 8, query_len, head_dim]

K/V 转置前: [batch, key_len, 8, head_dim]
K/V 转置后: [batch, 8, key_len, head_dim]
```

注意力计算把第 2 维当作 head 维度，把倒数两维当作矩阵乘法维度，因此需要这种排列。

转置后，注意力计算使用的形状是：

```
Q: [batch, 8, query_len, head_dim]
K: [batch, 8, key_len, head_dim]
V: [batch, 8, key_len, head_dim]
```

`repeat_kv` 将每组 K/V 复制给对应的 2 个 Q 头，使 4 个 KV 头扩展为 8 个计算头。缓存本身仍然只保存 4 个 KV 头，扩展只发生在实际计算前。

第五步：计算缩放点积注意力

源码这里不是永远走同一条路径，而是在优化实现和手写实现之间选择：

```
if self.flash and (seq_len > 1) and (not self.is_causal or past_key_value is None)
and (attention_mask is None or torch.all(attention_mask == 1)):
    output = F.scaled_dot_product_attention(
        xq, xk, xv,
        dropout_p=self.dropout if self.training else 0.0,
        is_causal=self.is_causal
    )
else:
    ...
```

只有同时满足以下条件才使用 `scaled_dot_product_attention`：开启 Flash Attention、本轮序列长度大于 1、没有会与缓存冲突的因果场景、且没有需要处理的 padding mask。训练时传入 dropout 概率，推理时传入 `0.0`。增量生成通常 `seq_len=1`，因此会进入下面的手写分支。

手写分支的源码是：

```
scores = (xq @ xk.transpose(-2, -1)) / math.sqrt(self.head_dim)
if self.is_causal:
    scores[:, :, :, -seq_len:] += torch.full((seq_len, seq_len), float("-inf"),
device=scores.device).triu(1)
if attention_mask is not None:
    scores += (1.0 - attention_mask.unsqueeze(1).unsqueeze(2)) * -1e9
output = self.attn_dropout(F.softmax(scores.float(), dim=-1).type_as(xq)) @ xv
```

下面逐行看它如何改变矩阵。假设经过前面的转置后：

```
xq: [batch, heads, query_len, head_dim]
xk: [batch, heads, key_len,   head_dim]
xv: [batch, heads, key_len,   head_dim]
```

1. Q 与 K 做矩阵乘法

```
scores = xq @ xk.transpose(-2, -1)
```

`xk.transpose(-2, -1)` 只交换最后两个维度：

```
xk:                     [batch, heads, key_len, head_dim]
xk.transpose(-2, -1):   [batch, heads, head_dim, key_len]
```

因此最后两维可以相乘：

```
[query_len, head_dim] @ [head_dim, key_len]
                    = [query_len, key_len]
```

整个 `scores` 的形状是：

```
scores: [batch, heads, query_len, key_len]
```

其中 `scores[..., i, j]` 表示第 `i` 个 query 对第 `j` 个 key 的相关性。它还不是概率，只是未经 softmax 的分数。

2. 除以 `sqrt(head_dim)`

```
scores = scores / math.sqrt(self.head_dim)
```

每个分数都除以 $\sqrt{d}$。如果 `head_dim=96`，除数就是 $\sqrt{96}$。这样可以控制点积的数值范围，避免分数过大后 softmax 几乎变成 one-hot，导致梯度和训练不稳定。

3. `is_causal`：是否使用因果注意力

```
self.is_causal = True
```

`is_causal` 是一个布尔值：

```
True  ：当前位置只能看自己和过去，不能看未来
False ：不使用这种时间顺序限制，可以看全部 key
```

语言模型必须使用因果注意力，因为生成第 3 个 token 时不能提前看到真实答案中的第 4 个 token。否则训练时模型会“偷看未来”，推理时却没有未来信息，训练和推理会不一致。

在手写分支中，它控制是否执行下面的代码：

```
if self.is_causal:
    scores[:, :, :, -seq_len:] += causal_mask
```

也就是说，`False` 时不添加上三角 `-inf`，每个 query 都可以读取所有 key。在 Flash Attention 分支中，它作为参数直接交给 PyTorch：

```
F.scaled_dot_product_attention(
    xq, xk, xv,
    is_causal=self.is_causal
)
```

此时由 PyTorch 内部完成同样的因果屏蔽，不需要我们手动创建 mask。

`is_causal` 和 `attention_mask` 不是一回事：

```
is_causal      : 限制时间方向，屏蔽未来 token
attention_mask : 限制哪些位置有效，通常用于屏蔽 padding
```

二者可以同时使用。例如一个长度为 5 的序列最后两个位置是 padding，那么有效位置既要满足“不是未来”，也要满足“不是 padding”。

4. `torch.full` 创建 mask 矩阵

```
torch.full((seq_len, seq_len), float("-inf"), device=scores.device)
```

`torch.full(shape, value)` 会创建一个指定形状、所有元素都相同的张量。比如：

```
torch.full((3, 3), float("-inf"))
```

得到：

```
[[-∞, -∞, -∞],
 [-∞, -∞, -∞],
 [-∞, -∞, -∞]]
```

`device=scores.device` 表示把这个 mask 放到和 `scores` 相同的 CPU/GPU 设备上，避免设备不一致报错。

5. `.triu(1)` 保留上三角

```
torch.full((3, 3), -inf).triu(1)
```

`.triu(k)` 返回矩阵的上三角部分，主对角线以下全部置零；参数 `1` 表示从主对角线往上偏移一格开始保留。因此结果是：

```
[[ 0, -∞, -∞],
 [ 0,  0, -∞],
 [ 0,  0,  0]]
```

这就是因果 mask：第 0 个 token 不能看第 1、2 个 token；第 1 个 token 不能看第 2 个 token；第 2 个 token 可以看全部当前位置及之前的位置。

6. 把因果 mask 加到 `scores`

```
scores[:, :, :, -seq_len:] += mask
```

假设没有 KV cache，`seq_len=3`，此时：

```
scores: [batch, heads, 3, 3]
mask:   [3, 3]
```

`[3,3]` 的 mask 会通过广播加到每个 batch、每个 head 的 score 矩阵上。被加上 `-∞` 的位置在 softmax 后概率为 0，于是未来 token 不会被看到。

有 KV cache 时，情况更容易混淆。假设历史长度为 4，本轮新 token 数为 2：

```
xq:    [batch, heads, 2, head_dim]
xk:    [batch, heads, 6, head_dim]
scores:[batch, heads, 2, 6]
```

此时 `-seq_len:` 只选中最后两个 key 位置：

```
scores[:, :, :, -2:]: [batch, heads, 2, 2]
mask:                 [2, 2]
```

mask 仍然是：

```
[[ 0, -∞],
 [ 0,  0]]
```

前 4 个历史 key 没有被屏蔽，因为新 token 可以读取全部历史；mask 只限制本轮两个新 token 不能偷看彼此后面的 token。

7. `attention_mask` 屏蔽 padding

`attention_mask` 是由外部传入的有效位置标记，通常每个 token 对应一个值：

```
1：这个位置是真实 token，可以被注意
0：这个位置是 padding，不应该被注意
```

例如为了把不同长度的句子补齐成相同长度：

```
token           我   喜   欢   <pad>
attention_mask  1    1    1     0
```

它解决的是“哪些位置根本不存在”的问题；`is_causal` 解决的是“真实位置之间能不能看未来”的问题。一个位置可能不是未来，但仍然是 padding，因此两种 mask 需要分别处理。

```
if attention_mask is not None:
    scores += (1.0 - attention_mask.unsqueeze(1).unsqueeze(2)) * -1e9
```

通常 `attention_mask` 的形状是 `[batch, key_len]`，有效 token 为 1，padding 为 0。例如：

```
attention_mask = [1, 1, 1, 0]
```

两次 `unsqueeze` 后：

```
[batch, key_len]
 -> [batch, 1, key_len]
 -> [batch, 1, 1, key_len]
```

`1.0 - attention_mask` 会把有效位置变成 0、padding 位置变成 1；乘以 `-1e9` 后，padding 的 score 会被减去一个极大值。它和因果 mask 通过广播相加，共同决定哪些 key 可以参与注意力。

例如 `key_len=4` 时，某个 query 的原始分数为：

```
scores = [2.1, 0.7, 1.3, 4.0]
mask   = [  0,   0,   0, -1e9]
结果   = [2.1, 0.7, 1.3, -1e9]
```

softmax 后最后一个 padding 位置的概率约等于 0，所以它对应的 `V` 不会参与输出。注意，`attention_mask` 是沿 `key_len` 维度屏蔽被查询的 key；它不是删除 Q，也不会改变 Q 的数量。

在生成函数中，每生成一个新 token，代码会把 attention mask 追加一个 1：

```
attention_mask = torch.cat(
    [attention_mask, attention_mask.new_ones(attention_mask.shape[0], 1)],
    dim=-1
)
```

这是因为新生成的 token 是有效位置，下一轮它也应该成为可被后续 token 查询的 key。如果原始输入是 `[1,1,1,0]`，生成一个新 token 后就变成 `[1,1,1,0,1]`；原来的 padding 仍然被屏蔽，新 token 则可以参与后续注意力。

8. softmax、dropout 与 V 的加权求和

```
probs = F.softmax(scores.float(), dim=-1).type_as(xq)
output = self.attn_dropout(probs) @ xv
```

`dim=-1` 表示沿着 `key_len` 这一维做 softmax：每个 query 对所有 key 的分数被转换成概率，并且每一行概率和为 1。被设为 `-∞` 或 `-1e9` 的位置概率接近 0。

形状变化为：

```
scores: [batch, heads, query_len, key_len]
probs:  [batch, heads, query_len, key_len]
xv:     [batch, heads, key_len, head_dim]
output: [batch, heads, query_len, head_dim]
```

最后一个矩阵乘法本质上是：每个 query 用自己的注意力概率，对所有 V 做加权平均，得到该 query 的输出向量。`scores.float()` 先用更高精度计算 softmax，`.type_as(xq)` 再转回 Q 的数据类型；`attn_dropout` 只在训练时随机丢弃部分注意力权重，推理时通常关闭。

对应公式：

$$S = \frac{QK^{\mathsf{T}}}{\sqrt{d}} + M$$

$$P = \mathrm{softmax}(S)$$

$$O = PV$$

除以 $\sqrt{d}$ 是为了避免 head dimension 较大时点积数值过大，导致 softmax 过于尖锐。因果 mask 的上三角被设为 $-\infty$，保证当前位置不能看到未来 token。代码只对最后 `seq_len` 个 key 位置加这个 `[seq_len, seq_len]` mask：使用 KV cache 时，前面的历史 key 都允许被当前 token 读取，最后这段 mask 只约束本轮新 token 之间的未来关系。

`attention_mask` 会通过 `unsqueeze(1).unsqueeze(2)` 变成可广播的形状，并把被屏蔽位置的分数减去约 `1e9`，softmax 后这些位置的权重接近 0。手写分支先用 `scores.float()` 做 softmax，再转换回 Q 的 dtype，最后应用 `attn_dropout`。两条路径的数学目标相同，但 Flash 路径由 PyTorch 内核完成，手写路径把每一步显式写出。

第六步：拼接所有头并映射回 hidden size

```
output = output.transpose(1, 2).reshape(bsz, seq_len, -1)
output = self.o_proj(output)
return output, past_kv
```

注意力输出先从 `[batch, heads, seq_len, head_dim]` 转回 `[batch, seq_len, heads, head_dim]`，再把所有头拼接成 `[batch, seq_len, heads × head_dim]`，最后经过 `o_proj` 回到 `[batch, seq_len, hidden_size]`。返回的第二项 `past_kv` 会交给下一次生成使用。

8. FFN：为什么需要以及如何实现

8.1 作用/为什么需要

1. 提供非线性能力（最根本作用）

- 问题：自注意力机制本质是加权求和，属于线性操作
- FFN的解决方案：通过激活函数（ReLU、GELU等）引入非线性
- 重要性：没有非线性，无论堆叠多少层Transformer，整体仍等价于单层线性模型，无法学习复杂模式

2. 对信息进行深度加工

- 多头注意力的角色：让词语之间"开会讨论"，收集全局信息
- FFN的角色：让每个词语"独立深度思考"，消化和提炼信息
- 工作方式：对注意力层输出的每个位置表示进行独立、并行的深度处理

3. 存储模型知识

- 参数量对比：在标准Transformer中，FFN的参数量通常占全层参数的2/3以上
- 作用：如同模型的"长期记忆"，存储从训练数据中学到的各种模式和规律
- 工作方式：通过大量参数学习特征之间的复杂组合关系

8.2 原理

8.2.1 标准 FFN 的数学表达

标准的两层前馈网络可以写成：

$$\mathrm{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

其中：

符号	形状	含义
x	(N, d_{model})	输入序列的 token 表示，N 是 token 数量
W_1	(d_{model}, d_{ff})	第一层权重，把维度从 d_{model} 升到 d_{ff}
b_1	(d_{ff})	第一层偏置

符号	形状	含义
σ	-	非线性激活函数，例如 ReLU、GELU 或 SiLU
W_2	(d_{ff}, d_{model})	第二层权重，把维度从 d_{ff} 压回 d_{model}
b_2	(d_{model})	第二层偏置

因此，数据流的维度变化是：

阶段	维度	说明
输入	(N, d_{model})	原始表示空间
升维	(N, d_{ff})	在更大的特征空间中展开特征
激活	(N, d_{ff})	进行逐元素非线性变换
降维	(N, d_{model})	压回 Transformer 的隐藏维度

例如 $d_{model} = 4096$、$d_{ff} = 4d_{model} = 16384$ 时：

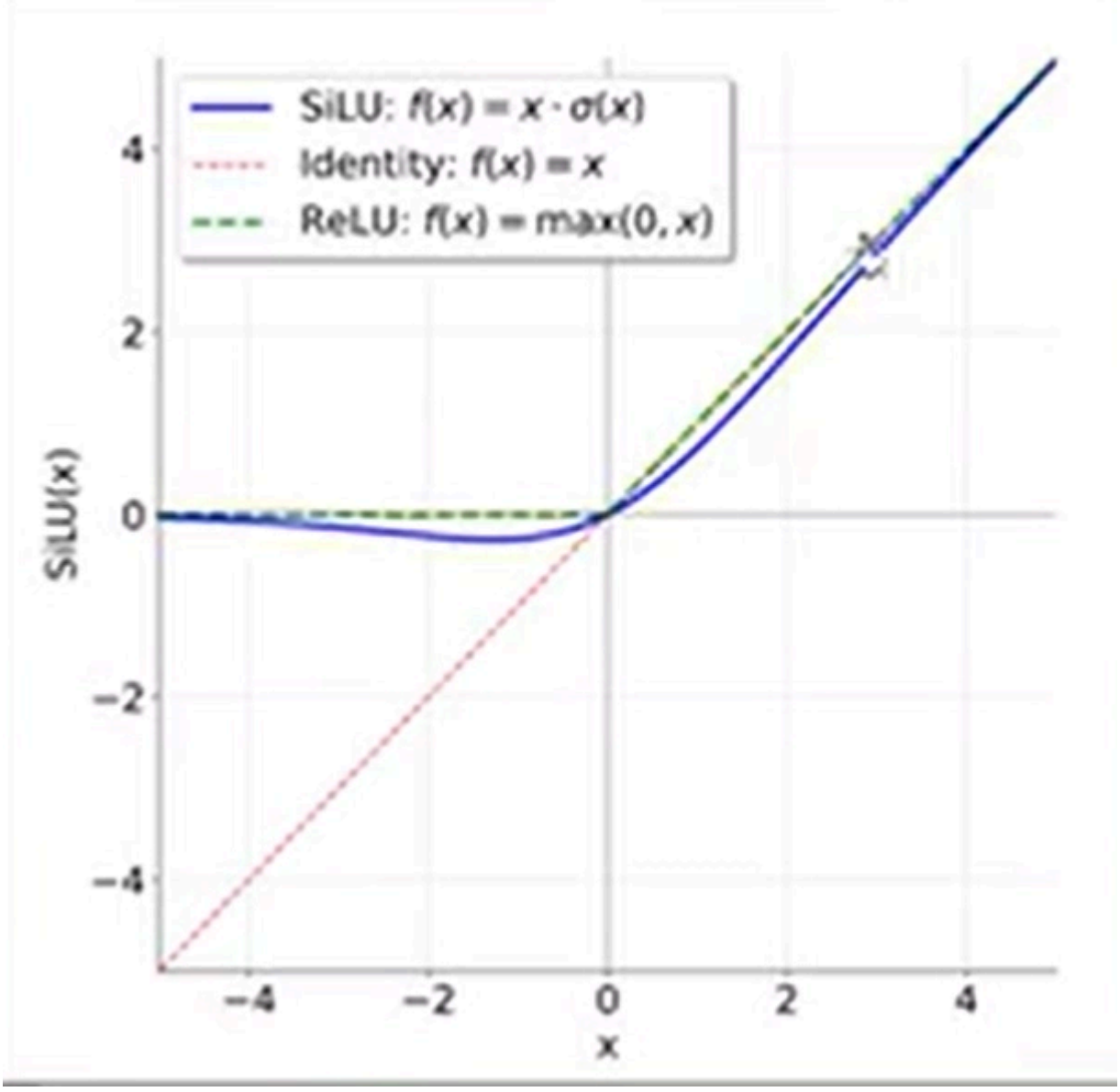
```
输入：   (N, 4096)   原始表示空间
升维：   (N, 16384)  高维展开空间
激活：   (N, 16384)  非线性变换
降维：   (N, 4096)   压缩回原始维度
```

升维和降维是 FFN 的核心结构：如果只有线性层，前后多个线性变换仍可以合并成一个线性变换；中间加入激活函数后，网络才能学习复杂的非线性特征组合。现代大语言模型通常省略 b_1, b_2，因此代码里的 `nn.Linear(..., bias=False)` 与这一做法一致。

8.2.2 激活函数：Swish/SiLU

MiniMind 默认使用 `SiLU`，它是 `Swish` 在 $\beta = 1$ 时的特殊情况。一般的 Swish 定义为：

$$\text{Swish}_\beta(x) = x \cdot \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$



其中 σ 是 sigmoid 函数，β 可以是固定值，也可以是可学习参数。当 $\beta = 1$ 时：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x)$$

直观上，sigmoid 输出 0 到 1 之间的门控值，再乘以输入 x，因此 Swish 具有"用输入控制自身通过多少信息"的自门控效果：

```
x 很大且为正：sigmoid(βx) 接近 1，Swish(x) 接近 x
x 很大且为负：sigmoid(βx) 接近 0，Swish(x) 接近 0
x 接近 0：输出也接近 0，但整体变化平滑
```

以 $\beta = 1$ 的 SiLU 为例，它的输出范围约为 $[-0.278, +\infty)$。它在负半轴有一个很小的下凹区域，因此不是单调函数；但它处处连续且可微，零点附近没有 ReLU 那样的折角。

与常见激活函数相比：

特性	ReLU	GELU	Swish/SiLU
公式	$\max(0, x)$	$x\Phi(x)$	$x\sigma(x)$

特性	ReLU	GELU	Swish/SiLU
输出范围	$[0, +\infty)$	约 $[-0.17, +\infty)$	约 $[-0.278, +\infty)$
零点处可微	否	是	是
单调性	是	近似单调	否
负值输出	无	极小	较小
神经元死亡风险	有	无	无

Swish/SiLU 的平滑性使梯度在整个定义域中连续变化，通常比 ReLU 更适合深层网络优化；代价是计算 sigmoid 的开销高于 ReLU。MiniMind 通过：

```
self.act_fn = ACT2FN[config.hidden_act]
```

根据配置选择激活函数。默认 `config.hidden_act="silu"`，因此 `self.act_fn` 就是 SiLU，并在门控分支中执行 `self.act_fn(self.gate_proj(x))`。

`SiLU` 的公式是：

$$\mathrm{SiLU}(z) = z \cdot \sigma(z)$$

它给 gate 分支加入平滑非线性。逐元素相乘实现门控：gate 接近 0 时抑制对应特征，gate 较大时保留或增强对应特征。这种结构是带门控的 SwiGLU 风格 FFN。

降维之后，FFN 输出在 `MiniMindBlock` 中与残差相加：

```
hidden_states = hidden_states + self.mlp(
    self.post_attention_layernorm(hidden_states)
)
```

图片中的 Dropout 是常见 FFN 结构的一部分，但当前 `FeedForward` 实现没有在 `down_proj` 后显式调用 Dropout；当前代码直接把 `down_proj` 的结果交给残差相加。

8.3 MiniMind代码

MiniMind 使用的不是最简单的单分支标准 FFN，而是带门控的 SwiGLU 风格结构：

$$g = \mathrm{SiLU}(xW_g), \qquad u = xW_u$$

$$h = g \odot u, \qquad F(x) = hW_d$$

其中 `gate_proj` 和 `up_proj` 都把输入从 d_{model} 升到 d_{ff}；`gate_proj` 的结果经过激活函数后作为门，和 `up_proj` 的特征逐元素相乘；`down_proj` 再把结果从 d_{ff} 降回 d_{model}。它可以看作是在标准 FFN 的升维阶段增加了一条并行分支，用门控控制哪些特征被保留或增强。

MiniMind 的完整 FFN 代码为：

```
class FeedForward(nn.Module):
    def __init__(self, config: MiniMindConfig, intermediate_size: int = None):
        super().__init__()
        intermediate_size = intermediate_size or config.intermediate_size
        self.gate_proj = nn.Linear(config.hidden_size, intermediate_size,
bias=False)
        self.down_proj = nn.Linear(intermediate_size, config.hidden_size,
bias=False)
        self.up_proj = nn.Linear(config.hidden_size, intermediate_size,
bias=False)
        self.act_fn = ACT2FN[config.hidden_act]

    def forward(self, x):
        return self.down_proj(self.act_fn(self.gate_proj(x)) * self.up_proj(x))
```

初始化部分逐行含义如下：

```
class FeedForward(nn.Module):
```

定义一个继承自 PyTorch `nn.Module` 的 FFN 网络模块。

```
def __init__(self, config: MiniMindConfig, intermediate_size: int = None):
    super().__init__()
```

接收模型配置；调用父类初始化，使这个模块中的参数能够被 PyTorch 注册和训练。`intermediate_size` 可以由外部传入，若不传则使用配置中的值：

```
intermediate_size = intermediate_size or config.intermediate_size
```

```
self.gate_proj = nn.Linear(config.hidden_size, intermediate_size, bias=False)
```

门控分支，把每个 token 从 `hidden_size` 维映射到 `intermediate_size` 维。它的输出之后会经过 SiLU，作为门控信号。

```
self.up_proj = nn.Linear(config.hidden_size, intermediate_size, bias=False)
```

另一条并行分支，同样把输入升维到 `intermediate_size`，负责提供被门控的特征内容。

```
self.down_proj = nn.Linear(intermediate_size, config.hidden_size, bias=False)
```

把门控相乘后的中间表示降回 `hidden_size`，这样才能和 Transformer block 的残差分支逐元素相加。

```
self.act_fn = ACT2FN[config.hidden_act]
```

根据配置选择激活函数。MiniMind 默认 `hidden_act='silu'`，所以这里得到的是 SiLU 函数。三个 Linear 都设置 `bias=False`，表示只学习权重矩阵，不额外学习偏置向量。

令输入 `x` 的形状为 `[batch, seq_len, hidden_size]`，对应公式：

$$g = \mathrm{SiLU}(xW_g), \qquad u = xW_u$$

$$h = g \odot u, \qquad F(x) = hW_d$$

其中 `gate_proj` 和 `up_proj` 是两条并行的升维分支，`down_proj` 将中间结果降回 `hidden_size`，$\odot$ 是逐元素相乘。

按图片逐步看：

```
RMSNorm:
    [batch, seq_len, hidden_size]

gate_proj(x), up_proj(x):
    [batch, seq_len, hidden_size]
        → [batch, seq_len, intermediate_size]

SiLU(gate_proj(x)) * up_proj(x):
    [batch, seq_len, intermediate_size]
    ⊙ [batch, seq_len, intermediate_size]
    = [batch, seq_len, intermediate_size]

down_proj(...):
    [batch, seq_len, intermediate_size]
        → [batch, seq_len, hidden_size]
```

8.4 MoE：混合专家前馈网络

8.4.1 为什么需要 MoE

普通 Dense FFN 使用同一套参数处理每个 token。增大 FFN 通常会同时增加参数容量和每个 token 的计算量。MoE（Mixture of Experts）准备多个独立 FFN，由路由器为每个 token 选择其中少数几个执行，从而增加总参数容量，同时让单个 token 只激活部分专家参数。

```
普通 FFN:
token → 同一个 FFN → 输出
```

```
MoE：
token → 路由器 gate → 选择 K 个专家 FFN → 加权合并输出
```

例如一共有 8 个专家，每个 token 只选 2 个，并不需要让它经过全部 8 个 FFN。但所有专家的参数仍需存储，还存在路由和分发开销，所以 MoE 不意味着显存占用按激活比例减少，也不保证实际运行一定更快。

本项目在 Transformer block 中通过配置选择：

```python
self.mlp = FeedForward(config) if not config.use_moe else MOEFeedForward(config)
```

MoE 输入与输出的隐藏维度相同，可以放在普通 FFN 的位置，与原有残差连接配合。这里的"专家"是可训练的 FFN，并不是人工指定的数学专家、写作专家；是否形成某种分工取决于训练。

8.4.2 原理

路由、稀疏选择与加权合并。

设 token 向量为 $x_t \in \mathbb{R}^H$，共有 E 个专家，每个 token 选择 K 个。路由器先计算专家概率：

$$s_t = \mathrm{softmax}(W_{\mathrm{gate}} x_t), \qquad s_t \in \mathbb{R}^E$$

再取概率最高的 K 个专家，组成集合 $\mathcal{I}_t$。若开启入选权重归一化：

$$w_{t,e} = \frac{s_{t,e}}{\sum_{j \in \mathcal{I}_t} s_{t,j} + \varepsilon}, \qquad e \in \mathcal{I}_t$$

否则直接使用 $w_{t,e} = s_{t,e}$。最终输出是：

$$y_t = \sum_{e \in \mathcal{I}_t} w_{t,e} E_e(x_t)$$

其中 E_e 表示第 e 个专家 FFN。不要混淆 MoE 的路由器 `gate` 与普通 FFN 内部的 `gate_proj`：前者选择专家，后者参与单个 FFN 的门控特征计算。

固定以下小例子，贯穿后续代码：

```
batch_size = 1
seq_len = 3                  共 3 个 token
hidden_dim = 4               每个 token 是 4 维向量
num_experts = 3              共有 3 个专家
num_experts_per_tok = 2      每个 token 选 2 个专家

路由概率：        专家0  专家1  专家2
token0:        [0.6,  0.3,  0.1]
token1:        [0.1,  0.7,  0.2]
token2:        [0.4,  0.1,  0.5]

入选结果：
token0 → 专家0、专家1
```

```
token1 → 专家1、专家2
token2 → 专家2、专家0
```

若开启归一化，输出近似为：

```
y0 = 0.667 × 专家0(x0) + 0.333 × 专家1(x0)
y1 = 0.778 × 专家1(x1) + 0.222 × 专家2(x1)
y2 = 0.556 × 专家2(x2) + 0.444 × 专家0(x2)
```

为什么还需要负载均衡辅助损失?

路由器可能长期将多数 token 分给少数专家，其他专家很少获得有效训练。`aux_loss` 鼓励更均衡的专家使用，它不直接评价答案对错。

设展平后的 token 数为 N，平均路由概率为：

$$p_e = \frac{1}{N}\sum_{t=1}^{N} s_{t,e}$$

本实现的专家选择频率可以按全部 top-k 槽位合并为：

$$f_e = \frac{1}{N}\sum_{t=1}^{N}\sum_{k=1}^{K}\mathbf{1}[\text{topk_idx}_{t,k} = e]$$

由于同一 token 的 top-k 专家索引互不重复，f_e 也就是选择专家 e 的 token 比例，并且 $\sum_e f_e = K$。本代码使用：

$$L_{\text{aux}} = \lambda E \sum_e f_e p_e$$

其中 λ 是 `router_aux_loss_coef`。这份实现没有再除以 K，比较其他 MoE 实现时需要注意尺度差别。辅助损失在均衡状态下通常不为零；它是促进均衡的优化信号，不是每一步都严格保证专家使用相等的约束。

离散 top-k 索引和选择计数不提供可微的选择路径；辅助梯度通过路由概率 p_e 传播。主任务损失则通过已选专家输出和路由权重传播。在未选中专家上，不会凭空产生来自该 token 的有效专家计算梯度。

8.4.3 代码实现

对应源码：model_minimind.py 中的 `MOEFeedForward`。按"初始化 → 展平 → 路由 → 分发 → 合并 → 辅助损失 → 恢复形状"阅读。

第一步：创建路由器和多个独立专家。

```python
self.gate = nn.Linear(config.hidden_size, config.num_experts, bias=False)
self.experts = nn.ModuleList([
    FeedForward(config, intermediate_size=config.moe_intermediate_size)
    for _ in range(config.num_experts)
```

```
])
self.act_fn = ACT2FN[config.hidden_act]
```

`gate` 将每个 token 的 H 维向量变成 E 个专家分数。`ModuleList` 注册多个独立 FFN，使它们的参数能随父模型保存、移动设备和参与优化。各专家结构相同，但参数不同。

本类的 `self.act_fn` 在所示 `forward` 中没有使用，专家内部激活由 `FeedForward` 自己处理。

第二步：展平批次与序列维度。

```
batch_size, seq_len, hidden_dim = x.shape
x_flat = x.view(-1, hidden_dim)
```

```
x:       [B, L, H] = [1, 3, 4]
                ↓ view
x_flat:  [N, H]    = [3, 4], N = B × L

token0: [……, ……, ……, ……]
token1: [……, ……, ……, ……]
token2: [……, ……, ……, ……]
```

这一步不把 token 向量混合起来，只改变张量形状。

第三步：计算路由概率，选出 top-k。

```
scores = F.softmax(self.gate(x_flat), dim=-1)
topk_weight, topk_idx = torch.topk(
    scores, k=self.config.num_experts_per_tok,
    dim=-1, sorted=False
)
```

```
scores [3, 3]:
[[0.6, 0.3, 0.1],
 [0.1, 0.7, 0.2],
 [0.4, 0.1, 0.5]]

一种可能的返回顺序：
topk_idx [3, 2]:      topk_weight [3, 2]:
[[0, 1],             [[0.6, 0.3],
 [1, 2],              [0.7, 0.2],
 [2, 0]]              [0.5, 0.4]]
```

`sorted=False` 不保证返回顺序按分数排列，但同位置的专家编号与权重始终对应。后文采用上面的排列举例。

```
if self.config.norm_topk_prob:
    topk_weight = topk_weight / (
        topk_weight.sum(dim=-1, keepdim=True) + 1e-20
    )
```

```
token0: [0.6, 0.3] / 0.9 → [0.667, 0.333]
token1: [0.7, 0.2] / 0.9 → [0.778, 0.222]
token2: [0.5, 0.4] / 0.9 → [0.556, 0.444]
```

`keepdim=True` 使分母形状为 `[N, 1]`，通过广播除以每一行。入选权重归一化后每行和为 1；完整 `scores` 没有被这一步修改，后面辅助损失仍用原始完整概率。

第四步：逐专家找出分给自己的 token。

```
y = torch.zeros_like(x_flat)
for i, expert in enumerate(self.experts):
    mask = (topk_idx == i)
    if mask.any():
        token_idx = mask.any(dim=-1).nonzero().flatten()
        weight = topk_weight[mask].view(-1, 1)
        ...
```

以专家0，即 `i=0` 为例：

```
topk_idx:        mask = (topk_idx == 0):
[[0, 1],         [[True,  False],
 [1, 2],          [False, False],
 [2, 0]]          [False, True ]]

mask.any(dim=-1) → [True, False, True]
.nonzero()       → [[0], [2]]
.flatten()       → [0, 2]
```

因此 `token_idx=[0, 2]`，专家0需要处理 token0 和 token2。这里外层循环遍历专家，而不是逐 token 调用所有专家。

取出对应权重：

```
topk_weight:
[[0.667, 0.333],
 [0.778, 0.222],
 [0.556, 0.444]]
        ↓ 用同样形状的布尔 mask 选取
[0.667, 0.444]
```

```
        ↓ view(-1, 1)
[[0.667],
 [0.444]]                形状 [2, 1]
```

同一个 token 不会在 top-k 中重复选择专家0，因此这些权重与 `token_idx` 的行顺序对齐。

单独展开 `weight = topk_weight[mask].view(-1, 1)`。

这行先选出当前专家对应的权重，再将它们排成一列。当前处理专家0时，编号、权重与布尔 mask 的对应关系是：

```
topk_idx:        topk_weight:           mask = (topk_idx == 0):
[[0, 1],        [[0.667, 0.333],        [[True,  False],
 [1, 2],         [0.778, 0.222],         [False, False],
 [2, 0]]         [0.556, 0.444]]         [False, True ]]
```

`topk_weight[mask]` 是布尔索引，只取 `True` 位置的元素，不是取整行：

```
token0 第一个槽位：选取 0.667
token1 两个槽位：   都不选
token2 第二个槽位：选取 0.444

topk_weight[mask] → [0.667, 0.444]，形状 [2]
```

接着 `.view(-1, 1)` 指定一列，让 PyTorch 自动推算行数；两个元素排成两行：

```
[0.667, 0.444]       [2]
        ↓ view(-1, 1)
[[0.667],
 [0.444]]            [2, 1]
```

为什么需要这一列？因为每个权重应该乘到对应 token 的整个向量上。假设专家0处理 token0 和 token2 后输出：

```
expert_output:        weight:
[[1, 2, 3, 4],        [[0.667],
 [5, 6, 7, 8]]         [0.444]]

形状 [2, 4]           形状 [2, 1]
```

相乘时，大小为 1 的最后一维通过广播匹配四个特征：

```
[[1, 2, 3, 4],       [[0.667, 0.667, 0.667, 0.667],
 [5, 6, 7, 8]]   ×    [0.444, 0.444, 0.444, 0.444]]

token0 的四个特征全部乘 0.667
token2 的四个特征全部乘 0.444
```

这是广播的概念展开，不要求代码真的复制出整张权重矩阵。如果保留权重为 `[2]`，广播会从最后一维对齐，把 2 与输出的特征维度 4 比较，两者不匹配，会报错。即使特征维度碰巧也是 2，一维权重也会沿列缩放，不是这里所需的逐行缩放。

```
所有入选专家的权重 [3, 2]
       ↓ [mask]
当前专家对应的权重 [2]
       ↓ view(-1, 1)
每个 token 一行权重 [2, 1]
       ↓ 乘专家输出 [2, 4]
每行向量按自己的权重缩放，结果仍为 [2, 4]
```

第五步：专家计算，加权后累加回原位置。

```
y.index_add_(
    0, token_idx,
    (expert(x_flat[token_idx]) * weight).to(y.dtype)
)
```

```
x_flat [3, 4]
   ↓ 选行 [0, 2]
专家0输入 [2, 4]
   ↓ expert(...)
专家0输出 [2, 4]
   ↓ 乘 weight [2, 1]，广播到四个特征维度
加权输出 [2, 4]
   ↓ index_add_，按行号加回 y
y[0] += token0 的专家0加权结果
y[2] += token2 的专家0加权结果
```

`index_add_` 是按索引累加，不是覆盖。token0 还会接收专家1的结果，token2还会接收专家2的结果。

`index_add_` 的三个参数分别是什么？

调用格式是 `y.index_add_(dim, index, source)`，对应这段代码：

```
y.index_add_(
    0, token_idx,
```

```
    (expert(x_flat[token_idx]) * weight).to(y.dtype)
)
```

参数	本例传入值	含义
dim	0	沿第 0 维操作，即按行累加
index	token_idx	结果要加到 y 的哪些行
source	专家输出乘权重，再转换 dtype	实际要累加的数值

y 是操作的目标张量，形状为 [3, 4]，每行对应一个 token 的输出。这里 token_idx=[0, 2] 是 token 行号，不是专家编号：

```
source 的第0行 → 加到 y 的第0行，即 token0
source 的第1行 → 加到 y 的第2行，即 token2
```

第三个参数可以拆成四步：

```
selected_tokens = x_flat[token_idx]       # 取 token0、token2：[2, 4]
expert_output = expert(selected_tokens)   # 当前专家处理：[2, 4]
weighted_output = expert_output * weight  # 乘各自权重：[2, 4]
source = weighted_output.to(y.dtype)      # 与目标 y 使用相同数据类型
```

用具体数字看累加过程。假设 y 已有其他专家的贡献，当前专家的加权结果为 source：

```
y = torch.tensor([
    [10., 10., 10., 10.],
    [20., 20., 20., 20.],
    [30., 30., 30., 30.],
])
token_idx = torch.tensor([0, 2])
source = torch.tensor([
    [1., 2., 3., 4.],
    [5., 6., 7., 8.],
])

y.index_add_(0, token_idx, source)
```

本例索引不重复，可以直观理解为：

```
y[0] += source[0]
y[2] += source[1]
```

结果是：

```
[[11, 12, 13, 14],   第0行加了 source[0]
 [20, 20, 20, 20],   第1行不变
 [35, 36, 37, 38]]   第2行加了 source[1]
```

`source` 的行数要等于索引数量，本例都是 2；其余维度需要与目标 `y` 对应，本例特征维度都是 4。方法名末尾的 `_` 表示原地修改 `y`，所以不需要再写 `y = ...`。每个 token 会接收多个专家的贡献，因此这里使用加法累积，而不是覆盖已有结果。

以 token0 的假设专家输出举例：

```
专家0(x0) = [1, 2, 3, 4]
专家1(x0) = [4, 3, 2, 1]

y0 = (2/3) × [1, 2, 3, 4] + (1/3) × [4, 3, 2, 1]
   = [2, 7/3, 8/3, 3]
```

这就是公式中的加权混合，不是把两个专家的向量拼接成 8 维。

第六步：没有被选择的专家，建立零梯度连接。

```
elif self.training:
    y[0, 0] += 0 * sum(p.sum() for p in expert.parameters())
```

数值上加的是零，不改变输出。计算图上让没有 token 使用的专家参数也连接到结果，为分布式训练中的未使用参数处理提供零梯度路径。这不表示这些专家获得了有效任务学习信号，也不保证优化器的权重衰减等机制不会改变它们的参数。

第七步：重点拆解 `one_hot(...).float().mean(0)`。

```
load = F.one_hot(topk_idx, self.config.num_experts).float().mean(0)
```

仍使用：

```
topk_idx = torch.tensor([
    [0, 1],  # token0
    [1, 2],  # token1
    [2, 0],  # token2
])
```

`one_hot(..., 3)` 将每个专家编号替换为一个长度为 3 的向量：

```
专家0 → [1, 0, 0]
专家1 → [0, 1, 0]
专家2 → [0, 0, 1]
```

因此整个张量变成：

```
token0 的 [0, 1]:     token1 的 [1, 2]:     token2 的 [2, 0]:
[[1, 0, 0],          [[0, 1, 0],          [[0, 0, 1],
 [0, 1, 0]]           [0, 0, 1]]           [1, 0, 0]]

形状：[N, K] → [N, K, E] = [3, 2, 3]
                           token 槽位 专家
```

`.float()` 将整数 0、1 转成浮点 0.0、1.0，便于求平均。`.mean(0)` 沿最外层的 token 维度，对三块矩阵的对应位置相加，再除以 3：

```
                token0          token1          token2
第一个槽位:    [1, 0, 0]   +   [0, 1, 0]   +   [0, 0, 1]
第二个槽位:    [0, 1, 0]   +   [0, 0, 1]   +   [1, 0, 0]
                                ↓ 除以 3

load:               专家0   专家1   专家2
第一个槽位:        [ 1/3,    1/3,    1/3 ]
第二个槽位:        [ 1/3,    1/3,    1/3 ]

形状：[3, 2, 3] → [2, 3]
```

这里的槽位只是 top-k 返回的两个位置，不一定代表第一名、第二名。`mean(0)` 只消去 token 维度，保留槽位和专家维度，所以 `load` 不是 `[3]`。

如果再按槽位求和：

```
load.sum(dim=0)
# tensor([0.6667, 0.6667, 0.6667])
```

含义就变得直观：

```
专家0：被 token0、token2 选择 → 2/3
专家1：被 token0、token1 选择 → 2/3
专家2：被 token1、token2 选择 → 2/3

总和：2/3 + 2/3 + 2/3 = 2 = K
```

这并不是每个 token 只选一个专家的分类概率分布，所以总和不要求为 1。若所有 token 都选 `[0, 1]`，则合并后的负载是 `[1, 1, 0]`，能看出专家2完全未被使用。

第八步：将实际选择频率与平均概率合成辅助损失。

```
self.aux_loss = (
    (load * scores.mean(0)).sum()
    * self.config.num_experts
    * self.config.router_aux_loss_coef
)
```

形状变化：

```
load:             [K, E] = [2, 3]
scores.mean(0):   [E]    = [3]
                        ↓ 广播相乘
                  [K, E]
                        ↓ sum()，对所有元素求和
                      标量
```

等价写法是：

```
expert_load = load.sum(dim=0)  # [E]
expert_prob = scores.mean(dim=0)  # [E]
self.aux_loss = (
    (expert_load * expert_prob).sum()
    * self.config.num_experts
    * self.config.router_aux_loss_coef
)
```

本例中：

```
expert_load = [2/3, 2/3, 2/3]
expert_prob = [(0.6+0.1+0.4)/3,
               (0.3+0.7+0.1)/3,
               (0.1+0.2+0.5)/3]
            = [1.1/3, 1.1/3, 0.8/3]

相乘求和 = (2/3) × (1.1/3 + 1.1/3 + 0.8/3) = 2/3
再乘专家数 3 → 2
再乘默认系数 5e-4 → aux_loss = 0.001（本层）
```

计算条件为训练模式且辅助系数大于零，否则：

```
self.aux_loss = scores.new_zeros(1).squeeze()
```

得到同设备、同类型的零标量。这个 `forward` 没有接收单独的有效 token mask，负载统计使用进入该层的全部展平位置，不能假定它自动排除了 PAD。

各 MoE 层的 `aux_loss` 在模型中相加，再通过输出对象交给训练脚本。保存 `self.aux_loss` 本身不会更新参数，只有训练端将它加到总损失并反向传播时才产生相应训练作用。

第九步：恢复原始形状。

```
return y.view(batch_size, seq_len, hidden_dim)
```

```
输入 x [1, 3, 4]
       ↓ 展平
     [3, 4]
       ↓ gate + softmax
专家概率 [3, 3]
       ↓ top-k
专家索引和权重 [3, 2]
       ↓ 按专家收集 token，分别运行 FFN
       ↓ 乘权重，index_add_ 回填
输出 y [3, 4]
       ↓ 恢复批次与序列维度
     [1, 3, 4]

路由概率与选择统计 → aux_loss → 交由训练脚本加入总损失
```

学习时先走通 `scores → topk → mask → token_idx → index_add_`，再研究辅助损失和零梯度连接。这样可以先看清每个 token 去了哪里、输出怎样回来，再理解训练如何调整路由分配。

9. ResNet 与残差连接

9.1 为什么需要残差连接

图片中的核心思想是：假设某个网络模块希望学习一个理想映射 $H(x)$。如果理想情况是恒等映射：

$$H(x) = x$$

没有残差连接时，模块只能直接学习：

$$H(x) = F(x)$$

也就是说，$F(x)$ 必须完整地拟合 x。当网络很深时，这个学习目标可能比较困难。

加入残差连接后，把模块改写为：

$$H(x) = F(x) + x$$

此时，如果理想映射接近恒等映射，只需要让 $F(x) \approx 0$ 即可，不必让一个复杂的网络直接拟合 x。因此，网络学习的是"在原输入上需要修改多少"，而不是重新生成全部输出。

深层网络通常需要堆叠很多层。如果每层都必须重新生成完整特征，优化难度会随深度增加；残差连接提供了一条直接传递原始特征的路径，使新增网络层至少可以学习为"近似不改变输入"。

9.2 梯度消失与梯度爆炸

神经网络通过反向传播（backpropagation）更新参数，本质上是链式法则的连续应用。设网络有 L 层，每层的变换为 f_l，则损失函数对第一层参数 W_1 的梯度包含从第 L 层一直乘回第一层的导数：

$$\frac{\partial \text{Loss}}{\partial W_1} = \frac{\partial \text{Loss}}{\partial f_L} \cdot \frac{\partial f_L}{\partial f_{L-1}} \cdots \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial W_1}$$

这是一连串乘法。若每层的雅可比矩阵谱范数近似为 σ，经过 L 层后，梯度的量级大致会随 σ^L 变化：

- 当 $\sigma < 1$ 时，梯度会指数衰减，靠近输入端的深层参数几乎收不到有效更新，这就是梯度消失。
- 当 $\sigma > 1$ 时，梯度会指数增长，参数更新剧烈震荡，严重时数值会变成 NaN，这就是梯度爆炸。
- 只有 σ 恰好接近 1 时，梯度才较稳定；但在实际训练中，无法让每一层都自然满足这个条件。

网络越深，这个问题越明显。LSTM 的门控机制能部分缓解梯度消失；而对几十层甚至上百层的 Transformer，更直接的工具就是残差连接。

9.3 深度网络的退化问题

何恺明等人在训练深层卷积网络时观察到：更深的网络有时反而比浅网络表现更差，并且训练误差也更高。这不是过拟合，而是优化困难造成的退化问题（degradation problem）。

理论上，若在一个 20 层网络上再叠加若干层，新增层只要学习恒等映射（identity mapping），更深的网络至少应能达到原网络的效果。实际上，让普通网络精确地学习"什么都不做"并不容易。

9.4 从 ResNet 到 Transformer

ResNet 用一个简洁的结构解决上述问题：

$$\text{output} = x + F(x)$$

其中 x 是层输入，$F(x)$ 是需要学习的残差函数。若该层不需要明显改变输入，只需使 $F(x)$ 接近 0；这比让整个网络直接学习 $H(x) = x$ 更容易。跳跃路径同时为梯度提供了一条直接通路。

Transformer 的每个 Decoder Block 也使用相同思想，注意力子层和 FFN 子层各有一个残差连接：

$$h = x + \text{Attention}(x)$$

$$\text{output} = h + \text{FFN}(h)$$

因此，一个包含 N 个 Block 的 Transformer 会有 $2N$ 个带残差的子层。残差连接让模型在保留已有特征的基础上逐层学习增量修改，也使梯度能更可靠地穿过深层网络。

9.5 残差连接的理论

残差连接对梯度传播也有帮助。以标量形式简化表示，根据链式法则：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial H} \times \frac{\partial H}{\partial x} = \frac{\partial L}{\partial H}\left(\frac{\partial F}{\partial x} + 1\right)$$

其中的 `+1` 来自直接从输入到输出的跳跃路径。即使 $\partial F/\partial x$ 很小，梯度仍可以沿恒等路径继续向前传播，从而缓解深层网络中的梯度消失问题。对于向量，公式中的 `1` 更准确地说是单位矩阵 I：

$$\frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + I$$

这里的 I 表示恒等映射的导数。反向传播时，梯度至少有一条系数为 I 的直接路径；因此即使残差分支的梯度较小，也能减少梯度逐层衰减。需要注意，残差连接不能保证梯度永远不消失，但它显著改善了深层网络的优化条件。

残差分支通常称为 `F(x)`，跳跃分支直接传递 `x`，两者必须具有相同的形状才能相加。如果输入输出维度不同，工程上会使用线性层或卷积层对 `x` 做投影后再相加。

9.6 MiniMind 中的实现

MiniMind 的 Transformer Layer 使用了这种结构。例如注意力子层对应：

```
residual = hidden_states
hidden_states, present_key_value = self.self_attn(...)
hidden_states += residual
```

这就是 $H(x) = F(x) + x$：`self_attn(...)` 计算 $F(x)$，`residual` 保存原输入 x，最后将两者相加。FFN 子层也采用同样的思想：

```
hidden_states = hidden_states + self.mlp(...)
```

因此，残差连接既保留了原始特征，又允许注意力层和 FFN 学习对特征进行增量修改。

10、组装Block和Model

```
class MiniMindBlock(nn.Module):
    def __init__(self, layer_id: int, config: MiniMindConfig):
        super().__init__()
        self.self_attn = Attention(config)
        self.input_layernorm = RMSNorm(config.hidden_size,
eps=config.rms_norm_eps)
        self.post_attention_layernorm = RMSNorm(config.hidden_size,
eps=config.rms_norm_eps)
        self.mlp = FeedForward(config) if not config.use_moe else
MOEFeedForward(config)

    def forward(self, hidden_states, position_embeddings, past_key_value=None,
use_cache=False, attention_mask=None):
        residual = hidden_states
        hidden_states, present_key_value = self.self_attn(
```

```
            self.input_layernorm(hidden_states), position_embeddings,
            past_key_value, use_cache, attention_mask
        )
        hidden_states += residual#残差连接
        hidden_states = hidden_states +
self.mlp(self.post_attention_layernorm(hidden_states))
        return hidden_states, present_key_value

class MiniMindModel(nn.Module):
    def __init__(self, config: MiniMindConfig):
        super().__init__()
        self.config = config
        self.vocab_size, self.num_hidden_layers = config.vocab_size,
config.num_hidden_layers#保存词表大小和层数，vocab_size = 6400：词表里有 6400 个
token，num_hidden_layers = 8：模型有 8 层 Transformer Block
        self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)#输
入文本通常先被转换成 token ID，例如：你好 -> [231, 872]，但是 token ID 只是整数，模型不
能直接很好地处理它们，所以要通过 Embedding 转成向量：231 -> [0.12, -0.45, ..., 0.31]
        # vocab_size = 6400 hidden_size = 512 那么这个嵌入层本质上是一个大小为：6400
× 512
        self.dropout = nn.Dropout(config.dropout)
        # Dropout 是一种防止模型过拟合的技术。
        self.layers = nn.ModuleList([MiniMindBlock(l, config) for l in
range(self.num_hidden_layers)])
        self.norm = RMSNorm(config.hidden_size, eps=config.rms_norm_eps)
        # 预计算旋转位置编码
        freqs_cos, freqs_sin = precompute_freqs_cis(dim=config.head_dim,
end=config.max_position_embeddings, rope_base=config.rope_theta,
rope_scaling=config.rope_scaling)
        # 告诉 PyTorch：这是模型需要使用的张量，但它不是需要训练的参数。
persistent=False 表示：保存模型参数时，不把这两个张量写入 checkpoint。
        # register_buffer是nn.Module的方法，用于把一个“不需要训练，但属于模型状态”的张
量注册到模型中
        self.register_buffer("freqs_cos", freqs_cos, persistent=False)
        self.register_buffer("freqs_sin", freqs_sin, persistent=False)

    def forward(self, input_ids, attention_mask=None, past_key_values=None,
use_cache=False, **kwargs):
        """
        input_ids
        ↓
        Token Embedding
        ↓
        处理 KV Cache（支持续写）
        ↓
        生成/读取 RoPE 位置编码
        ↓
        依次通过每个 Transformer Block
        ↓
        RMSNorm
        ↓
        返回隐藏状态、缓存、MOE 辅助损失
        """
        batch_size, seq_length = input_ids.shape
```

```
        if hasattr(past_key_values, 'layers'): past_key_values = None
        # 1、初始化：历史kv,start_pos
        past_key_values = past_key_values or [None] * len(self.layers)#是注意力机制保存的历史 Key/Value，用于加速文本生成。
        start_pos = past_key_values[0][0].shape[1] if past_key_values[0] is not None else 0#如果已有缓存，就读取缓存中已经处理过的 token 数量。例如之前已经处理了 20 个 token：start_pos = 20
        # 2、嵌入层计算（embedding)
        hidden_states = self.dropout(self.embed_tokens(input_ids))#input_ids->嵌入层->dropout，得到隐藏状态 hidden_states
        # Recompute RoPE buffers lost during meta-device init (transformers>=5.x)
        # 3、检查 RoPE 缓存是否有效，如果无效就重新计算
        if self.freqs_cos[0, 0] == 0:#检查 RoPE 缓存是否有效,如果=0即缓冲区无效，就重新计算
            freqs_cos, freqs_sin = precompute_freqs_cis(dim=self.config.head_dim, end=self.config.max_position_embeddings, rope_base=self.config.rope_theta, rope_scaling=self.config.rope_scaling)
            self.freqs_cos, self.freqs_sin = freqs_cos.to(hidden_states.device), freqs_sin.to(hidden_states.device)#然后移动到和输入一样的设备
        # 取出当前序列需要的位置编码
        position_embeddings = (self.freqs_cos[start_pos:start_pos + seq_length], self.freqs_sin[start_pos:start_pos + seq_length])
        presents = []
        # 4、依次通过每一层 Transformer Block，传入隐藏状态、位置编码、缓存、是否使用缓存、注意力掩码
        for layer, past_key_value in zip(self.layers, past_key_values):
            hidden_states, present = layer(
                hidden_states,
                position_embeddings,
                past_key_value=past_key_value,
                use_cache=use_cache,
                attention_mask=attention_mask
            )
            presents.append(present)
        # 5、最后的归一化
        hidden_states = self.norm(hidden_states)
        # 计算 MOE 辅助损失
        aux_loss = sum([l.mlp.aux_loss for l in self.layers if isinstance(l.mlp, MOEFeedForward)], hidden_states.new_zeros(1).squeeze())
        return hidden_states, presents, aux_loss
```

11. `MiniMindForCausalLM`：把 Transformer 变成可训练、可生成的语言模型

第 10 节的 `MiniMindModel` 已经完成了词嵌入、Transformer Block、RoPE、KV Cache 和最终 RMSNorm，但它的直接输出仍是隐藏状态：

```
input_ids [batch_size, seq_len]
        -> MiniMindModel
```

```
hidden_states [batch_size, seq_len, hidden_size]
```

隐藏状态是一串向量，还不是“词表中每个 token 的分数”，因此不能直接用于预测下一个词。`MiniMindForCausalLM` 是外层包装：它增加 `lm_head` 将隐藏状态映射到词表，训练时计算 loss，推理时通过 `generate()` 持续生成 token。

```
input_ids
  -> MiniMindModel
  -> hidden_states
  -> lm_head
  -> logits（词表中每个 token 的分数）
  -> loss（仅训练时）或 generate（仅推理时）
```

11.1 类继承与职责

```
class MiniMindForCausalLM(PreTrainedModel, GenerationMixin):
    config_class = MiniMindConfig
    _tied_weights_keys = {"lm_head.weight": "model.embed_tokens.weight"}
```

成员	作用
`PreTrainedModel`	Hugging Face 的模型基类，负责配置、权重保存/加载、初始化等通用行为。
`GenerationMixin`	Hugging Face 的生成接口混入类；本类还自己实现了 `generate()`。
`config_class`	声明本模型对应的配置类型是 `MiniMindConfig`。
`_tied_weights_keys`	告诉框架 `lm_head.weight` 和 `model.embed_tokens.weight` 是一组共享权重。

“CausalLM” 中的 *causal* 表示因果掩码：位置 `t` 只能看见自己和过去的 token，并根据位置 `t` 的输出预测位置 `t+1` 的 token。

11.2 `__init__`：组装模型和输出层

```
def __init__(self, config: MiniMindConfig = None):
    self.config = config or MiniMindConfig()
    super().__init__(self.config)
    self.model = MiniMindModel(self.config)
    self.lm_head = nn.Linear(
        self.config.hidden_size,
        self.config.vocab_size,
        bias=False,
    )
    if self.config.tie_word_embeddings:
        self.model.embed_tokens.weight = self.lm_head.weight
    self.post_init()
```

`config or MiniMindConfig()` 的意思是：调用者传了配置就使用它；没有传时就创建默认配置。`super().__init__(self.config)` 初始化 `PreTrainedModel` 这一父类部分。

`self.model` 是第 10 节的主体 Transformer；`lm_head` 则是一个线性层，负责把隐藏维度投影到词表维度。假设：

```
hidden_size = 4
vocab_size  = 6
```

则各张量/参数的形状为：

名称	形状	含义
`embed_tokens.weight`	`[6, 4]`	每个词表 token 对应一个长度为 4 的输入向量。
`hidden_states`	`[batch_size, seq_len, 4]`	Transformer 对每个位置输出的隐藏向量。
`lm_head.weight`	`[6, 4]`	`nn.Linear(4, 6)` 的权重；每一行对应一个 token 的输出评分方向。
`logits`	`[batch_size, seq_len, 6]`	每个位置对 6 个 token 各给一个分数。

线性层计算可以写成：

$$\text{logits} = \text{hidden_states}\ W^T$$

其中 `W` 就是 `lm_head.weight`，形状为 `[vocab_size, hidden_size]`；因此最后一维会从 `hidden_size` 变为 `vocab_size`。

当 `tie_word_embeddings=True` 时：

```
self.model.embed_tokens.weight = self.lm_head.weight
```

让输入嵌入层和输出层使用同一个 `Parameter`，而不是两份内容相同但彼此独立的参数。这称为**权重共享**。它能减少约 `vocab_size * hidden_size` 个参数；并且一个 token 作为输入时学到的表示，会和它作为输出候选时的评分使用同一套向量。

最后的 `post_init()` 是 `PreTrainedModel` 的初始化收尾步骤。此时所有子模块都已创建，它会执行框架规定的权重初始化流程，并处理与预训练模型兼容的初始化行为；共享权重关系也会由模型配置和框架机制统一维护。

11.3 `forward`：从隐藏状态得到 logits 和训练损失

`forward()` 同时服务于训练和推理：不传 `labels` 时只输出预测结果；传入 `labels` 时额外计算交叉熵损失。

```
def forward(
    self,
    input_ids,
    attention_mask=None,
    past_key_values=None,
    use_cache=False,
    logits_to_keep=0,
    labels=None,
    **kwargs,
):
    hidden_states, past_key_values, aux_loss = self.model(
        input_ids, attention_mask, past_key_values, use_cache, **kwargs
    )
    slice_indices = (
        slice(-logits_to_keep, None)
        if isinstance(logits_to_keep, int)
        else logits_to_keep
    )
    logits = self.lm_head(hidden_states[:, slice_indices, :])
    ...
```

先经过 `self.model`：

```
input_ids:     [B, L]
hidden_states: [B, L, H]
```

其中 `B` 是 batch size，`L` 是序列长度，`H` 是 `hidden_size`。`past_key_values` 是 KV Cache；`aux_loss` 是仅 MoE 模型可能产生的辅助损失，例如让 token 更均匀地分配给不同专家。

接着：

```
logits = self.lm_head(hidden_states[:, slice_indices, :])
```

将每个保留位置的长度为 `H` 的隐藏向量，转换为长度为 `V` 的词表评分向量：

```
[B, L, H] -- lm_head --> [B, L, V]
```

11.3.1 `logits_to_keep` 和 `slice`

`logits_to_keep` 决定“最后保留多少个位置的词表分数”。整数时：

```
slice(-logits_to_keep, None)
```

就是普通切片 `[-logits_to_keep:]` 的对象写法：

logits_to_keep	slice_indices	取到的位置
0	slice(0, None)	全部位置。因为 -0 就是 0。
1	slice(-1, None)	最后一个位置。
3	slice(-3, None)	最后三个位置。

例如：

```
hidden_states[:, slice(-3, None), :]
# 等价于 hidden_states[:, -3:, :]
```

生成下一个 token 时，只需要最后一个位置的输出，所以可以设置 `logits_to_keep=1`，避免对前面所有位置都计算巨大的词表 logits。训练时通常保留默认值 `0`，因为每个位置都要参与下一个 token 的损失计算。

11.3.2 训练标签为什么要错开一位

```
loss = None
if labels is not None:
    x = logits[..., :-1, :].contiguous()
    y = labels[..., 1:].contiguous()
    loss = F.cross_entropy(
        x.view(-1, x.size(-1)),
        y.view(-1),
        ignore_index=-100,
    )
```

以 token 序列为例：

```
input_ids / labels: [我, 喜欢, 学习, 。, <EOS>]
位置:                 0    1     2    3     4
```

因果语言模型在位置 0 的输出应预测“喜欢”，位置 1 的输出应预测“学习”。所以 logits 和 labels 必须错开一位：

模型输出位置	`x = logits[..., :-1, :]` 的预测目标	`y = labels[..., 1:]` 中的正确答案
0	下一个 token	喜欢
1	下一个 token	学习
2	下一个 token	。
3	下一个 token	<EOS>

若完整 logits 的形状为 `[B, L, V]`，则错位后：

```
x.shape = [B, L - 1, V]
y.shape = [B, L - 1]
```

`[..., :-1, :]` 去掉最后一个没有"下一个真实标签"的预测；`[..., 1:]` 去掉第一个不能由序列内部前一位置预测的标签。

`contiguous()` 的作用是保证张量在内存中连续存放。切片得到的张量可能只是一个跨步访问的视图；后面的 `view()` 不复制数据、只重新解释内存形状，因此要求内存连续。`.contiguous()` 在需要时会创建连续副本，保证 `view()` 可用。

之后 `view()` 将 batch 和序列维合并：

```
x: [B, L - 1, V] -> [B * (L - 1), V]
y: [B, L - 1]    -> [B * (L - 1)]
```

这样每一行预测分数都和一个正确 token ID 对应，正好符合 `F.cross_entropy` 的输入格式。`ignore_index=-100` 表示标签为 `-100` 的位置不计入 loss，常用于 padding 或聊天训练中不需要监督的 prompt 部分。

11.4 返回 `MoeCausalLMOutputWithPast`

```
return MoeCausalLMOutputWithPast(
    loss=loss,
    aux_loss=aux_loss,
    logits=logits,
    past_key_values=past_key_values,
    hidden_states=hidden_states,
)
```

这是一个带字段名的结果对象。调用方可以清楚地按字段取值：

```
outputs = model(input_ids, labels=labels)
train_loss = outputs.loss
scores = outputs.logits
```

字段	含义
`loss`	监督训练的交叉熵损失；未传 `labels` 时为 `None`。
`aux_loss`	MoE 辅助损失；非 MoE 模型通常为 0。
`logits`	每个保留位置对整个词表的预测分数。

字段	含义
`past_key_values`	当前轮产生的 KV Cache，下一轮生成时传回模型加速。
`hidden_states`	`lm_head` 之前的最后一层隐藏状态。

11.5 `generate()`：逐个 token 续写

`@torch.inference_mode()` 表示生成只做推理、不做反向传播，因此 PyTorch 不保存梯度计算图，显存占用更低、速度更快。

`generate()` 不是一次性写完整段回答，而是不断重复同一件事：根据已有 token 预测下一个 token，把它拼到末尾，再继续预测。

参数	作用
`inputs` / `input_ids`	已经过 Tokenizer 编码的提示词 token ID。传 `input_ids` 时优先使用它。
`attention_mask`	`1` 表示真实 token，`0` 表示 padding。
`max_new_tokens`	最多再生成多少个 token。
`temperature`	控制分数差异和抽样随机性。
`top_k` / `top_p`	生成前过滤候选 token。
`do_sample`	`True` 时按概率抽样，`False` 时总选最高分。
`repetition_penalty`	降低已经出现过 token 的分数。
`eos_token_id`	生成到该 token 时标记该序列结束。
`use_cache`	是否使用 KV Cache 加速下一轮。
`num_return_sequences`	对同一个提示词并行生成多少条结果。
`streamer`	用于将 token 按生成顺序实时交给终端或界面显示。

```
已有 token -> 模型给词表中每个 token 打分 -> 按规则选一个 token
          -> 拼到已有 token 后面 -> 判断是否结束
```

11.5.1 一个可计算的例子

为了让数据足够小，假设词表只有 6 个 token：

token ID	token	说明
0	我	普通 token
1	喜欢	普通 token
2	<EOS>	结束标记
3	学习	普通 token

token ID	token	说明
4	。	标点
5	AI	普通 token

输入提示词是“我喜欢”，Tokenizer 编码后为：

```
input_ids = tensor([[0, 1]])
attention_mask = tensor([[1, 1]])
```

它们的形状分别都是 [batch_size, sequence_length]：

```
input_ids.shape      = [1, 2]
attention_mask.shape = [1, 2]
```

第 0 维的 1 表示这次只有一条输入文本；第 1 维的 2 表示“我喜欢”当前有两个 token。

这一轮假设模型在最后一个位置输出的分数为：

```
logits = tensor([[0.2, 0.3, -1.0, 2.4, 1.0, 1.7]])
```

它的形状是 [1, 6]。每一列正好对应一个词表 token：

token	我	喜欢	<EOS>	学习	。	AI
logits	0.2	0.3	-1.0	2.4	1.0	1.7

此时模型最倾向于生成“学习”，但在真正选 token 前，代码还会执行重复惩罚、top_k、top_p 和抽样等步骤。

11.5.2 初始化部分

```
input_ids = kwargs.pop("input_ids", inputs).repeat(num_return_sequences, 1)
attention_mask = attention_mask.repeat(num_return_sequences, 1) if attention_mask
is not None else None
past_key_values = kwargs.pop("past_key_values", None)
finished = torch.zeros(input_ids.shape[0], dtype=torch.bool,
device=input_ids.device)
```

kwargs.pop("input_ids", inputs) 的意思是：若调用者通过关键字参数传了 input_ids，就取出并从 kwargs 删除它；否则使用位置参数 inputs。pop 的目的是避免同一个参数之后又通过 **kwargs 重复传给 forward()。

`input_ids` 是 token ID 的二维表，形状为 `[batch_size, sequence_length]`。
`repeat(num_return_sequences, 1)` 会在第 0 维复制输入，从而让同一个提示词一次生成多个版本。

例如：

```
input_ids = tensor([[0, 1]])
num_return_sequences = 3

input_ids.repeat(3, 1)
# tensor([
#     [0, 1],
#     [0, 1],
#     [0, 1],
# ])
```

`finished` 是“每一条文本是否已经结束”的布尔表。上面有 3 条文本，因此初始结果是：

```
finished = tensor([False, False, False])
```

`device=input_ids.device` 的作用是让 `finished` 和 `input_ids` 同在 CPU 或同在 GPU；不同设备上的张量不能直接一起计算。

同样，`attention_mask.repeat(num_return_sequences, 1)` 必须同步复制 mask，才能继续和复制后的 `input_ids` 一一对应。这里的“末尾补 1”逻辑适用于未 padding 或左侧 padding 的生成输入；若对右侧 padding 的不同长度输入直接生成，padding 的 `0` 会夹在真实 token 和新 token 之间，通常应先改用左侧 padding 或逐条处理。

11.5.3 每轮先让模型预测

生成循环的骨架是：

```
for _ in range(max_new_tokens):
    ...
```

`max_new_tokens` 限制的是**新生成** token 的数量，而不是提示词加生成文本的总长度。

```
past_len = past_key_values[0][0].shape[1] if past_key_values else 0
outputs = self.forward(
    input_ids[:, past_len:], attention_mask,
    past_key_values, use_cache=use_cache, **kwargs
)
```

第一轮没有缓存：

```
past_key_values = None
past_len = 0
input_ids[:, past_len:] == input_ids[:, 0:]
```

因此模型会读取完整提示词 `[0, 1]`，输入形状为 `[1, 2]`。`forward()` 的完整 logits 通常是：

```
outputs.logits.shape = [1, 2, 6]
                         |  |  |
                         |  |  词表大小
                         |  序列中每个位置都有一组预测分数
                        batch size
```

虽然有两个位置的分数，生成下一个 token 时只需要最后一个位置：

```
logits = outputs.logits[:, -1, :] / temperature
```

取出后形状变成 `[1, 6]`。`temperature` 用于调节分数差异：小于 1 会拉大高低分差异，让选择更保守；大于 1 会缩小差异，让抽样更随机。

生成一个新 token 后，代码还会提前给注意力掩码补一个 `1`：

```
attention_mask = torch.cat(
    [attention_mask, attention_mask.new_ones(attention_mask.shape[0], 1)],
    -1,
)
```

例子中的 mask 会从：

```
[[1, 1]]
```

变成：

```
[[1, 1, 1]]
```

最后的 `1` 对应即将拼入的真实新 token。`new_ones(1, 1)` 创建的是形状 `[1, 1]`、全为 1、且与原 mask 类型和设备一致的张量；`torch.cat(..., -1)` 表示沿最后一维，也就是序列长度这一维拼接。

11.5.4 重复惩罚：降低已经出现过的 token 分数

```
if repetition_penalty != 1.0:
    for i in range(input_ids.shape[0]):
        seen = torch.unique(input_ids[i])
        score = logits[i, seen]
        logits[i, seen] = torch.where(
            score > 0,
            score / repetition_penalty,
            score * repetition_penalty,
        )
```

在例子中：

```
input_ids[0] = tensor([0, 1])
seen = torch.unique(input_ids[0])
# tensor([0, 1])
```

`seen` 中保存的是**词表 ID**，不是"第几个位置"。因此：

```
logits[0, seen]
# 等价于 logits[0, [0, 1]]
# tensor([0.2, 0.3])
```

它取到了第 0 条文本里，token `我` 和 `喜欢` 当前的预测分数。若 `repetition_penalty=1.2`，正分数会除以 1.2：

```
我:    0.2 -> 0.2 / 1.2 = 0.167
喜欢:  0.3 -> 0.3 / 1.2 = 0.250
```

若分数为负数，代码改用乘法，例如 `-1.0 * 1.2 = -1.2`。两种写法都会使分数变小，从而降低已经出现过的 token 再次被选中的概率。`repetition_penalty=1.0` 表示不进行任何惩罚。

11.5.5 `top_k`：固定保留分数最高的 K 个候选

```
if top_k > 0:
    logits[logits < torch.topk(logits, top_k)[0][..., -1, None]] = -float("inf")
```

拆开后等价于：

```
topk_values, _ = torch.topk(logits, top_k)
threshold = topk_values[..., -1, None]
```

```
remove_mask = logits < threshold
logits[remove_mask] = -float("inf")
```

继续使用前面的 logits，并令 top_k=3：

```
topk_values, topk_indices = torch.topk(logits, 3)

topk_values  # [[2.4, 1.7, 1.0]]
topk_indices # [[3,   5,   4]]
```

最高的 3 个分数分别属于 token 学习、AI、。。第 3 名的分数是 1.0，因此所有小于 1.0 的分数都被改成 -inf：

token	我	喜欢	<EOS>	学习	。	AI
top-k 前	0.2	0.3	-1.0	2.4	1.0	1.7
top-k 后	-inf	-inf	-inf	2.4	1.0	1.7

-float("inf") 是负无穷。后续经过 softmax 后，它的概率是 0，因此不会被抽到。

topk_values[..., -1, None] 中的 -1 表示取每一行最后一个值，也就是第 K 名分数；None 把形状从 [batch_size] 变为 [batch_size, 1]，使这个门槛能自动和整行词表分数比较。这里 top_k 必须不大于词表大小。

11.5.6 top_p：按累计概率动态保留候选

```
if top_p < 1.0:
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    mask = torch.cumsum(torch.softmax(sorted_logits, dim=-1), dim=-1) > top_p
    mask[..., 1:], mask[..., 0] = mask[..., :-1].clone(), 0
    logits[mask.scatter(1, sorted_indices, mask)] = -float("inf")
```

top_p 不固定候选数量，而是保留“累计概率达到 top_p 所需的最少候选”。在 top-k 后，只剩下这三个有效候选：

```
学习：2.4
AI：  1.7
。：   1.0
```

先按分数降序排序：

```
sorted_logits  = [[2.4, 1.7, 1.0, -inf, -inf, -inf]]
sorted_indices = [[3,   5,   4,   ...]]
```

对前三个有效分数做 softmax，概率约为：

```
学习：0.574
AI：  0.285
。：   0.141
```

累积概率为：

```
0.574, 0.859, 1.000
```

若 `top_p=0.8`，先比较“累计概率是否大于 0.8”：

```
mask = [False, True, True, True, True, True]
```

但第二个 token `AI` 是让累计概率从 `0.574` 第一次达到 `0.8` 的 token，所以它也应当保留。下面这句把 mask 向右移动一格：

```
mask[..., 1:], mask[..., 0] = mask[..., :-1].clone(), 0
```

移动后：

```
mask = [False, False, True, True, True, True]
```

在排序后的顺序里，这表示保留 `学习` 和 `AI`，删除 `。` 以及后面本来就无效的候选。

`sorted_indices` 保存“排序后位置对应的原 token ID”。`scatter` 将排序后的删除标记放回原词表位置：

```
remove_mask = mask.scatter(1, sorted_indices, mask)
```

`scatter` 的三个参数可以理解为：

```
原张量.scatter(
    1,              # 沿第 1 维（词表这一维）写入
    sorted_indices, # 每个值该写回哪个 token ID
    mask,           # 写入的 True / False 值
)
```

在这个例子中，最终会把 token 。 的位置标为 True，然后执行：

```
logits[remove_mask] = -float("inf")
```

结果只剩：

```
学习：2.4
AI：  1.7
其他：-inf
```

11.5.7 从概率中选出 next_token

```
next_token = (
    torch.multinomial(torch.softmax(logits, dim=-1), num_samples=1)
    if do_sample
    else torch.argmax(logits, dim=-1, keepdim=True)
)
```

softmax(logits, dim=-1) 会将每行词表分数转换为总和为 1 的概率。经过 top-p 后，例子中的概率重新归一化为：

```
学习：0.668
AI：  0.332
其他：0.000
```

当 do_sample=True 时：

```
torch.multinomial(probabilities, num_samples=1)
```

会按概率抽样。因此大约 66.8% 的情况下选到 学习，约 33.2% 的情况下选到 AI。同样的提示词可能产生不同答案。

当 do_sample=False 时：

```
torch.argmax(logits, dim=-1, keepdim=True)
```

每次都取最大分数的位置，因此一定选 token ID 3，即 学习。

无论采用哪种方式，next_token 的形状均为 [batch_size, 1]。本例中若抽到“学习”：

```
next_token = tensor([[3]])
```

11.5.8 拼接 token、缓存与停止条件

```
input_ids = torch.cat([input_ids, next_token], dim=-1)
past_key_values = outputs.past_key_values if use_cache else None
```

`torch.cat(..., dim=-1)` 沿序列长度维拼接。例子中：

```
input_ids  = tensor([[0, 1]])
next_token = tensor([[3]])

input_ids = tensor([[0, 1, 3]])
# 文本从“我喜欢”变成“我喜欢学习”
```

若 `use_cache=True`，模型还会保存每层注意力的 Key/Value，作为 `past_key_values`。下一轮的关系如下：

轮次	当前完整 `input_ids`	`past_len`	本轮送入模型的 token	缓存含义
第 1 轮	`[0, 1]`	0	`[0, 1]`	首轮无缓存，处理两个 token
第 2 轮	`[0, 1, 3]`	2	`[3]`	前两个 token 已在缓存中，只计算新 token
第 3 轮	`[0, 1, 3, ...]`	3	`[...]`	历史继续从缓存读取

因此代码中的：

```
input_ids[:, past_len:]
```

在第二轮会得到 `input_ids[:, 2:]`，即只包含刚生成的 token `[3]`，形状是 `[1, 1]`。这就是 KV Cache 能加速长文本生成的原因：已经算过的历史 token 不必每轮重复送入所有 Transformer 层计算。

11.5.9 多条文本同时生成时的 `<EOS>` 处理

```
if eos_token_id is not None:
    next_token = torch.where(
        finished.unsqueeze(-1),
        next_token.new_full((next_token.shape[0], 1), eos_token_id),
```

```
            next_token,
        )
```

`torch.where(条件, 值A, 值B)` 的意思是：条件为 `True` 时取值 A，否则取值 B。

假设并行生成三条文本，其中第 2 条已经结束：

```
finished = tensor([False, True, False])
next_token = tensor([[3], [5], [4]])
eos_token_id = 2
```

`finished.unsqueeze(-1)` 把形状从 `[3]` 变成 `[3, 1]`：

```
[[False], [True], [False]]
```

`next_token.new_full((3, 1), 2)` 创建：

```
[[2], [2], [2]]
```

最后 `torch.where` 的结果为：

```
next_token = tensor([
    [3],  # 未结束，保留模型新选出的 token
    [2],  # 已结束，强制继续写入 <EOS>
    [4],  # 未结束，保留模型新选出的 token
])
```

这是为了让同一个 batch 中的各条序列始终具有相同长度。已经结束的序列不会再生成正常文本，只会补 `<EOS>`。

随后：

```
finished |= next_token.squeeze(-1).eq(eos_token_id)
if finished.all():
    break
```

`eq(eos_token_id)` 会逐项判断新 token 是否为结束标记；`|=` 表示和旧状态做“或”运算，所以已经结束的序列会保持结束。若三条序列都已结束，例如：

```
finished = tensor([True, True, True])
```

`finished.all()` 为 `True`，循环提前退出。否则，即使没有遇到 `<EOS>`，循环也会在生成 `max_new_tokens` 个 token 后停止。

11.5.10 整段代码的简化版本

忽略批量生成、流式输出等工程细节后，`generate()` 可以概括为：

```
input_ids = prompt_token_ids
cache = None

for _ in range(max_new_tokens):
    logits, cache = model(input_ids_since_cache, cache)
    logits = lower_scores_for_repeated_tokens(logits)
    logits = keep_top_k_candidates(logits)
    logits = keep_top_p_candidates(logits)

    next_token = sample_or_choose_max(logits)
    input_ids = append(input_ids, next_token)

    if next_token == eos_token_id:
        break

return input_ids
```

最终返回的仍是 token ID，例如 `[0, 1, 3, 4, 2]`。调用方通常还需要交给 Tokenizer 解码，才能得到人能阅读的文本：

```
text = tokenizer.decode(input_ids[0], skip_special_tokens=True)
# "我喜欢学习。"
```

实际代码还支持两种返回/输出方式：

```
if streamer:
    streamer.put(input_ids.cpu())  # 循环前先发送提示词
    ...
    streamer.put(next_token.cpu()) # 每轮发送一个新 token
    ...
    streamer.end()                 # 通知流结束

if kwargs.get("return_kv"):
    return {"generated_ids": input_ids, "past_kv": past_key_values}
return input_ids
```

`streamer` 常用于命令行或网页中的逐字显示；`.cpu()` 将 token ID 移到 CPU，方便一般的解码/显示逻辑使用。默认返回完整 token ID 张量；`return_kv=True` 时额外返回最终 KV Cache，调用方可以在该上下文后继

续续写。

第三部分：模型训练

1. Pretrain：完整参数训练

本节对应 `trainer/train_pretrain.py` 中的 `train_epoch` 函数。它负责重复执行“取一批数据 → 前向传播 → 计算损失 → 反向传播 → 更新参数”，让模型逐步学会预测下一个 token。

1.1 一次训练迭代的整体流程

```
DataLoader 取出 input_ids、labels
        ↓
把数据移动到 GPU / CPU
        ↓
根据当前步数设置学习率
        ↓
前向传播，得到 loss
        ↓
反向传播，计算梯度
        ↓
达到梯度累积步数后：取消缩放 → 裁剪梯度 → 更新参数 → 清空梯度
```

对应的核心代码是：

```
for step, (input_ids, labels) in enumerate(loader, start=start_step + 1):
    input_ids = input_ids.to(args.device)
    labels = labels.to(args.device)

    with autocast_ctx:
        res = model(input_ids, labels=labels)
        loss = (res.loss + res.aux_loss) / args.accumulation_steps

    scaler.scale(loss).backward()

    if step % args.accumulation_steps == 0:
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)
```

1.2 输入数据与学习率

```
for step, (input_ids, labels) in enumerate(loader, ...):
    input_ids = input_ids.to(args.device)
```

```
        labels = labels.to(args.device)
```

`loader` 每次给出一个 batch 的训练样本：

- `input_ids`：输入给模型的 token 编号，例如 `[BOS, 我, 喜欢, 学习]`。
- `labels`：期望模型预测出的下一个 token。模型内部会将预测结果和标签错开一位后计算交叉熵损失。
- `.to(args.device)`：将张量移动到训练设备，例如 `cuda:0` 或 CPU；模型和数据必须位于同一设备。

每一步训练前还会更新学习率：

```
lr = get_lr(epoch * iters + step, args.epochs * iters, args.learning_rate)
for param_group in optimizer.param_groups:
    param_group['lr'] = lr
```

这表示学习率会随总训练步数变化，而不是从头到尾保持不变。训练初期通常需要较平稳地进入训练状态，后期则逐渐减小学习率以稳定收敛。

1.3 混合精度：`autocast`

```
with autocast_ctx:
    res = model(input_ids, labels=labels)
    loss = res.loss + res.aux_loss
```

`autocast` 是自动混合精度的上下文管理器，主要作用于前向传播：PyTorch 会让适合低精度的矩阵乘法等操作使用 `float16` 或 `bfloat16`，而容易产生数值问题的操作保留较高精度。这样可以减少显存占用并加快 GPU 训练。

它只包住前向传播和损失计算，不包住 `backward()` 与优化器更新。原因是：前向计算最耗时、最适合使用低精度；反向传播的梯度稳定性则由后面的 `GradScaler` 管理。

`res.loss` 是语言模型预测下一个 token 的主损失；`res.aux_loss` 是使用 MoE 模型时的专家负载均衡等辅助损失。Dense 模型中辅助损失通常为 0。

1.4 梯度累积：为什么要除以 `accumulation_steps`

```
loss = loss / args.accumulation_steps
```

显存不足时，不能一次放入很大的 batch。梯度累积的做法是连续处理多个小 batch，只在最后更新一次参数，效果近似于使用一个更大的 batch。

假设 `accumulation_steps=4`，四次小 batch 的损失分别为 L_1, L_2, L_3, L_4。如果不除以 4，累积梯度会是四个梯度的**和**，比大 batch 的平均梯度大约 4 倍；因此每次反向传播前应计算：

$$\frac{L_i}{4}$$

累积四次后得到：

$$\frac{\nabla L_1 + \nabla L_2 + \nabla L_3 + \nabla L_4}{4}$$

这与一次处理四个小 batch 后取平均损失的梯度一致。

1.5 `GradScaler`：防止 FP16 梯度下溢

```
scaler.scale(loss).backward()
```

在 `float16` 下，特别小的梯度可能因为数值范围有限而直接变成 0，模型将无法正常学习。`scaler.scale(loss)` 会先将损失放大，再执行反向传播；梯度也随之放大，从而避免小梯度丢失。

MiniMind 中的初始化是：

```
scaler = torch.cuda.amp.GradScaler(enabled=(args.dtype == 'float16'))
```

所以当训练精度设置为 `float16` 时，`GradScaler` 生效；默认 `bfloat16` 时它处于禁用状态，相关调用仍可以保留，但相当于普通的 `loss.backward()` 和 `optimizer.step()`。`bfloat16` 的数值范围更大，通常不需要梯度缩放。

1.6 梯度裁剪与参数更新

达到设定的累积步数后，才真正更新参数：

```
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
scaler.step(optimizer)
scaler.update()
optimizer.zero_grad(set_to_none=True)
```

执行顺序及原因如下：

1. `scaler.unscale_(optimizer)`：将被放大的梯度还原；只有看到真实梯度，才能正确判断它是否过大。
2. `clip_grad_norm_`：梯度裁剪。若所有参数梯度组成的整体向量范数超过 `args.grad_clip`（默认 `1.0`），便按相同比例缩小所有梯度，使范数等于上限。它不改变梯度方向，只限制更新步长，避免梯度爆炸导致训练不稳定。
3. `scaler.step(optimizer)`：使用裁剪后的梯度调用优化器（本项目为 AdamW）更新模型参数；若 `GradScaler` 检测到 `inf` 或 `nan`，会跳过本次更新。
4. `scaler.update()`：根据本次是否出现溢出，自动调整下一轮的缩放倍率。
5. `optimizer.zero_grad(set_to_none=True)`：清除旧梯度。PyTorch 默认会累加梯度，若不清除，上一次的梯度会错误混入下一次更新。

1.7 一个训练周期的简化伪代码

```
for small_batch in dataloader:
    loss = model(small_batch).loss / accumulation_steps
    scale(loss).backward()       # 梯度先累积，不立即更新

    if 已累积到指定次数:
        unscale_gradients()
        clip_gradients(max_norm=1.0)
        optimizer.step()
        update_scaler()
        optimizer.zero_grad()
```

一句话总结：`autocast` 解决“前向计算如何更快、更省显存”，`GradScaler` 解决“FP16 梯度太小会消失”，梯度累积解决“小显存也能近似大 batch”，梯度裁剪解决“梯度过大导致训练不稳定”。

2. Full SFT：全参数监督微调

`train_full_sft.py` 是全参数监督微调（Full Supervised Fine-Tuning）脚本。它以预训练模型为起点，使用指令—回答、问答等监督数据，让模型从“能续写文本”进一步变成“能理解并遵循用户指令”。

它的训练循环和 Pretrain 很像，都是对完整模型做前向传播、反向传播和参数更新；但训练数据和训练起点不同：

```
model, tokenizer = init_model(lm_config, args.from_weight, device=args.device)
train_ds = SFTDataset(args.data_path, tokenizer, max_length=args.max_seq_len)
optimizer = optim.AdamW(model.parameters(), lr=args.learning_rate)
```

- `from_weight='pretrain'`：默认从预训练权重开始，而不是从随机参数开始。
- `SFTDataset`：使用指令/问答格式数据，而非普通连续文本。
- `model.parameters()`：优化器接收完整模型参数，因此 Embedding、Attention、FFN、RMSNorm 等全部会更新。

可以把 Full SFT 理解为：保留预训练模型已有的语言能力，再使用高质量示例教它“面对某种指令，应当以什么格式、什么风格回答”。它会保存完整的微调后模型权重，因此效果通常比 LoRA 更容易充分适配任务，但显存、训练时间和存储开销也更大。

3. LoRA：低秩参数高效微调

Pretrain、Full SFT 与 LoRA 的训练循环结构都很相似：它们都会取 batch、前向传播、计算损失、反向传播、裁剪梯度和更新参数；差异主要来自训练数据、训练起点和更新对象。

对比项	Pretrain（预训练）	Full SFT（全参数监督微调）	LoRA（低秩微调）
训练目标	从大量文本中学习语言规律和下一个 token 预测能力。	学会按指令或问答格式回答。	在较低资源开销下适配特定领域或任务。
初始	通常从零开始，或继续预训练。	通常加载预训练模型。	通常加载预训练模型。

对比项	Pretrain（预训练）	Full SFT（全参数监督微调）	LoRA（低秩微调）
权重			
训练数据	`PretrainDataset`，主要是普通连续文本。	`SFTDataset`，指令、问答等监督数据。	`SFTDataset`，指令、问答等监督数据。
可训练参数	整个模型的全部参数。	整个模型的全部参数。	仅 LoRA 的低秩矩阵 A、B 参数。
原模型权重	会被梯度更新。	会被梯度更新。	被冻结，不参与更新。
优化器	接收 `model.parameters()`。	接收 `model.parameters()`。	只接收 `lora_params`。
梯度裁剪对象	所有模型参数。	所有模型参数。	仅 LoRA 参数。
输出文件	保存完整模型权重。	保存完整指令模型权重。	通常只保存较小的 LoRA 权重文件。
资源开销	显存、时间和存储开销较大。	显存、时间和存储开销较大。	显存、时间和存储开销明显更低。

3.1 与 Pretrain 的核心区别

预训练将整个模型参数交给优化器：

```
optimizer = optim.AdamW(model.parameters(), lr=args.learning_rate)
```

因此每一次 `optimizer.step()` 都可能更新 Embedding、Attention、FFN、RMSNorm 等所有可训练参数。

LoRA 微调先冻结基础模型，并收集 LoRA 参数：

```
lora_params = []
for name, param in model.named_parameters():
    if 'lora' in name:
        param.requires_grad = True
        lora_params.append(param)
    else:
        param.requires_grad = False

optimizer = optim.AdamW(lora_params, lr=args.learning_rate)
```

因此 LoRA 训练时，基础权重 W 保持不变，只更新低秩增量：

$$\Delta W = BA$$

有效权重可写为：

$$W_{\text{effective}} = W + BA$$

3.2 梯度裁剪与保存的区别

预训练裁剪完整模型的梯度：

```
torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
```

LoRA 只裁剪实际会更新的参数：

```
torch.nn.utils.clip_grad_norm_(lora_params, args.grad_clip)
```

预训练通常保存完整模型；而 LoRA 使用：

```
save_lora(model, lora_save_path)
```

只保存每层的 `lora.A.weight` 与 `lora.B.weight`。推理时需要将它们重新挂载到对应的基础模型上，或通过 `merge_lora()` 将 BA 合并回基础权重 W。

3.3 小结

可以把两者概括为：

```
Pretrain：从零开始或继续训练，全面学习语言能力。
LoRA：不改变基础模型，只训练小型低秩“补丁”，获得特定领域或任务能力。
```

4. PPO 强化学习训练

PPO（Proximal Policy Optimization，近端策略优化）用于在 SFT 模型基础上继续优化回答质量。与 Pretrain、Full SFT、LoRA 的“给定正确答案，计算交叉熵损失”不同，PPO 的训练信号来自对完整回答的奖励分数。

```
prompt
  ↓
Actor 生成 response（rollout）
  ↓
Reward Model 对完整 response 打分
  ↓
Critic 估计每个回答 token 的未来奖励
```

```
  ↓
计算 advantage：实际结果比预期好还是差
  ↓
更新 Actor 与 Critic；同时限制 Actor 不偏离原模型太远
```

4.1 PPO 理论基础：先理解训练目标

PPO 属于强化学习。与 SFT 的“给定标准答案，要求模型模仿”不同，PPO 不要求每个问题只有唯一正确答案，而是让模型生成回答后，根据奖励信号学习“什么样的回答更好”。

对于语言模型，可以将一次生成过程看作：

```
状态 s_t：prompt 加上已经生成的前文 token
动作 a_t：当前要生成的下一个 token
策略 πθ(a_t|s_t)：Actor 为所有候选 token 给出的概率分布
轨迹 τ：从第一个 response token 到 EOS 的完整回答
奖励 R：对完整回答质量的评价分数
```

PPO 的最终目标是最大化期望奖励：

$$\max_{\theta} \ \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$$

但直接只追求高奖励会有两个风险：模型可能一次更新过大而训练崩溃，也可能为了迎合奖励模型而丢失原有的语言和对话能力。因此 PPO 同时使用 Critic、概率裁剪和 Reference KL 约束。

4.1.1 Actor、Critic 与 Advantage

Actor 是当前语言模型策略 π_θ，负责决定下一个 token 的概率。Critic 估计状态价值：

$$V(s_t) = \text{从 } s_t \text{ 继续生成时，未来能获得的预期奖励}$$

优势函数表示实际结果相对 Critic 预期的好坏：

$$A_t = Q(s_t, a_t) - V(s_t)$$

其中 $A_t > 0$ 表示该动作比预期更好，应提高它的概率；$A_t < 0$ 表示该动作较差，应降低它的概率。实际代码中通过 GAE 近似计算 A_t，而不是直接计算完整的 Q。

4.1.2 Return、TD 与 GAE

若只将最终奖励 R 分给整条回答，前面 token 无法知道自己是否有贡献。GAE 会将末尾奖励向前传播。

一步 TD 误差为：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE 优势为：

$$A_t^{\text{GAE}} = \delta_t + \gamma\lambda A_{t+1}^{\text{GAE}}$$

`gamma` 表示未来奖励的折扣程度，`lambda` 控制 GAE 在偏差和方差之间的折中。Critic 的训练目标为：

$$\text{return}_t = A_t + V_{\text{old}}(s_t)$$

4.1.3 为什么需要旧策略与概率比 `ratio`

回答由 rollout 阶段的旧策略 π_{old} 生成，但训练时 Actor 已经开始更新，变成 π_{new}。PPO 用重要性采样概率比连接二者：

$$r_t(\theta) = \frac{\pi_{\text{new}}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} = \exp(\log p_{\text{new}} - \log p_{\text{old}})$$

若 `ratio=1`，说明新旧策略未变；大于 1 表示当前 Actor 提高了该 token 概率；小于 1 表示降低了该 token 概率。

4.1.4 PPO Clip：限制更新步幅

普通策略梯度会按 $A_t r_t$ 调整概率，但若 `ratio` 过大，模型可能单次跳得太远。PPO 将它裁剪到 $[1-\epsilon, 1+\epsilon]$：

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(A_t r_t,\ A_t \,\text{clip}(r_t, 1-\epsilon, 1+\epsilon)\right)\right]$$

这意味着高优势 token 可以提高概率、低优势 token 可以降低概率，但超过允许范围后不会继续获得额外优化收益。

4.1.5 Reference KL：保留 SFT 能力

PPO 通常从 SFT 模型开始。为避免 Actor 为追求奖励而偏离 SFT 模型太远，会加入：

$$\beta D_{\text{KL}}(\pi_{\text{new}} \| \pi_{\text{ref}})$$

其中 `ref_model` 是固定参考模型，β 对应代码中的 `kl_coef`。本项目把 KL 惩罚直接加在 Actor 的 update loss 中；另一种常见实现会将逐 token KL 作为负奖励，先进入 GAE。二者目标相同，都是限制策略偏离，但实现位置不同。

4.1.6 一次 PPO 训练的四个阶段

```
1. Rollout：Actor 自回归生成 response，并记录旧策略 token 概率。
2. Reward：Reward Model 和规则为完整回答打分。
3. Advantage：Critic 估计价值，GAE 将奖励转换为每个 token 的 advantage 与 return。
4. Update：使用 PPO clip 更新 Actor，使用 value loss 更新 Critic，并加入 Reference KL 约束。
```

4.2 PPO 中的模型角色

组件	作用
`actor_model`	当前要训练的语言模型；它生成回答并学习提高高奖励 token 的概率。
`critic_model`	价值模型；它预测"生成到当前位置，最终预期能获得多少奖励"。

组件	作用
`ref_model`	固定的参考模型，通常是 PPO 前的 SFT 模型；用于 KL 约束。
`reward_model`	对完整回答评分，给出外部奖励。
`rollout_engine`	使用当前 Actor 生成回答，并记录生成时的 token 概率。

可将它们概括为：

```
Actor：下一 token 应该生成什么？
Critic：当前回答预计能得多少分？
Reward Model：这份完整回答实际质量如何？
Reference Model：Actor 不应偏离原来的 SFT 能力太远。
```

4.3 Critic：从隐藏状态预测价值

Critic 复用语言模型的 Transformer 主干，但不使用词表输出头来预测下一个词，而是新增一个输出单一数值的 `value_head`：

```
class CriticModel(MiniMindForCausalLM):
    def __init__(self, params):
        super().__init__(params)
        self.value_head = nn.Linear(params.hidden_size, 1)

    def forward(self, input_ids=None, attention_mask=None, **kwargs):
        outputs = self.model(input_ids=input_ids, attention_mask=attention_mask,
**kwargs)
        hidden_states = self.model.norm(outputs[0])
        return self.value_head(hidden_states).squeeze(-1)
```

`outputs[0]` 是 Transformer 最后一层的隐藏状态，形状为 `[B, L, H]`：B 是 batch 大小，L 是序列长度，H 是隐藏维度。`self.model.norm(outputs[0])` 是最后一次 RMSNorm，用于稳定隐藏状态的数值尺度。

`value_head` 将每个 token 的 H 维向量压缩为一个数：

$$V(s_t) = W_v h_t + b_v$$

输出形状为 `[B, L]`。其中 $V(s_t)$ 是 Critic 对“当前状态 s_t 之后能获得多少未来奖励”的估计。注意，代码是新增 `value_head` 并在 Critic 的 `forward` 中使用它；并非真正删除父类的 `lm_head`。

4.4 Rollout：Actor 先生成回答

```
rollout_result = rollout_engine.rollout(
    prompt_ids=enc.input_ids,
    attention_mask=enc.attention_mask,
    num_generations=1,
    max_new_tokens=args.max_gen_len,
```

```
        temperature=0.8,
    )
```

一次 rollout 的重要输出如下：

变量	含义	常见形状
`gen_out`	完整的 `prompt + response` token 序列。	`[B, P+R]`
`completion_ids`	只包含生成回答的 token。	`[B, R]`
`responses_text`	回答的文本形式。	长度为 `B` 的列表
`old_resp_logp`	生成当时，旧 Actor 对每个回答 token 的对数概率。	`[B, R]`
`prompt_lens`	每条 prompt 的长度。	`[B]`
`completion_mask`	response 中真实 token 为 1、PAD 为 0。	`[B, R]`

`old_resp_logp` 很重要：PPO 后续会计算当前 Actor 的新概率，再比较“新旧概率变化是否过大”。

4.5 奖励：评价完整回答

```
rewards = calculate_rewards(prompts, responses_text, reward_model)
```

每一条回答得到一个总奖励，形状为 `[B]`。该项目的奖励由规则分与奖励模型分相加：

```
总奖励 = 回答长度奖励 + 思考格式奖励 - 重复惩罚 + Reward Model 评分
```

4.5.1 从 prompt 中提取对话历史

```
pattern = r"<\|im_start\|>(system|user|assistant)\s+(.*?)<\|im_end\|>"
matches = re.findall(pattern, prompt, re.DOTALL)
messages = [{"role": role, "content": content.strip()} for role, content in
matches]
```

正则中的两个捕获组是：

```
(system|user|assistant)  → role
(.*?)                    → content
```

`re.findall()` 在正则含多个捕获组时，会为每次匹配返回元组。因此：

```
matches = [("system", "你是助手"), ("user", "什么是 LoRA? ")]
```

列表推导式将元组解包，得到奖励模型可用的消息格式。`re.DOTALL` 只表示 `.` 可以匹配换行符，不影响返回元组的规则。

4.5.2 长度、思考格式与重复惩罚

回答长度在 20 到 800 字符之间时奖励 `+0.5`，否则 `-0.5`。若回答包含 `</think>`，代码会分离思考内容与最终答案，并奖励适中长度的思考内容和恰好一个结束标签。

```
answer = response
if '</think>' in response:
    thinking_content, answer_content = response.split('</think>', 1)
    answer = answer_content.strip()

rewards[i] -= rep_penalty(answer)
```

`rep_penalty` 将文本拆为 token，统计重复的连续 n 元组（默认 $n = 3$）。重复越严重，惩罚越接近上限 `cap=0.5`。它用于降低"同一句话反复出现"的回答分数。

最后：

```
score = reward_model.get_score(messages, answer)
rewards += reward_model_scores
```

奖励模型结合对话历史和最终答案，从相关性、帮助性、正确性等角度给出质量评分。

4.6 回答 token 的位置对齐与掩码

PPO 只能训练 response，不能把 prompt、PAD 或 EOS 后面的无效内容算入损失。因此需要一组位置和 mask。

```
full_mask = (gen_out != tokenizer.pad_token_id).long()
labels = gen_out[:, 1:].clone()
resp_labels = completion_ids
resp_idx = torch.arange(resp_labels.size(1), device=gen_out.device).unsqueeze(0)
logp_pos = prompt_lens.unsqueeze(1) - 1 + resp_idx
```

4.6.1 `full_mask` 与 `.long()`

```
full_mask = (gen_out != tokenizer.pad_token_id).long()
```

先得到布尔值：真实 token 为 `True`，PAD 为 `False`；`.long()` 将其转成 `torch.int64`：

```
[True, True, False] → [1, 1, 0]
```

该 mask 传入 Actor、Critic 和 Reference Model，使它们忽略 PAD。

4.6.2 为什么 `labels = gen_out[:, 1:]`

语言模型在位置 t 输出 logits，用来预测位置 $t+1$ 的 token。因此标签应向左移动一位：

```
gen_out: [10, 11, 12, 13, 51, 52, EOS]
labels:  [11, 12, 13, 51, 52, EOS]
```

4.6.3 `logp_pos`：找到预测 response 的 logits 位置

假设 prompt 长度 $P = 4$，response 是 `[51, 52, EOS]`：

```
完整位置:  0   1   2   3 | 4   5    6
token:     prompt …      | 51  52  EOS
```

第一个 response token 位于完整序列位置 4，但预测它的是位置 3 的 logits。因此：

```
resp_idx = [[0, 1, 2]]
logp_pos = [[3, 4, 5]]
```

即回答 token 0、1、2 分别对应完整序列的预测位置 3、4、5。

4.6.4 `gather(1, logp_pos)` 的含义

对二维张量，`gather(1, index)` 表示“每一行按照 `index` 指定的列取值”。例如：

```
values_seq = torch.tensor([
    [10, 11, 12, 13, 14, 15],
    [20, 21, 22, 23, 24, 25]
])
logp_pos = torch.tensor([
    [3, 4, 5],
    [2, 3, 4]
])

values_seq.gather(1, logp_pos)
# [[13, 14, 15], [22, 23, 24]]
```

在 PPO 中，它从完整序列的价值或概率中，只取 response 对应的位置。

4.6.5 response 有效长度和 mask

```
resp_pad_mask = rollout_result.completion_mask.to(args.device).bool()
resp_lengths = resp_pad_mask.sum(dim=1)
valid_resp = resp_lengths > 0
eos_mask = resp_labels.eq(tokenizer.eos_token_id) & resp_pad_mask
has_eos = eos_mask.any(dim=1)
eos_pos = torch.argmax(eos_mask.int(), dim=1)
resp_lengths = torch.where(has_eos, eos_pos + 1, resp_lengths).long().clamp(min=1)

resp_policy_mask = ((resp_idx < resp_lengths.unsqueeze(1)) &
resp_pad_mask).float()
resp_value_mask = resp_policy_mask.clone()
```

`resp_pad_mask` 屏蔽 batch 对齐时补出的 PAD；若发现 EOS，则 `resp_lengths` 截断到 EOS 并包含 EOS 本身。最终：

```
真实 response token：mask = 1
EOS 后 token / PAD：mask = 0
```

`resp_policy_mask` 用于 Actor 的策略损失，`resp_value_mask` 用于 Critic 的价值损失。当前代码中二者相同，但分开命名，便于未来采用不同的训练范围。

4.7 旧价值、参考概率与 GAE

以下部分使用 `torch.no_grad()`，因为它们只是 PPO 更新时的固定旧数据，不应建立梯度图：

```
with torch.no_grad():
    values_seq = critic_for_rollout(input_ids=gen_out, attention_mask=full_mask)
    old_resp_values = values_seq.gather(1, logp_pos) * resp_value_mask
```

`values_seq` 是 Critic 对完整序列的价值预测，形状 `[B, P+R]`；通过 `gather` 后只保留回答部分，得到 `[B, R]` 的 `old_resp_values`。`resp_value_mask` 将 PAD 与 EOS 后位置设为 0。

4.7.1 `ref_resp_logp`：参考模型对实际回答 token 的概率

原代码可以拆为：

```
ref_output = ref_model(input_ids=gen_out, attention_mask=full_mask)
ref_logits = ref_output.logits[:, :-1]                    # [B, P+R-1, V]
ref_log_probs = F.log_softmax(ref_logits, dim=-1)          # 对词表维度求 log 概率
```

```
all_token_logp = ref_log_probs.gather(2, labels.unsqueeze(-1)).squeeze(-1)
ref_resp_logp = all_token_logp.gather(1, logp_pos)         # [B, R]
```

第一次 `gather(2, ...)` 在词表维度中选出"实际发生的下一个 token"的 log 概率；第二次 `gather(1, logp_pos)` 在序列维度中只保留 response。例如它最终得到：

```
[log P_ref(51), log P_ref(52), log P_ref(EOS)]
```

这些值后续用于 KL 惩罚，防止当前 Actor 偏离固定 `ref_model` 太远。

4.7.2 将整条回答的奖励放到最后一个 token

```
token_rewards = torch.zeros_like(old_resp_logp)
last_idx = resp_lengths - 1
token_rewards[
    torch.arange(B, device=args.device)[valid_resp],
    last_idx[valid_resp]
] += rewards[valid_resp]
```

Reward Model 通常只能评价完整回答，因此每条回答只有一个总分。代码先将该总分放到回答最后一个有效 token：

```
response token:  [A, B, EOS]
总奖励:          1.5
token_rewards:  [0, 0, 1.5]
```

`valid_resp` 会跳过没有生成 token 的异常样本。

4.7.3 GAE：将最终奖励向前传播

```
for t in reversed(range(gen_len)):
    nv = old_resp_values[:, t + 1] if t < gen_len - 1 else 0.0
    delta = token_rewards[:, t] + args.gamma * nv - old_resp_values[:, t]
    lastgaelam = delta + args.gamma * args.lam * lastgaelam
    advs_rev.append(lastgaelam)

advantages = torch.stack(advs_rev[::-1], dim=1)
returns = advantages + old_resp_values
```

其中一步 TD 误差为：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE（Generalized Advantage Estimation）为：

$$A_t = \delta_t + \gamma\lambda A_{t+1}$$

优势 A_t 可以理解为“实际结果比 Critic 原本预期好多少”：

```
advantage > 0：该 token 对好结果有贡献，应提高其概率
advantage < 0：该 token 导致较差结果，应降低其概率
```

例如 response 是 `[A, B, EOS]`，总奖励为 `1.5`，旧价值为 `[0.2, 0.4, 0.7]`，且假设 `gamma=lambda=1`：

```
token_rewards: [0,   0,   1.5]
advantages:    [1.3, 1.1, 0.8]
returns:       [1.5, 1.5, 1.5]
```

这说明末尾奖励会通过 GAE 传回前面的 token。

4.7.4 Advantage 归一化

```
adv_mean = (advantages * resp_policy_mask).sum() /
resp_policy_mask.sum().clamp(min=1)
adv_var = ((advantages - adv_mean) ** 2 * resp_policy_mask).sum() /
resp_policy_mask.sum().clamp(min=1)
advantages = (advantages - adv_mean) * torch.rsqrt(adv_var + 1e-8) *
resp_policy_mask
```

其公式为：

$$A_{\text{norm}} = \frac{A - \mu}{\sqrt{\sigma^2 + 10^{-8}}}$$

`torch.rsqrt(x)` 是 $1/\sqrt{x}$。归一化只统计 mask 为 1 的有效 response token，最后再次乘 mask 使无效位置保持为 0。这样可使 PPO 更新的数值尺度更稳定。

4.8 PPO 更新：新旧策略、裁剪与 KL 约束

对同一批已生成回答，PPO 会进行多轮、小 batch 更新：

```
for ppo_epoch in range(args.ppo_update_iters):
    for i in range(0, B, mb_size):
        ...
```

当前 Actor 重新计算 response token 的概率 `mb_resp_logp`；当前 Critic 重新计算价值 `mb_resp_values`。随后：

```
log_ratio = mb_resp_logp - old_resp_logp[inds]
ratio = torch.exp(log_ratio)
```

对应：

$$r_t = \frac{\pi_{\text{new}}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

若旧概率为 `0.2`、新概率为 `0.3`，则 `ratio=1.5`。

4.8.1 Actor 的 clipped policy loss

```
policy_loss = (
    (torch.max(
        -advantages[inds] * ratio,
        -advantages[inds] * torch.clamp(
            ratio, 1.0 - args.clip_epsilon, 1.0 + args.clip_epsilon
        )
    ) * resp_policy_mask[inds]).sum()
    / resp_policy_mask[inds].sum().clamp(min=1)
)
```

当 advantage 为正时，Actor 应提高该 token 的概率；当 advantage 为负时，应降低其概率。`clip_epsilon` 限制 `ratio` 的有效变化范围，例如 `epsilon=0.2` 时约束在 `[0.8, 1.2]`，避免一次更新过猛。

4.8.2 Reference KL 惩罚

```
kl_ref_penalty = ...
policy_loss = policy_loss + args.kl_coef * kl_ref_penalty
```

它衡量当前 Actor 与固定 `ref_model` 的差异。其作用是：允许模型为获取更高奖励而改变，但防止模型为了迎合奖励规则而丢失原有的 SFT 对话能力。

4.8.3 Critic 的 value loss

```
value_loss = 0.5 * ...
```

Critic 学习让当前预测靠近前面计算的 `returns`：

$$V_{\text{new}}(s_t) \approx \text{return}_t$$

代码也对价值预测使用裁剪，避免 Critic 在一次 PPO 更新中变化过大。

4.8.4 早停与梯度更新

```
approx_kl = 0.5 * (log_ratio ** 2)
if approx_kl_val > args.early_stop_kl:
    stop_ppo = True
```

若当前 Actor 相比生成回答时的旧 Actor 改变过大，则提前停止本次 batch 后续 PPO 更新。多卡训练中仍让各卡完成 forward/backward，只将 loss 乘 0，避免有的进程提前退出造成 DDP 通信死锁。

最终总损失为：

```
loss = (policy_loss + args.vf_coef * value_loss + aux_loss) /
args.accumulation_steps
loss.backward()
```

其中 `policy_loss` 训练 Actor，`value_loss` 训练 Critic，`aux_loss` 是可选 MoE 辅助损失。达到梯度累积步数后：

```
clip_grad_norm_(actor_model.parameters(), args.grad_clip)
clip_grad_norm_(critic_model.parameters(), args.grad_clip)
actor_optimizer.step()
critic_optimizer.step()
actor_scheduler.step()
critic_scheduler.step()
actor_optimizer.zero_grad()
critic_optimizer.zero_grad()
```

这会裁剪两套模型的梯度、分别更新 Actor 和 Critic、更新学习率调度器并清空梯度。若一个 epoch 结束时尚未凑满梯度累积步数，代码还会执行一次相同的更新，避免最后累积的梯度被丢弃。

4.8.5 Mini-batch 与多轮 PPO 更新

```
mb_size = max(1, min(args.mini_batch_size, B))
for ppo_epoch in range(args.ppo_update_iters):
    b_inds = torch.randperm(B, device=args.device)
    for i in range(0, B, mb_size):
        inds = b_inds[i:i + mb_size]
```

`mb_size` 将一次 rollout 的 B 条回答分成更小的 mini-batch，减少显存压力。`torch.randperm(B)` 会产生 `0` 到 `B-1` 的随机排列，例如 `B=5` 时可能为 `[3, 0, 4, 1, 2]`；每一轮 PPO 更新都重新打乱样本顺序。

`args.ppo_update_iters` 表示同一批已生成回答可被重复训练多轮。每个 mini-batch 中，Critic 重新计算当前价值 `mb_resp_values`，Actor 重新计算当前回答 token 的 log 概率 `mb_resp_logp`。

4.8.6 `approx_kl`：新 Actor 相对旧 Actor 的变化量

```
approx_kl = (
    0.5 * (log_ratio ** 2) * resp_policy_mask[inds]
).sum() / resp_policy_mask[inds].sum().clamp(min=1)
```

其中：

$$\text{log_ratio} = \log \pi_{\text{new}}(a_t|s_t) - \log \pi_{\text{old}}(a_t|s_t)$$

所以：

$$\text{approx_kl} = \text{mean}_{\text{有效 token}} \left[\frac{1}{2} (\text{log_ratio})^2 \right]$$

它是新旧 Actor KL 距离的小变化近似，用于监控当前 Actor 是否改变过大，而不是直接加入损失。假设有效 token 的 `log_ratio` 为 `[0.2, -0.1, 0.3]`，则：

```
0.5 × [0.2², (-0.1)², 0.3²]
= [0.020, 0.005, 0.045]
approx_kl = (0.020 + 0.005 + 0.045) / 3 = 0.0233
```

若它超过 `args.early_stop_kl`，便设置 `stop_ppo=True`，停止对本轮 rollout 数据进行后续 PPO 更新。

4.8.7 `clipfrac`：有多少 token 触发了 PPO 裁剪

```
clipfrac = (
    (((ratio - 1.0).abs() > args.clip_epsilon).float()
     * resp_policy_mask[inds]).sum()
    / resp_policy_mask[inds].sum().clamp(min=1)
)
```

`ratio=\pi_{\text{new}}/\pi_{\text{old}}`。若 `clip_epsilon=0.2`，允许区间为 `[0.8, 1.2]`。上述代码统计有效 response token 中有多少比例的 `ratio` 超出此区间。

例如 `ratio=[1.10, 0.70, 1.25, 0.95]`，其中 `0.70` 与 `1.25` 超出范围；若四个位置均有效，则：

```
clipfrac = 2 / 4 = 0.5
```

它是监控指标，并不直接修改 loss。`clipfrac` 很高通常说明学习率过大、PPO 更新轮数过多，或 advantage 的变化尺度过大。

4.8.8 Actor 的策略损失：优势、PPO clip 与 Reference KL

```
policy_loss = (
    (torch.max(
        -advantages[inds] * ratio,
        -advantages[inds] * torch.clamp(
            ratio, 1.0 - args.clip_epsilon, 1.0 + args.clip_epsilon
        )
    ) * resp_policy_mask[inds]).sum()
    / resp_policy_mask[inds].sum().clamp(min=1)
    + args.kl_coef * kl_ref_penalty
)
```

PPO 原本希望最大化：

$$\min\left(A_t r_t,\ A_t\,\mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\right)$$

优化器执行最小化，因此代码写成数学等价的：

$$-\min(x, y) = \max(-x, -y)$$

`advantage>0` 表示该 token 对高奖励回答有贡献，Actor 应提高其概率；`advantage<0` 则应降低其概率。`clip` 确保概率比超过范围后不再获得额外优化收益，限制单次更新幅度。

`kl_ref_penalty` 用于限制当前 Actor 偏离固定参考模型：

```
d = ref_resp_logp[inds] - mb_resp_logp
kl_ref_penalty = (
    (torch.exp(d) - d - 1.0) * resp_policy_mask[inds]
).sum() / resp_policy_mask[inds].sum().clamp(min=1)
```

对 $a \sim \pi_{\text{new}}$ 求期望时：

$$\mathbb{E}_{a\sim\pi_{\text{new}}}[e^d - d - 1] = D_{\text{KL}}(\pi_{\text{new}} \| \pi_{\text{ref}})$$

因此 `args.kl_coef * kl_ref_penalty` 是“偏离参考 SFT 模型”的惩罚。单个 token 的 $e^d - d - 1$ 不是整个 KL；对当前策略采样的 token 求期望后才等于 KL。在 PPO 中使用的是 rollout 样本，因此它是近似的采样式约束。

4.8.9 Critic 的裁剪价值损失

```
value_loss = 0.5 * (
    torch.max(
        (mb_resp_values - returns[inds]) ** 2,
        (torch.clamp(
            mb_resp_values,
            old_resp_values[inds] - args.cliprange_value,
            old_resp_values[inds] + args.cliprange_value
        ) - returns[inds]) ** 2
```

```
    ) * resp_value_mask[inds]
).sum() / resp_value_mask[inds].sum().clamp(min=1)
```

其中：

```
mb_resp_values：当前 Critic 的价值预测 V_new
old_resp_values：更新前 Critic 的预测 V_old
returns：Critic 应学习的目标价值 R
cliprange_value：一次更新允许的价值变化范围 εv
```

对应公式：

$$L_{\text{value}} = \frac{1}{2}\,\text{mean}\left[\max\left((V_{\text{new}} - R)^2,\ (\text{clip}(V_{\text{new}}, V_{\text{old}} - \epsilon_v, V_{\text{old}} + \epsilon_v) - R)^2\right)\right]$$

例如 `V_old=0.4`、`R=1.5`、`cliprange_value=0.2`，则本轮允许范围是 `[0.2, 0.6]`。若 `V_new=0.9`，裁剪后是 `0.6`：

```
未裁剪误差：(0.9 - 1.5)² = 0.36
裁剪误差：  (0.6 - 1.5)² = 0.81
取较大误差：0.81
value_loss：  0.5 × 0.81 = 0.405
```

这样可以防止 Critic 在一次 PPO 更新中相对旧价值预测跳动过大。`resp_value_mask` 会屏蔽 PAD 和 EOS 后的无效位置。

4.9 PPO 一句话总结

```
Actor 先回答；Reward Model 打分；Critic 判断结果是否超出预期；
高优势 token 的概率上升、低优势 token 的概率下降；
PPO clip 限制更新步幅，Reference KL 限制模型偏离原 SFT 模型的程度。
```

5. GRPO：组相对策略优化

5.1 为什么需要 GRPO

SFT 让模型模仿示范答案，但面对同一个问题，往往有多个可行回答，仅靠模仿不容易直接利用“哪个回答更好”的评价。强化学习允许先生成回答，再通过奖励调整生成策略。

前面的 PPO 使用 Critic 预测价值，并通过 GAE 计算优势。Critic 本身需要训练，会增加模型、显存和计算开销，而且价值估计的质量会影响策略更新。GRPO（Group Relative Policy Optimization）改为：对同一个问题生成一组回答，用组内奖励的均值作为比较基准，不再训练 Critic。

```
PPO：  回答 → 奖励 + Critic 价值预测 → GAE 优势 → 更新 Actor
```

GRPO：同题多条回答 → 各自奖励 → 组内比较得到优势 → 更新策略模型

GRPO 省去了 Critic，但仍需要生成多条回答并评价它们，因此并不是没有采样成本。奖励规则也只是一种评价标准，模型会优化这套标准，规则得分高不必然等于真实回答质量高。

5.2 理论

5.2.1 奖励和优势是两个不同的量

设一批有 B 个问题，每题生成 G 条回答，共 $N = B \times G$ 条；回答补齐后的长度为 R。

对问题 q_i 的第 j 条回答 $o_{i,j}$，奖励为 $r_{i,j}$。本项目使用规则奖励与奖励模型评分之和：

$$r_{i,j} = r_{\text{长度}} + r_{\text{思考格式}} - p_{\text{重复}} + r_{\text{奖励模型}}$$

奖励回答"得了多少分"，优势回答"相对同题其他回答，好了多少"。组内标准化为：

$$\mu_i = \frac{1}{G}\sum_{j=1}^{G} r_{i,j}, \qquad \sigma_i = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_{i,j}-\mu_i)^2}$$

$$A_{i,j} = \frac{r_{i,j} - \mu_i}{\sigma_i + 10^{-4}}$$

正优势意味着高于组内平均，负优势意味着低于组内平均。本实现将一条回答的优势共享给它的所有有效 token，不通过 Critic 区分各个 token 的价值。

5.2.2 当前策略、采样策略和参考策略

符号/对象	作用	是否参与本次反向传播
π_θ，`model`	正在训练的当前策略	是
π_{old}，rollout 记录的概率	生成这批回答时的采样策略	否
π_{ref}，`ref_model`	提供固定参考基准	否
`reward_model`	对回答评分	否

对于已采样回答中的 token，令 s_t 表示问题与此前已生成的 token，概率比为：

$$\rho_t = \frac{\pi_\theta(o_t \mid s_t)}{\pi_{\text{old}}(o_t \mid s_t)} = \exp(\log \pi_\theta(o_t \mid s_t) - \log \pi_{\text{old}}(o_t \mid s_t))$$

例如旧概率为 0.2、当前概率为 0.3，概率比就是 1.5。采样策略与参考策略职责不同，不能混为一谈。

5.2.3 裁剪策略目标与 KL 约束

普通分支使用逐 token 的裁剪目标：

$$\ell_t = -\min\left(\rho_t A, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A\right) + \beta k_t$$

其中：

$$d_t = \log \pi_{\text{ref}}(o_t \mid s_t) - \log \pi_\theta(o_t \mid s_t), \qquad k_t = e^{d_t} - d_t - 1$$

最小化损失会推动正优势回答的已采样 token 概率提高，负优势回答的概率降低。裁剪限制过大策略变化带来的额外收益，但不是对更新后概率比的硬性范围保证。

k_t 是非负的采样式 KL 估计项，概率相同时为 0；它不是对整个词表直接求和计算的精确 KL。在当前策略下取期望时对应 $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$；这里使用 rollout 样本，因此要注意采样策略可能与当前策略不同。

本实现先对每条回答的有效 token 损失取平均，再对回答取平均：

$$L_{\text{policy}} = \frac{1}{N} \sum_{n=1}^{N} \frac{\sum_t m_{n,t} \ell_{n,t}}{\max(1, \sum_t m_{n,t})}$$

因此每条回答在最终平均中权重相同，长回答不会仅因 token 更多而占更大的总权重。

5.3 代码分析

对应源码：train_grpo.py。以下沿用两个问题、每题两条回答，即 $B = 2, G = 2, N = 4$。

5.3.1 Rollout：生成回答并记录旧概率

```
prompt_inputs = tokenizer(
    prompts, return_tensors="pt", padding=True,
    return_token_type_ids=False, padding_side="left",
    add_special_tokens=False
).to(args.device)

rollout_result = rollout_engine.rollout(
    prompt_ids=prompt_inputs["input_ids"],
    attention_mask=prompt_inputs["attention_mask"],
    num_generations=args.num_generations,
    max_new_tokens=args.max_gen_len,
    temperature=0.8,
)
old_per_token_logps = rollout_result.per_token_logps.to(args.device).detach()
```

问题先进行左侧补齐；若设置 `max_seq_len`，只保留输入的最后这些位置。`detach()` 将采样概率当作固定数据。

```
问题 A → A1、A2
问题 B → B1、B2

completions:          [A1, A2, B1, B2]，回答文本
completion_ids:       [4, R]，回答 token
outputs:              [4, 完整序列长度]，问题 + 回答
old_per_token_logps:  [4, R]，采样策略的逐 token log 概率
```

5.3.2 `calculate_rewards`：每条回答独立打分

```
rewards = torch.zeros(len(responses), device=args.device)
for i in range(len(prompts)):
    for j in range(args.num_generations):
        response_idx = i * args.num_generations + j
        prompt = prompts[i]
        response = responses[response_idx]
```

索引配对为：

i	j	response_idx	对应数据
0	0	0	问题 A + A1
0	1	1	问题 A + A2
1	0	2	问题 B + B1
1	1	3	问题 B + B2

每次循环先用正则将 prompt 的对话标记解析为 `messages`，然后执行：

```
整条回答去首尾空白后的字符数在 20～800：+0.5，否则 -0.5
    ↓
如果包含 </think>：
    按第一个 </think> 分成思考部分和最终答案
    思考部分字符数在 20～300：+1.0，否则 -0.5
    </think> 恰好一个：+0.25，否则 -0.25
    answer 改为最终答案
否则：answer 保持为整条回答
    ↓
扣除 rep_penalty(answer)
    ↓
reward_model.get_score(messages, answer)，将结果追加到列表
```

这里的长度是字符数，不是 token 数。思考部分长度包含可能存在的开头 `<think>` 标签；代码没有验证这个开头标签。重复惩罚的细则取决于 `rep_penalty`。

最后统一相加，假设得到下面的示例分数：

```
                            A1      A2      B1      B2
规则奖励 rewards          [1.55,   0.50,  -0.50,   1.75]
奖励模型评分              [2.00,   1.00,   0.30,   3.00]
                           ----------------------------
逐元素相加后的 rewards    [3.55,   1.50,  -0.20,   4.75]
```

```
reward_model_scores = torch.tensor(reward_model_scores, device=args.device)
```

```
rewards += reward_model_scores
```

`+=` 是对应位置相加，不是将所有元素合并成一个数。因此结果形状仍是 `[4]`，每条回答一个分数；只有 `rewards.sum()` 才会将它们求和成标量。

本项目 PPO 的单条回答评分规则与此相同，主要是这里使用双重循环处理一题多答。GRPO 的组内比较发生在下面的优势计算阶段，而不在打分函数里。

5.3.3 分组、求均值和标准差，再计算优势

```
grouped_rewards = rewards.view(-1, args.num_generations)
mean_r = grouped_rewards.mean(dim=1).repeat_interleave(args.num_generations)
std_r = grouped_rewards.std(dim=1,
unbiased=False).repeat_interleave(args.num_generations)
advantages = (rewards - mean_r) / (std_r + 1e-4)
```

第一步，`view(-1, 2)` 中的 `-1` 自动推算为 `4 / 2 = 2`：

```
[3.55, 1.50, -0.20, 4.75]             [4]
                ↓ view(-1, 2)
问题 A: [ 3.55, 1.50 ]
问题 B: [-0.20, 4.75 ]                [2, 2]
```

第二步，`dim=1` 沿每行求均值：

```
A 组均值: (3.55 + 1.50) / 2 = 2.525
B 组均值: (-0.20 + 4.75) / 2 = 2.275

[2.525, 2.275]                        [2]
        ↓ repeat_interleave(2)
[2.525, 2.525, 2.275, 2.275]          [4]
 └── A 组 ──┘  └── B 组 ──┘
```

`repeat_interleave(2)` 是将每个元素连续重复两次，让每条回答对应上自己组的均值。它与 `repeat(2)` 不同：

```
x = torch.tensor([10, 20])
x.repeat_interleave(2)  # [10, 10, 20, 20]
x.repeat(2)             # [10, 20, 10, 20]
```

第三步，`unbiased=False` 表示方差除以组内数量 G，不是 $G-1$：

```
A 组标准差：sqrt((1.025² + (-1.025)²) / 2) = 1.025
B 组标准差：sqrt(((-2.475)² + 2.475²) / 2) = 2.475

std_r = [1.025, 1.025, 2.475, 2.475]
```

第四步，逐位置计算：

```
                        A1       A2        B1       B2
rewards               3.550    1.500    -0.200    4.750
减 mean_r             2.525    2.525     2.275    2.275
差值                 +1.025   -1.025    -2.475   +2.475
除以 std_r + 1e-4    1.0251   1.0251    2.4751   2.4751
advantages            ≈ +1     ≈ -1      ≈ -1     ≈ +1
```

比较只发生在同题内部。两条回答分数不同时，忽略稳定项，标准化优势恰好是 `+1` 和 `-1`；考虑稳定项后，分差明显大于 `1e-4` 时才接近这两个值。分数相同时，两者优势都是 0。若 `G=1`，这种组内算法也只能得到 0 优势，因此无法提供奖励驱动的策略信号。

5.3.4 当前模型与参考模型的逐 token 概率

```
logp_pos = prompt_lens.unsqueeze(1) - 1 + torch.arange(
    completion_ids.size(1), device=args.device
).unsqueeze(0)

res = model_unwrapped(outputs, attention_mask=full_mask)
per_token_logps = F.log_softmax(res.logits[:, :-1, :], dim=-1).gather(
    2, outputs[:, 1:].unsqueeze(-1)
).squeeze(-1).gather(1, logp_pos)
```

自回归模型用当前位置预测下一个 token，因此回答的预测位置要减 1：

```
序列位置：  0   1   2   3   4
token：    P1  P2  P3  A1  A2
                   │   │
                   │   └─位置 3 的 logits 预测 A2
                   └─────位置 2 的 logits 预测 A1
```

这里要求引擎返回的 `prompt_lens` 与完整序列中的实际回答起点一致，包括补齐布局。

`log_softmax` 得到整个词表的 log 概率；第一个 `gather` 取实际下一个 token 的 log 概率；第二个 `gather` 只保留回答位置。结果为 `[N, R]`，保留梯度。参考模型用同样的计算方式，但放在 `torch.no_grad()` 中，不参与反向传播。

5.3.5 有效回答 mask

代码结合 completion_mask 与第一个有效 EOS 位置构造最终掩码：

```
回答 token:  A1 A2 A3 EOS PAD PAD
最终 mask:    1  1  1   1   0   0
```

第一个 EOS 参与训练，其后位置不参与；没有 EOS 时保留所有有效回答 token。completion_pad_mask 进一步排除补齐位置。

5.3.6 概率比、KL 与损失分支

```
kl_div = ref_per_token_logps - per_token_logps
per_token_kl = torch.exp(kl_div) - kl_div - 1
ratio = torch.exp(per_token_logps - old_per_token_logps)

clipped_ratio = torch.clamp(ratio, 1 - args.epsilon, 1 + args.epsilon)
per_token_loss1 = ratio * advantages.unsqueeze(1)
per_token_loss2 = clipped_ratio * advantages.unsqueeze(1)
per_token_loss = -(torch.min(per_token_loss1, per_token_loss2)
                   - args.beta * per_token_kl)
```

advantages.unsqueeze(1) 将 [4] 变成 [4, 1]，随后广播到 [4, R]，即每条回答的所有有效 token 共享该回答的优势。

loss_type == "cispo" 是另一种实现分支：

```
clamped_ratio = torch.clamp(ratio, max=args.epsilon_high).detach()
per_token_loss = -(clamped_ratio * advantages.unsqueeze(1) * per_token_logps
                   - args.beta * per_token_kl)
```

它把截断后的概率比当作固定权重，策略项的梯度通过 per_token_logps 传播。这里截断上限直接是 epsilon_high，不是 1 + epsilon_high，也不是普通分支的双侧裁剪。

5.3.7 汇总损失、更新、记录与同步

```
policy_loss = ((per_token_loss * completion_mask).sum(dim=1)
               / completion_mask.sum(dim=1).clamp(min=1)).mean()
loss = (policy_loss + aux_loss) / args.accumulation_steps
loss.backward()
```

达到梯度累积间隔后，进行梯度裁剪、optimizer.step()、scheduler.step() 和梯度清空。MoE 模型另外加入辅助损失。

日志与同步有几个实现细节：

- 日志 `policy_loss_val` 从总 `loss` 还原，包含 `aux_loss`，不完全是纯策略损失。
- 日志 `KL_ref` 记录 `ref_logp - current_logp` 的平均值，与训练使用的 `exp(d)-d-1` 不同，可能为负。
- `rollout_engine.update_policy(model)` 在保存间隔或最后一步调用，采样引擎可能落后于正在训练的模型。
- epoch 末尾会更新不足一个累积周期的剩余梯度；这次尾部更新发生在循环内保存和同步之后，因此保存与同步的权重可能尚未包含它。

6. DPO：直接偏好优化

6.1 为什么需要 DPO

如果已经有"同一个问题下，回答 A 比回答 B 更好"的偏好数据，就可以直接学习这种排序关系。相比先训练奖励模型、再在线采样并进行强化学习，这份 DPO 训练流程更直接：读取固定的偏好回答对，计算它们的概率并更新模型。

DPO（Direct Preference Optimization）不需要在这个训练循环中额外调用奖励模型，也不需要 Critic、GAE 或组内标准化。它需要高质量的偏好数据；数据中的偏好错误或覆盖不足，也会影响训练结果。

```
SFT:  给定示范答案 → 学习生成它
DPO:  给定 chosen / rejected 回答对 → 学习更偏向 chosen
GRPO: 当前策略生成多条回答 → 评分 → 学习组内表现较好的回答
```

`chosen` 是一对回答中被偏好的答案，不意味着绝对正确；`rejected` 也不一定完全错误。

6.2 理论

6.2.1 从偏好标签到概率目标

设问题为 x，被偏好的回答为 y_w，另一条为 y_l。训练目标是让当前模型提高 y_w 相对于 y_l 的概率比，并以冻结参考模型作为基准。

完整回答的条件概率为逐 token 条件概率的乘积，取对数后为求和：

$$\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t})$$

这解释了代码为什么对有效回答 token 的 log 概率求和，而不是求平均。

6.2.2 为什么减去参考模型的偏好差

考虑带参考策略 KL 约束的奖励优化目标：

$$\max_\pi \ \mathbb{E}_{y\sim\pi(\cdot|x)}[r(x,y)] - \beta D_{\mathrm{KL}}(\pi(\cdot \mid x) \| \pi_{\mathrm{ref}}(\cdot \mid x))$$

其最优策略与奖励具有如下关系：

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + C(x)$$

其中 $C(x)$ 只与问题有关。若用 sigmoid 描述两条回答的偏好概率：

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

则同题的 $C(x)$ 在相减时消失。用待训练策略参数化上述关系，可以直接构造 DPO 的偏好损失，无须显式拟合并调用独立奖励模型。

6.2.3 DPO 损失

定义当前策略和参考策略的偏好差：

$$\Delta_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)$$

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x)$$

损失为：

$$L_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(\beta(\Delta_\theta - \Delta_{\text{ref}}) \right) \right]$$

其中 $\sigma(z) = 1/(1 + e^{-z})$。偏好改善量 $\Delta_\theta - \Delta_{\text{ref}}$ 越大，损失越小。

`beta` 在推导中来自参考策略约束，在代码中直接缩放 sigmoid 的输入，同时影响梯度尺度和饱和程度，不能仅理解为学习率。代码没有像 GRPO 那样额外加一项显式逐 token KL；参考策略已经进入偏好损失。

这个目标鼓励提高 chosen 相对 rejected 的概率比，不保证它们的绝对概率一定分别上升和下降。当当前模型与参考模型输出完全相同时，改善量为 0，损失为 $-\log(0.5) \approx 0.693$，但梯度并不为 0，仍然可以学习。

6.3 代码分析

对应源码：train_dpo.py，数据处理见 lm_dataset.py 中的 `DPODataset`。

6.3.1 数据集构造输入、标签和掩码

仍使用两个问题，每题两条回答，但此时答案由数据集提供并标注偏好：

```
问题 A → A+（chosen）、A-（rejected）
问题 B → B+（chosen）、B-（rejected）
```

`DPODataset` 将 chosen 与 rejected 的对话应用聊天模板、分词、截断并补齐，再生成：

```
x_chosen = torch.tensor(chosen_input_ids[:-1], dtype=torch.long)
y_chosen = torch.tensor(chosen_input_ids[1:], dtype=torch.long)
mask_chosen = torch.tensor(chosen_loss_mask[1:], dtype=torch.long)
```

rejected 的处理相同。`x` 与 `y` 已经错开一位：

```
完整序列：  P1 P2 A1 A2 EOS
x：          P1 P2 A1 A2
```

```
y:          P2 A1 A2 EOS
```

因此训练函数不需要再次执行 GRPO 中的 `logits[:, :-1]` 对齐。`generate_loss_mask` 标记 assistant 内容及其结束标记，随后同步右移成目标掩码；多轮对话中会标记多个 assistant 片段，不是自动只取最后一轮回答。

6.3.2 拼接批次：先 chosen，再 rejected

```
x = torch.cat([x_chosen, x_rejected], dim=0)
y = torch.cat([y_chosen, y_rejected], dim=0)
mask = torch.cat([mask_chosen, mask_rejected], dim=0)
```

令输入长度为 L，拼接顺序为：

```
x_chosen [2, L]       x_rejected [2, L]
[问题A + A+]          [问题A + A-]
[问题B + B+]          [问题B + B-]
              ↓ cat(dim=0)
x [4, L]:
第 0 行：问题A + A+  ┐
第 1 行：问题B + B+  ┘ 前半段 chosen
第 2 行：问题A + A-  ┐
第 3 行：问题B + B-  ┘ 后半段 rejected
```

注意它与 GRPO 的 `[A1, A2, B1, B2]` 排列不同；后续切分必须和这里的顺序一致。

6.3.3 两个模型前向计算

```
with torch.no_grad():
    ref_outputs = ref_model(x)
    ref_logits = ref_outputs.logits
ref_log_probs = logits_to_log_probs(ref_logits, y)

outputs = model(x)
policy_log_probs = logits_to_log_probs(outputs.logits, y)
```

当前模型与参考模型分别评价同一批已有回答，不在这里生成回答。参考模型通过 `eval()` 和 `requires_grad_(False)` 冻结；当前模型保留梯度。

6.3.4 `logits_to_log_probs`：从词表中选出目标 token

```
def logits_to_log_probs(logits, labels):
    log_probs = F.log_softmax(logits, dim=2)
```

```
    log_probs_per_token = torch.gather(
        log_probs, dim=2, index=labels.unsqueeze(2)
    ).squeeze(-1)
    return log_probs_per_token
```

假设某个位置的实际目标为“猫”，模型概率为猫 0.6、狗 0.3、其他 0.1，则 `gather` 取出的是 `log(0.6)`，约为 -0.511。

```
logits:                    [4, L, V]，V 为词表大小
log_softmax:               [4, L, V]
labels.unsqueeze(2):       [4, L, 1]
gather:                    [4, L, 1]
squeeze(-1):               [4, L]
```

这里 `dim=2` 是词表维度，`gather` 按目标 token 编号选择对应分数。

6.3.5 用 mask 筛选，再按序列求和

```
ref_log_probs = (ref_log_probs * mask).sum(dim=1)
policy_log_probs = (policy_log_probs * mask).sum(dim=1)
```

示意如下：

```
位置:            问题部分       回答部分       PAD
log 概率:      -0.2  -0.3     -0.5  -0.8     -1.0
mask:             0     0        1     1        0
相乘:             0     0     -0.5  -0.8        0
                              ↓ sum
                             -1.3

整批形状：[4, L] → [4]
```

该 mask 控制哪些目标位置计入损失，与控制模型关注哪些输入位置的 attention mask 不是同一个概念。

6.3.6 切分回答并计算偏好差

```
batch_size = ref_log_probs.shape[0]  # 这里是 4，不是问题数 2
chosen_ref_log_probs = ref_log_probs[:batch_size // 2]
reject_ref_log_probs = ref_log_probs[batch_size // 2:]
chosen_policy_log_probs = policy_log_probs[:batch_size // 2]
reject_policy_log_probs = policy_log_probs[batch_size // 2:]

pi_logratios = chosen_policy_log_probs - reject_policy_log_probs
```

```
ref_logratios = chosen_ref_log_probs - reject_ref_log_probs
logits = pi_logratios - ref_logratios
```

用假设的序列 log 概率进行可视化，注意 log 概率越大、越接近 0，概率越大：

回答	当前模型	参考模型
A+	-2	-3
B+	-5	-4
A-	-4	-4
B-	-3	-3

```
                                问题 A          问题 B
当前 chosen - rejected:        -2 - (-4)       -5 - (-3)
pi_logratios:                      2               -2

参考 chosen - rejected:        -3 - (-4)       -4 - (-3)
ref_logratios:                     1               -1
                                 ↓ 相减
logits:                          +1               -1
```

A 的偏好相对参考模型改善，B 的偏好相对参考模型变差。函数里的这个 `logits` 形状为 `[2]`，是每个回答对的偏好改善量，不是模型输出的词表 logits。

6.3.7 `logsigmoid` 转换为训练损失

```
loss = -F.logsigmoid(beta * logits)
return loss.mean()
```

`F.logsigmoid` 用数值稳定的方式计算 $\log \sigma(z)$。取脚本参数默认值 `beta=0.15`：

```
问题 A：改善量 +1 → beta × logits = +0.15 → loss ≈ 0.621
问题 B：改善量 -1 → beta × logits = -0.15 → loss ≈ 0.771
                                                  ↓ mean
                                              loss ≈ 0.696
```

6.3.8 反向传播与训练基础设施

```
dpo_loss_val = dpo_loss(ref_log_probs, policy_log_probs, mask, beta=beta)
loss = (dpo_loss_val + outputs.aux_loss) / args.accumulation_steps
scaler.scale(loss).backward()
```

达到累积间隔后，执行 `scaler.unscale_`、梯度裁剪、`scaler.step`、`scaler.update` 和梯度清空。`GradScaler` 在配置为 FP16 时启用；辅助损失用于模型内部的额外训练目标，例如 MoE 辅助项。

主程序负责解析参数、初始化分布式环境和随机种子、加载模型与冻结参考模型、构造 `DPODataset`、配置优化器和混合精度、恢复断点、按需编译和包装 DDP，最后进入 epoch 循环。

恢复训练时，`SkipBatchSampler` 负责跳过已完成批次；`enumerate(..., start=...)` 只负责步骤编号。epoch 末尾会处理未满一个累积周期的梯度，但循环内最后一次保存发生在这次尾部更新之前。

6.3.9 与 PPO、GRPO 的对应关系

对比项	本项目 PPO	本项目 GRPO	本项目 DPO
回答来源	rollout 生成	每题 rollout 生成一组	数据集中的偏好对
训练信号	规则与奖励模型评分	规则与奖励模型评分	chosen / rejected 标签
优势计算	Critic + GAE	同题奖励标准化	不显式计算 advantage
Critic	需要	不需要	不需要
参考策略作用	loss 中显式 KL 约束	loss 中显式 KL 约束	进入偏好差公式
当前优化单位	token 优势	回答优势共享给 token	一对回答的相对概率

```
GRPO：回答 → 奖励 [4] → 分组 [2, 2] → 优势 [4] → token 损失
DPO：偏好对 → token log 概率 [4, L] → 序列分数 [4]
            → chosen / rejected 分别 [2] → 偏好损失 [2] → 平均
```

7. 知识蒸馏：让学生学习教师的预测分布

7.1 为什么需要知识蒸馏

较大的模型往往有较好的预测能力，但部署时可能受到显存、推理速度和计算成本限制。知识蒸馏（Knowledge Distillation）使用教师模型的预测分布来指导学生模型，让学生学习教师对不同候选 token 的判断。学生通常更小，但这段训练代码并不要求两者一定有大小差异。

普通监督训练使用真实标签：例如“我喜欢吃”后面的目标 token 是“苹果”。交叉熵要求提高“苹果”的概率，但标签本身没有告诉学生“香蕉也比较合理，石头很不合理”。教师的概率分布可以提供这些额外信息。

下一个 token	教师概率	学生概率
苹果	0.7	0.3
香蕉	0.2	0.3
石头	0.1	0.4

```
真实标签：下一个 token 是“苹果”
    ↓ 交叉熵 CE
学生提高“苹果”的概率

教师分布：[苹果 0.7，香蕉 0.2，石头 0.1]
    ↓ 蒸馏 KL
学生学习整个候选分布的相对关系
```

这里属于输出分布蒸馏，没有比较教师与学生的隐藏状态或注意力矩阵。教师也可能犯错，所以代码允许同时使用真实标签损失和教师蒸馏损失。

与前面训练方法的关系为：

方法	直接学习的信号
SFT	数据中的示范 token
DPO	chosen 相对 rejected 的偏好
GRPO	同题多条回答的相对奖励
本节蒸馏	教师的逐 token 词表分布，可结合真实标签

7.2 理论

7.2.1 硬标签与软标签

设某个预测位置的真实 token 编号是 y，学生词表 logits 为 z_s，教师 logits 为 z_t。

真实标签可以理解为 one-hot 分布：正确 token 为 1，其余为 0，称为硬标签。教师给出的概率分布允许多个 token 拥有非零概率，称为软标签。

普通交叉熵为：

$$L_{\mathrm{CE}} = -\log p_s(y), \qquad p_s = \mathrm{softmax}(z_s)$$

例如学生给“苹果”的概率为 0.3，真实标签是“苹果”，则该位置交叉熵为 $-\log(0.3) \approx 1.204$。

7.2.2 温度：调整分布的集中程度

蒸馏时用温度 $T > 0$ 调整教师与学生的分布：

$$p_t^{(T)}(v) = \frac{\exp(z_{t,v}/T)}{\sum_u \exp(z_{t,u}/T)}, \qquad p_s^{(T)}(v) = \frac{\exp(z_{s,v}/T)}{\sum_u \exp(z_{s,u}/T)}$$

温度低时分布更集中，最高分候选更加突出；温度高时分布更平缓，非首选候选的相对关系更容易参与学习。

```
logits = [2, 1, 0]
```

```
T = 1: softmax([2, 1, 0])       ≈ [0.665, 0.245, 0.090]
T = 2: softmax([1, 0.5, 0])     ≈ [0.506, 0.307, 0.186]
```

这里的温度用于训练损失中的概率转换，并没有在这个循环里执行带温度的文本采样。

7.2.3 KL 蒸馏损失及其方向

本实现计算：

$$L_{\text{distill}} = T^2 D_{\text{KL}}(p_t^{(T)} \| p_s^{(T)}) = T^2 \sum_v p_t^{(T)}(v) \log \frac{p_t^{(T)}(v)}{p_s^{(T)}(v)}$$

教师固定，学生更新；最小化这个损失会让学生分布接近教师分布。教师与学生完全一致时，KL 为 0。

以 $T = 1$ 和前面的概率为例：

```
教师: [0.7, 0.2, 0.1]
学生: [0.3, 0.3, 0.4]

KL = 0.7 × log(0.7 / 0.3)
   + 0.2 × log(0.2 / 0.3)
   + 0.1 × log(0.1 / 0.4)
   ≈ 0.373
```

PyTorch 这里的调用是 `F.kl_div(student_log_probs, teacher_probs)`：第一个参数要求学生的 **log 概率**，第二个参数默认要求教师的 **概率**。虽然学生在参数列表中排在前面，数学方向仍然是 $D_{\text{KL}}(\text{教师} \| \text{学生})$。

乘 T^2 是常用的梯度尺度补偿。未补偿的 KL 对学生 logits 的梯度为 $(p_s^{(T)} - p_t^{(T)})/T$；高温下概率差也会缩小，因此用 T^2 补偿高温造成的梯度减弱。这不是说任意温度下梯度都严格保持不变。

7.2.4 为什么有的教程用软标签交叉熵，代码却用 KL

有的教程将蒸馏分成两部分：教师软标签的交叉熵 L_{soft}，以及真实硬标签的交叉熵 L_{hard}。这与本代码的"蒸馏 KL + 真实标签 CE"并不矛盾，需要分别比较损失形式与缩放系数。

先对齐符号：

教程符号	含义	本代码对应
p^T	教师在温度 T 下的概率分布	`teacher_probs`
q^T	学生在温度 T 下的概率分布	`student_log_probs.exp()`
c	真实目标的 one-hot 分布	`shift_labels` 表示的类别
L_{soft}	教师软标签与学生预测的交叉熵	与未乘 T^2 的 KL 相差教师熵
L_{hard}	真实标签与学生预测的交叉熵	`ce_loss_raw`

第一步：写出软标签交叉熵。

$$L_{\text{soft}} = H(p^T, q^T) = -\sum_j p_j^T \log q_j^T$$

教师给一个 token 的概率越大，这个 token 对软交叉熵的贡献权重就越大。

第二步：展开代码使用的 KL。

为方便推导，暂时省略温度上标：

$$D_{\text{KL}}(p\|q) = \sum_j p_j \log \frac{p_j}{q_j} = \sum_j p_j \log p_j - \sum_j p_j \log q_j$$

教师分布的熵定义为：

$$H(p) = -\sum_j p_j \log p_j$$

因此：

$$\boxed{D_{\text{KL}}(p\|q) = L_{\text{soft}} - H(p)}$$

第三步：教师冻结，所以相差项不影响学生梯度。

对于给定输入与固定温度，教师分布 p 不依赖学生参数 θ_s，因此：

$$\nabla_{\theta_s} H(p) = 0$$

$$\boxed{\nabla_{\theta_s} D_{\text{KL}}(p\|q) = \nabla_{\theta_s} L_{\text{soft}}}$$

也就是说，在相同温度、位置筛选、归约方式和损失权重下，两者对学生提供相同的梯度，但损失数值不同。
教师熵可以随输入变化，并不是整个数据集都共享同一个数；关键是它对学生参数而言是常数。

```
当学生完全匹配教师，即 q = p：

KL：              D_KL(p || p) = 0
软交叉熵：        H(p, p) = H(p)，通常不为 0

两者都已达到针对该教师分布的最小值。
不能仅用损失是否接近 0 来直接比较这两种实现。
```

第四步：硬标签部分与代码直接对应。

教程写作：

$$L_{\text{hard}} = -\sum_j c_j \log q_j^1$$

例如真实 token 是"苹果"，$c = [1, 0, 0]$，就只留下：

$$L_{\text{hard}} = -\log q^1(\text{苹果})$$

代码的 `F.cross_entropy(student_logits, shift_labels, ...)` 正是在计算它。输入没有除以蒸馏温度，使用的是 $T = 1$ 的学生分布。代码用整数类别标签表达目标，不需要真的构造 one-hot 张量。

第五步：单独检查 T^2，不能把它与 KL/CE 等价混为一谈。

代码中的 `distillation_loss` 返回的是 $T^2 D_{\mathrm{KL}}$，而不只是 KL。忽略 MoE 与梯度累积，实际总损失为：

$$L_{\text{code}} = \alpha L_{\text{hard}} + (1-\alpha) T^2 D_{\mathrm{KL}}(p^T \| q^T)$$

若改成软标签交叉熵，要保留相同的学生梯度，可以写为：

$$L_{\text{equiv}} = \alpha L_{\text{hard}} + (1-\alpha) T^2 L_{\text{soft}}$$

两者相差 $(1-\alpha) T^2 H(p^T)$，在温度与权重固定、教师冻结时仍与学生参数无关。

```
# 与当前 KL 蒸馏项具有相同学生梯度的软交叉熵写法
# 输入都是筛选后的 [M, V]，其中 M > 0
with torch.no_grad():
    teacher_probs = F.softmax(teacher_logits / temperature, dim=-1)

student_log_probs = F.log_softmax(student_logits / temperature, dim=-1)
soft_ce = -(teacher_probs * student_log_probs).sum(dim=-1).mean()
distill_loss = temperature ** 2 * soft_ce
```

截图只展示了 L_{soft} 的定义，没有展示总损失中的温度补偿。如果完整实现也不乘 T^2，又保持相同的混合权重，那么蒸馏项相对硬标签项的梯度尺度会不同；也可能由总损失系数吸收这个因子，需要看完整公式。

变化	是否影响学生梯度
同条件下把软标签 CE 换成 KL，教师冻结	不影响，只改变损失数值
增加或移除 T^2，其余权重不变	通常影响梯度尺度；$T = 1$ 时无区别
改变温度 T	改变目标与学生软分布，通常影响梯度
改变求和/平均方式或有效 token mask	可能改变梯度尺度或参与训练的位置

7.2.5 总损失和 MoE 辅助项

没有 MoE 时，总损失为：

$$L = \alpha L_{\text{CE}} + (1-\alpha) L_{\text{distill}}$$

`alpha`	使用的训练信号
1.0	只使用真实标签 CE
0.5	CE 与蒸馏损失各乘 0.5
0.0	只使用教师蒸馏损失

通常将 `alpha` 设在 `[0, 1]`。权重相同不意味着两种损失的实际梯度贡献相同，还取决于各自的尺度。

本代码先把 MoE 辅助损失加到 CE 中，因此开启 MoE 后实际是：

$$L = \alpha L_{\text{CE}} + \alpha L_{\text{aux}} + (1-\alpha) L_{\text{distill}}$$

`aux_loss` 是本项目的专家负载均衡损失，鼓励路由器避免长期集中使用少数专家。它已经在模型内部乘过 `router_aux_loss_coef`，各层再累加输出。普通 Dense 模型没有这项 MoE 损失。

这里特别要注意：`alpha=0` 不仅排除了真实标签 CE，也排除了 MoE 辅助损失。如果希望纯蒸馏仍保留专家均衡约束，需要将辅助项单独放在混合损失外；这不是当前代码的行为。

7.3 代码分析

对应源码：train_distillation.py。以下用 $B = 2$ 条输入、原始长度 $L = 5$、词表大小 V 说明形状变化。

7.3.1 `distillation_loss` 的完整数据流

```
def distillation_loss(student_logits, teacher_logits,
                      temperature=1.0, reduction='batchmean'):
    with torch.no_grad():
        teacher_probs = F.softmax(
            teacher_logits / temperature, dim=-1
        ).detach()

    student_log_probs = F.log_softmax(
        student_logits / temperature, dim=-1
    )

    kl = F.kl_div(student_log_probs, teacher_probs, reduction=reduction)
    return (temperature ** 2) * kl
```

```
教师 logits [M, V]                    学生 logits [M, V]
       ↓ 除以 T                              ↓ 除以 T
     softmax                             log_softmax
       ↓                                     ↓
教师概率 [M, V]                       学生 log 概率 [M, V]
不记录梯度                              保留梯度
       └──────────────────┬──────────────────┘
                          ↓ KL：逐词表项求和，再对 M 个位置平均
                        标量 KL
                          ↓ × T²
                      distill_loss
```

`M` 是有效预测位置的数量，由后面的 mask 筛选得到。这里 `batchmean` 除以传入张量的第一维大小 `M`，不是原始输入条数 `B`。`no_grad()` 内再 `detach()` 是额外明确切断梯度，通常属于重复保护。

7.3.2 冻结教师与读取数据

```
if teacher_model is not None:
    teacher_model.eval()
    teacher_model.requires_grad_(False)

for step, (input_ids, labels) in enumerate(loader, start=start_step + 1):
    input_ids = input_ids.to(args.device)
    labels = labels.to(args.device)
    loss_mask = (labels[..., 1:] != -100).float()
```

`eval()` 切换评估模式；`requires_grad_(False)` 冻结参数，两者不是同一个作用。`-100` 表示不计入监督损失的目标位置。

代码随后通过 `get_lr` 按全局训练进度设置优化器各参数组的学习率。`start_step` 控制步骤编号，实际跳过已训练数据由外部的数据加载逻辑负责。

7.3.3 自回归对齐：输入位置预测下一个 token

```
with autocast_ctx:
    res = model(input_ids)
    student_logits = res.logits[..., :-1, :].contiguous()

shift_labels = labels[..., 1:].contiguous()
```

```
完整输入：  P1  P2  A1  A2  EOS
预测位置：  P1  P2  A1  A2
预测目标：  P2  A1  A2  EOS

原始 logits：    [2, 5, V]
student_logits：[2, 4, V]，去掉最后位置
shift_labels：   [2, 4]，去掉最前位置
loss_mask：      [2, 4]
```

`...` 表示保留前面的维度；`.contiguous()` 将结果整理为连续存储，方便后续 `.view()`。学生前向保留梯度，`autocast_ctx` 控制混合精度。

7.3.4 教师前向与词表对齐

```
if teacher_model is not None:
    with torch.no_grad():
        teacher_logits = teacher_model(input_ids).logits[..., :-1, :].contiguous()
        vocab_size_student = student_logits.size(-1)
        teacher_logits = teacher_logits[..., :vocab_size_student]
```

教师与学生读取相同输入，评价相同的下一 token 预测任务。代码将教师词表截取到学生词表大小。

这个操作要求教师至少具有对应的词表范围，且保留范围内相同编号代表相同 token。仅把维度裁成相同大小，不能解决 tokenizer 编号语义不一致的问题。教师词表被裁剪后，后续 softmax 会在保留的候选范围内重新归一化。

7.3.5 CE：逐位置学习真实标签

```
loss_mask_flat = loss_mask.view(-1)
ce_loss = F.cross_entropy(
    student_logits.view(-1, student_logits.size(-1)),
    shift_labels.view(-1),
    ignore_index=-100,
    reduction='none'
)
ce_loss_raw = torch.sum(ce_loss * loss_mask_flat) / (loss_mask_flat.sum() + 1e-8)
```

```
student_logits [2, 4, V] → [8, V]
shift_labels   [2, 4]    → [8]
loss_mask      [2, 4]    → [8]
                           ↓ cross_entropy(reduction='none')
ce_loss                  [8]，每个预测位置一个损失
                           ↓ 有效位置求和 / 有效位置数量
ce_loss_raw              标量
```

`ignore_index=-100` 让这些标签位置被忽略。这里又乘了一次 mask，并用有效位置数量作为分母。CE 接收原始 logits，内部完成 log-softmax，不需要事先手动 softmax。它使用原始温度的 logits，没有应用蒸馏温度。

```
if lm_config_student.use_moe:
    ce_loss = ce_loss_raw + res.aux_loss
else:
    ce_loss = ce_loss_raw
```

此处 `ce_loss` 变量从“逐位置损失向量”变成“加入可选辅助损失的标量”，阅读时要留意这一变化。

7.3.6 KL：只比较有效位置的完整词表分布

```
distill_loss = distillation_loss(
    student_logits.view(-1, student_logits.size(-1))[loss_mask_flat == 1],
    teacher_logits.view(-1, teacher_logits.size(-1))[loss_mask_flat == 1],
    temperature=temperature
)
```

假设 8 个位置中有 5 个有效位置：

```
loss_mask_flat: [0, 1, 1, 1, 0, 0, 1, 1]

学生 [8, V] ──按 mask 筛选──→ [5, V]
教师 [8, V] ──相同位置筛选──→ [5, V]
                                  ↓
每个有效位置比较整个词表分布，再对 5 个位置平均
```

与 DPO 中的 `logits_to_log_probs` 不同，这里没有通过 `gather` 只取目标 token 的概率，而是保留所有词表候选，比较完整分布。

若没有教师，则 `distill_loss` 被设为设备上的零标量。若有教师但有效位置数量为 0，筛选结果为 `[0, V]`，当前代码没有单独保护这个情况，`batchmean` 可能产生非有限损失；训练数据需要确保有有效目标，或实现时增加相应处理。

7.3.7 混合损失与梯度累积

```
loss = (alpha * ce_loss + (1 - alpha) * distill_loss) / args.accumulation_steps
scaler.scale(loss).backward()
```

例如 `alpha=0.5`，忽略 MoE 和累积缩放，若当前 batch 的 CE 为 1.204、蒸馏损失为 0.373，则总损失约为：

```
0.5 × 1.204 + 0.5 × 0.373 = 0.7885
```

如果 `accumulation_steps=4`，每批 loss 再除以 4，累计四批梯度后更新一次。`scaler.scale` 在启用梯度缩放时放大损失，降低 FP16 梯度下溢风险。

函数签名默认 `alpha=0.0`，实际调用可以覆盖它。如果没有教师且 `alpha=0`，CE 与蒸馏项都没有有效训练信号；这不等于正常的监督训练，也不保证优化器状态或权重衰减不会改变参数。

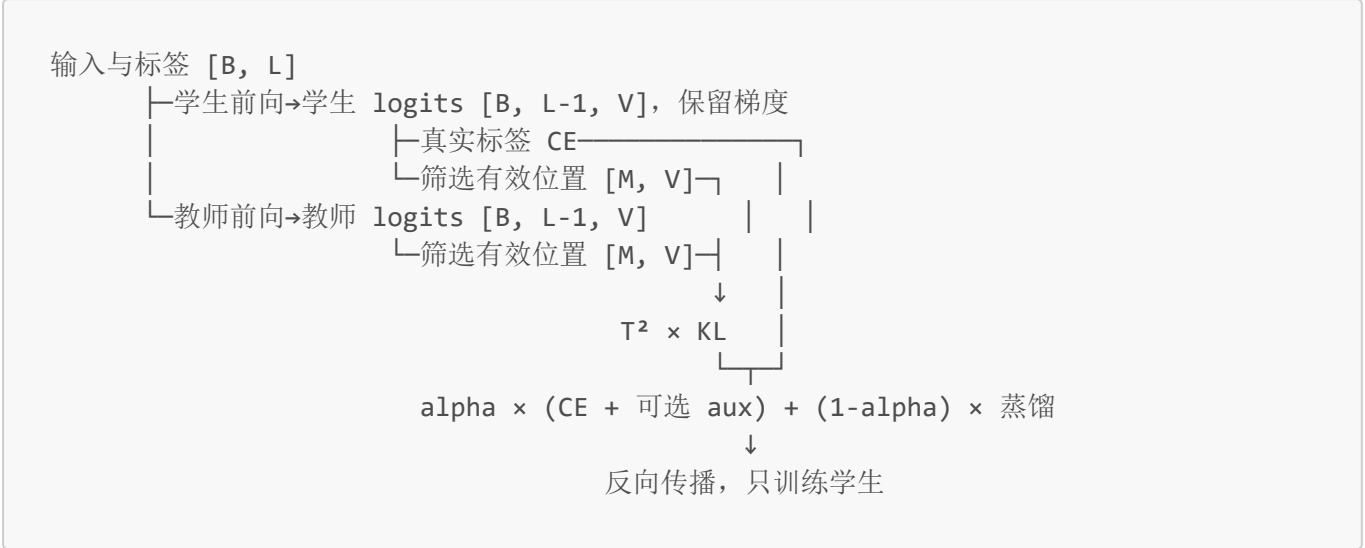
7.3.8 更新、日志和保存

```
if step % args.accumulation_steps == 0:
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
```

按顺序完成恢复梯度尺度、裁剪、更新学生参数、更新缩放因子和清空梯度。教师始终不参与参数更新。

日志分别记录原始 CE、辅助损失、蒸馏损失和混合后的总损失。`current_loss = loss.item() * accumulation_steps` 还原的是当前 batch 的混合损失，不是整个累积周期的平均损失。

保存时只由主进程写入学生权重，并另外保存可续训的优化器、scaler 和进度状态。循环结束后会处理不足一个累积周期的剩余梯度；与前面 DPO 一样，尾部仍按完整累积步数缩放，且循环内最后一次保存可能尚未包含这次尾部更新。



8. Agent 训练：在多轮工具交互中学习

8.1 为什么需要 Agent 训练

普通问答模型直接生成答案，但有些任务需要先计算、查询或转换，再结合结果回答。Agent 训练让模型学习何时提出工具调用、如何填写参数，以及如何根据工具返回的信息继续完成任务。

```
普通问答：
用户：计算 12 × 8。
模型：96。

工具交互：
用户：计算 12 × 8。
模型：<tool_call>{"name":"calculate_math","arguments":{"expression":"12*8"}}
</tool_call>
程序执行工具：{"result":"96"}
模型：12 × 8 = 96。
```

模型生成的仍然是 token，不会因为输出了函数名就自动执行函数。脚本需要识别调用文本、执行工具、把观察结果送回模型，然后继续采样。

这就是 `train_agent.py` 比普通 GRPO 脚本长的主要原因：它同时包含工具环境、多轮交互、轨迹整理、奖励规则和强化学习更新。优化核心仍然使用前面学过的组内优势与策略损失，默认选择 CISPO 分支，也支持 GRPO 分支。

文件部分	职责	阅读重点
`TOOLS`、模拟数据、`MOCK_RESULTS`	定义工具接口和执行环境	工具描述与工具实现的区别
`parse_tool_calls`、`execute_tool`	从文本识别调用并执行	模型输出如何变成操作

文件部分	职责	阅读重点
`rollout_single`、`rollout_batch`	运行并记录多轮交互	新增逻辑的核心
`calculate_rewards`	给完整交互评分	调用格式、参数与最终答案
`rl_train_epoch`、主程序	整理张量并训练	与普通 GRPO 的对应关系

建议先读 `rollout_single → calculate_rewards → rl_train_epoch`，再看工具字典和启动参数。

8.2 理论

8.2.1 从一条回答变成一条交互轨迹

一条轨迹是从初始问题到任务结束的整段交互，可以写成：

$$\tau = (s_0, a_0, o_1, a_1, o_2, \ldots, a_K)$$

其中，s_0 是初始上下文，a_k 是模型生成的文本或工具调用，o_k 是环境返回的观察。后续模型生成条件包含此前所有保留的交互信息。

```
问题与工具描述
      ↓ 模型生成
工具调用
      ↓ 程序执行
工具观察
      ↓ 模型继续生成
最终答案，或下一次调用
```

本实现没有对每次工具调用单独训练一个价值函数。它对整条轨迹打一个总分，再将同题轨迹进行组内比较。

8.2.2 模型动作与环境观察必须区分

工具返回的信息不是模型采样出来的，不能直接当作模型动作来计算策略损失。定义目标 token 的动作掩码 m_t：

$$m_t = \begin{cases} 1, & \text{该目标 token 由模型生成并被保留用于训练} \\ 0, & \text{问题、工具观察、补入模板或 PAD 等位置} \end{cases}$$

```
内容：        用户问题 | 模型工具调用 | 工具返回结果 | 模型最终回答
训练 mask：  0 0 0    | 1 1 1 1      | 0 0 0        | 1 1 1
```

工具结果虽然不直接承担策略损失，仍作为上下文参与后续模型预测。训练最终回答时，模型可以学习如何利用工具结果。

注意力掩码和动作掩码职责不同：

掩码	作用	工具结果位置
`full_mask`	区分有效输入和 PAD	有效观察通常为 1
`completion_mask`	指定哪些目标 token 参与策略损失	为 0

8.2.3 轨迹奖励与组相对优势

设每个问题采样 G 条轨迹，奖励为 $r_{i,1},\dots,r_{i,G}$，则：

$$A_{i,j}=\frac{r_{i,j}-\mu_i}{\sigma_i+10^{-4}}$$

这里仍然是前面 GRPO 的组内标准化，但评分对象从单次回答变成了多轮轨迹。

```
问题 A：轨迹 A1 正确调用并答对 → 较高分 → 正优势
        轨迹 A2 调用错误         → 较低分 → 负优势

问题 B：轨迹 B1 完成任务         → 较高分 → 正优势
        轨迹 B2 达到轮数上限     → 较低分 → 负优势
```

同一轨迹的优势共享给其中所有被动作掩码选中的 token。它没有判断某一轮调用应获得多少独立优势，也不保证每个正优势轨迹中的局部动作都正确。

8.2.4 策略更新仍是 GRPO / CISPO

对轨迹中模型生成的 token，计算当前策略相对采样策略的概率比：

$$\rho_t=\exp(\log\pi_\theta(a_t\mid s_t)-\log\pi_{\text{old}}(a_t\mid s_t))$$

参考策略约束使用：

$$d_t=\log\pi_{\text{ref}}(a_t\mid s_t)-\log\pi_\theta(a_t\mid s_t),\qquad k_t=e^{d_t}-d_t-1$$

GRPO 分支为：

$$\ell_t=-\min(\rho_tA,\text{clip}(\rho_t,1-\epsilon,1+\epsilon)A)+\beta k_t$$

CISPO 分支为：

$$\ell_t=-\,\text{stopgrad}(\min(\rho_t,\epsilon_{\text{high}}))A\log\pi_\theta(a_t\mid s_t)+\beta k_t$$

`stopgrad` 对应 `.detach()`。两种分支共用同一套组内优势；文件默认 `loss_type="cispo"`，其比值上限直接是 `epsilon_high`。

每条有效轨迹先按动作 token 数取平均，再在有效轨迹间平均：

$$L_{\text{policy}}=\frac{1}{|\mathcal{V}|}\sum_{n\in\mathcal{V}}\frac{\sum_t m_{n,t}\ell_{n,t}}{\sum_t m_{n,t}},\qquad \mathcal{V}=\{n:\sum_t m_{n,t}>0\}$$

8.3 代码分析

对应源码：train_agent.py。继续使用两个问题、每题两条轨迹，即 $B = 2, G = 2, N = 4$。

8.3.1 工具描述、解析与模拟执行

`TOOLS` 描述工具名、用途和参数 schema；`MOCK_RESULTS` 根据名称提供 Python 执行逻辑；`CHECK_ARGS` 进行基本参数检查，主要用于奖励阶段。

```
TOOLS：告诉模型“有哪些工具、参数怎么写”
parse_tool_calls：读取模型“想调用哪个工具”
execute_tool：程序真正执行对应逻辑
CHECK_ARGS：奖励阶段检查参数是否基本有效
```

模型通过聊天模板看到本题的工具列表：

```
context = tokenizer.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True,
    tools=tools, open_thinking=open_thinking
)
```

`parse_tool_calls` 使用正则提取 `<tool_call>...</tool_call>` 中的内容，再用 `json.loads` 解析。JSON 解析失败的片段会被忽略，因此“没有解析到调用”不必然等于模型完全没有尝试调用。

天气、时间、汇率、翻译等主要来自文件内固定的模拟数据，不是实时外部服务。`calculate_math` 会根据表达式计算；这些实现用于构造练习环境，不能把它们当作完整生产工具接口。

8.3.2 `rollout_single`：一轮一轮推进交互

```
for turn in range(max_turns):
    # 根据当前 messages 构造输入
    # 调用 rollout_engine 生成本轮输出
    # 记录模型生成的 token、mask、旧概率
    calls = parse_tool_calls(new_text)
    if not calls:
        break

    unfinished = turn == max_turns - 1
    messages.append({"role": "assistant", "content": new_text})
    for call in calls:
        result = execute_tool(name, raw)
        messages.append({"role": "tool", "content": result_str})
    # 编码新增观察，准备下一轮
```

这是流程摘录，参数解析、结果序列化等细节在源码中完成。训练调用设置 `max_turns=3`，表示最多生成三轮；单轮可以提出多个工具调用，所以不等于最多调用三次工具。

```
初始 messages:
[user]
    ↓ 第 1 轮模型提出调用，程序执行
[user, assistant工具调用, tool结果]
    ↓ 第 2 轮模型读取结果并给出最终回答
结束：最终文本另行保存在 all_outputs 中
```

最后一轮仍提出调用时，`unfinished=True`，即使程序继续执行了该轮工具，也没有下一轮生成预算来消费这些结果。`open_thinking` 在轨迹开始时按概率抽取，供各轮模板使用。

8.3.3 分别记录动作和观察

模型生成部分：

```
response_ids.extend(new_ids)
response_mask.extend([1] * len(new_ids))
response_old_logps.extend(new_logps)
```

这里保留采样引擎返回的 token 及其 log 概率；代码会先过滤 PAD 和 EOS。因此后续即使还有 EOS 截断逻辑，也不应理解为这里始终直接训练采样出的 EOS。

工具执行后，代码重新渲染上下文，再取新增片段：

```
current_len = len(prompt_ids) + len(response_ids)
obs_delta = observe_ids[current_len:]
response_ids.extend(obs_delta)
response_mask.extend([0] * len(obs_delta))
response_old_logps.extend([0.0] * len(obs_delta))
```

`obs_delta` 包含新增的工具观察及模板补入内容。这里的旧 log 概率 `0.0` 是占位值，不是说工具文本由模型以概率 1 生成；这些位置会被动作 mask 排除。

这个拼接方法依赖重新渲染的上下文与已记录轨迹前缀保持一致。阅读或修改时，需要特别留意模板、分词边界和被过滤特殊 token 的对齐关系。

8.3.4 为什么 rollout 返回这么多数据

数据	含义与用途
`final_output` / `completions`	最后一轮模型文本，用于最终答案评分
`final_context` / `contexts`	最终上下文文本，主要供检查与日志
`prompt_ids`	初始输入 token
`response_ids`	后续交互 token，既含动作也含观察

数据	含义与用途
response_mask	后续序列中的动作位置
response_old_logps	动作 token 的采样概率与观察占位值
all_outputs / turn_outputs	每轮模型文本，用于检查整条轨迹的调用
unfinished	是否用尽轮数仍停留在工具调用阶段

rollout_batch 对每个问题重复运行 rollout_single，并复制初始消息列表，避免某条采样轨迹改变下一条轨迹的起始上下文。

```
问题 A → A1 完整轨迹、A2 完整轨迹
问题 B → B1 完整轨迹、B2 完整轨迹

返回排列：[A1, A2, B1, B2]
```

虽然叫 rollout_batch，这份实现实际通过循环逐条调用 rollout_single，不是在该函数里同时批量推进所有轨迹。

8.3.5 奖励：先收集所有轮次的调用

```
sample_idx = idx // num_gen
turn_answers = [
    turn.split('</think>', 1)[-1].strip()
    if '</think>' in turn else turn.strip()
    for turn in turn_outputs
]
tool_calls = []
for turn_answer in turn_answers:
    tool_calls.extend(parse_tool_calls(turn_answer))
```

sample_idx 将轨迹索引映射回问题索引。检查调用时会汇总所有轮次，而不只看最后一轮。最终答案则主要取最后一轮输出去除思考部分后的内容。

标签开闭数量不匹配会扣分：

```
reward -= 0.5 * sum(
    abs(turn.count('<tool_call>') - turn.count('</tool_call>'))
    for turn in turn_answers
)
```

这是数量检查，不是完整的标签嵌套或 JSON schema 验证。

8.3.6 没有工具调用的评分分支

若整条轨迹没有解析到工具调用，使用：

```
回答长度在 5～800 字符：+0.5，否则 -0.5
存在 </think> 时：思考长度和结束标签规则加减分
如果提供 reward_model：加入回答质量分
扣除重复惩罚
最后将总分裁剪到 [-3, 3]
```

这个分支与前面的普通问答奖励相似，但此处整体长度下限是 5。它没有在该分支调用 GT 匹配来评分；评分函数允许不提供奖励模型，但脚本主程序实际会加载奖励模型。

8.3.7 有工具调用的评分分支

首先计算有效调用数量：

```
valid_call_count += int(bool(
    name in valid_names and check and check(raw)
))
```

这里的“有效”只表示名称在本题允许列表中，且通过基本参数检查；不表示工具一定成功执行，也不表示参数语义一定正确。执行路径调用 `execute_tool`，允许列表检查则在奖励路径进行，两者职责要区分。

接着：

```
tool_gap = abs(valid_call_count - len(gt)) + max(0, len(tool_calls) -
valid_call_count)
reward += 0.5 if tool_gap == 0 else -0.5 * tool_gap
```

这套规则以 `len(gt)` 作为期望调用数量的依据，是当前数据与奖励设计的假设，不是通用 Agent 原理。

最后检查答案：

```
final_text = "" if unfinished else (
    answer.split('</tool_call>')[-1] if '</tool_call>' in answer else answer
)
verified = validate_gt_in_text(final_text, gt) if gt else set()
if gt:
    reward += 2.5 * len(verified) / len(gt)
if unfinished:
    reward -= 0.5
```

`validate_gt_in_text` 使用字符串包含或数值近似相等来判断命中。它不是完整语义验证，也不证明答案确实通过正确工具执行得到。随后扣除重复惩罚，将总分限制在 `[-3, 3]`。

例如 gt=["96"]，一条轨迹只有一次有效工具调用，已完成，最终文本命中 96，且没有格式或重复扣分：

```
调用数量匹配：+0.5
GT 全部命中：  +2.5
最终奖励：      3.0
```

虽然源码注释叫“执行结果奖励”，理解时应以实际检查为准：这里主要评价调用的基本有效性和最终答案匹配。

8.3.8 打包轨迹、截断与补齐

这一段位于 rl_train_epoch 中，负责将实际采样的交互整理成可训练的批次。先读本批次数据：

```
last_step = start_step
for step, batch in enumerate(loader, start=start_step + 1):
    messages_batch = batch['messages']
    tools_batch = batch['tools']
    gt_batch = batch['gt']
    last_step = step
```

例如两个问题分别是“计算 12 × 8”和“计算 3 + 4”，则 messages_batch 存放两个问题的初始对话，tools_batch 存放各自允许的工具列表，gt_batch 可以是 [["96"], ["7"]]。last_step 记录实际处理到的步骤；enumerate 只调整编号，不负责跳过数据。

随后在 torch.no_grad() 中调用 rollout_batch。每题采样两次，得到 [A1, A2, B1, B2] 四条轨迹。采样阶段不建立训练计算图，后面会将这些轨迹重新输入当前模型，计算带梯度的概率。

```
prompts = [
    tokenizer.apply_chat_template(
        m, tokenize=False, add_generation_prompt=True, tools=t
    )
    for m, t in zip(messages_batch, tools_batch)
]
```

这里重新渲染的问题文本用于奖励计算；训练用的 token 则来自 rollout 返回的记录。

打包循环中四个短变量的含义是：

```
for p, r, m, old_lp in zip(
    prompt_ids_batch, response_ids_batch,
    response_masks_batch, response_old_logps_batch
):
    ...
```

变量	含义
`p`	初始问题与工具描述等上下文 token
`r`	后续交互 token，包含模型动作和工具观察
`m`	`r` 中哪些 token 是保留的模型动作
`old_lp`	动作 token 的采样 log 概率，观察位置用零占位

```
ids = p + r
mask = [0] * len(p) + m
old_logps = [0.0] * max(len(p) - 1, 0) + old_lp
```

`ids` 与 `mask` 按完整序列位置存储；旧 log 概率用于“预测下一个 token”，因此初始占位长度为 `len(p)-1`。

用一条简化轨迹逐个对齐。`P` 表示初始上下文 token，`C` 表示工具调用 token，`O` 表示工具观察 token，`A` 表示最终答案 token；它们只是位置示意，不是实际 tokenizer 编号。

```
p      = [P1, P2]
r      = [C1, C2, O1, A1, A2]
m      = [ 1,  1,  0,  1,  1]
old_lp = [-0.2, -0.3, 0.0, -0.4, -0.1]
```

这里 `p`、`r` 都是 Python 列表，`p + r` 表示拼接，不是逐元素加法：

```
ids:      P1   P2   C1   C2   O1   A1   A2
mask:      0    0    1    1    0    1    1
```

为什么旧概率少一位？长度为 7 的完整序列对应 6 个相邻预测任务：

```
模型预测位置:  P1    P2    C1    C2    O1    A1
实际预测目标:  P2    C1    C2    O1    A1    A2
旧 log 概率:    0   -0.2  -0.3    0   -0.4  -0.1
                     ↑
               最后一个问题位置 P2 预测第一个动作 C1

len(ids) = 7
len(old_logps) = 6
```

问题前缀中只有 `P2` 这个下一 token 目标需要补占位，因此这里补 `len(p)-1=1` 个零。`O1` 对应的零同样只是占位值，该位置不承担策略损失。

再用三个问题 token，把三行代码的拼接过程完全展开。

先记住 Python 列表的两个运算：

```
[1, 2] + [3, 4]  # [1, 2, 3, 4]：列表拼接
[0] * 3          # [0, 0, 0]：重复列表内容
```

这里不是张量逐元素相加。假设本次的列表为：

```
p      = [P1, P2, P3]              初始问题，3 个 token
r      = [C1, C2, O1, A1, A2]      后续交互，5 个 token
m      = [ 1,  1,  0,  1,  1]     模型动作标 1，工具观察标 0
old_lp = [-0.2, -0.3, 0.0, -0.4, -0.1]
             C1    C2   O1    A1    A2
```

第一行 `ids = p + r`，将问题与后续交互直接接起来：

```
p:     [P1, P2, P3]
r:                  [C1, C2, O1, A1, A2]
                      ↓ 拼接
ids:   [P1, P2, P3,  C1, C2, O1, A1, A2]

长度：3 + 5 = 8
```

第二行 `mask = [0] * len(p) + m`，先为问题生成三个零，再接上原来的动作 mask：

```
len(p) = 3
[0] * len(p) = [0, 0, 0]

问题 mask:  [0, 0, 0]
m:                    [1, 1, 0, 1, 1]
                        ↓ 拼接
mask:       [0, 0, 0,  1, 1, 0, 1, 1]

对齐 ids:    P1 P2 P3  C1 C2 O1 A1 A2
```

问题不直接承担策略损失，因此补零；工具观察在原来的 `m` 中已经标为零。完整 `mask` 的长度与 `ids` 一样，都是 8。

第三行 `old_logps = [0.0] * max(len(p) - 1, 0) + old_lp`，先算出需要补两个零：

```
max(len(p) - 1, 0) = max(3 - 1, 0) = 2
[0.0] * 2 = [0.0, 0.0]

问题占位:  [0.0, 0.0]
old_lp:              [-0.2, -0.3, 0.0, -0.4, -0.1]
```

```
                          ↓ 拼接
old_logps: [0.0, 0.0,  -0.2, -0.3, 0.0, -0.4, -0.1]

长度：2 + 5 = 7
```

`max(..., 0)` 防止问题长度为空时算出负的重复次数。正常非空 prompt 下，补的就是 `len(p)-1` 个零。

这七个数对应七个"预测下一个 token"的任务，而不是完整序列的八个 token 位置：

```
完整序列：       P1   P2   P3   C1   C2   O1   A1   A2
模型预测位置：   P1   P2   P3   C1   C2   O1   A1
预测目标：       P2   P3   C1   C2   O1   A1   A2
旧 log 概率：     0    0  -0.2 -0.3   0  -0.4 -0.1
                        ↑
                    P3 预测 C1
```

最关键的是：最后一个问题 token `P3` 已经开始预测第一个动作 `C1`，所以它的预测概率应当使用 `old_lp` 中的第一个值 `-0.2`，不能再放一个问题占位零。如果前面补三个零，就会把 `C1` 的概率错后一位。

最后把三个列表放在一起：

```
ids:          [P1, P2, P3,   C1,   C2, O1,   A1,   A2]  长度 8
mask:         [ 0,  0,  0,    1,    1,  0,    1,    1]  长度 8

ids[1:]:         [P2, P3,   C1,   C2, O1,   A1,   A2]  预测目标
old_logps:       [ 0,  0, -0.2, -0.3,  0, -0.4, -0.1]  长度 7
mask[1:]:        [ 0,  0,    1,    1,  0,    1,    1]  长度 7
```

后面批量计算时的 `full_response_masks[:, 1:]`，就是把每条样本的 `mask[1:]` 取出来，与预测目标和 log 概率统一对齐。

```
ids:  [问题 | 模型调用 | 工具观察 | 模型答案]
mask: [ 0   |    1     |    0     |    1   ]
```

超过 `max_total_len` 时保留最后这些 token，mask 和旧概率同步截取。训练可能因此只看到原轨迹的后半段；奖励仍来自采样出的轨迹文本。

```python
if len(ids) > args.max_total_len:
    ids = ids[-args.max_total_len:]
    mask = mask[-args.max_total_len:]
    old_logps = old_logps[-(len(ids) - 1):]

prompt_len = next((i for i, v in enumerate(mask) if v == 1), len(mask))
packed_samples.append((ids, mask, prompt_len, old_logps))
```

`next(...)` 寻找 mask 中第一个 `1` 的索引；没有 `1` 则返回 `len(mask)`。上例的 `prompt_len=2`。它更准确地表示“当前保留序列中第一个动作的位置”，截断后不一定还是原始 prompt 长度。

随后按当前批次最长长度 S 右侧补齐：

```
轨迹 A1: [问题 | 调用 | 结果 | 答案 | PAD PAD]
轨迹 A2: [问题 | 回答 | PAD PAD PAD PAD PAD]
轨迹 B1: [问题 | 调用 | 结果 | 调用 | 结果 | 答案]
轨迹 B2: [问题 | 调用 | 结果 | PAD PAD PAD]

input_ids:             [4, S]
full_response_masks:   [4, S]
old_per_token_logps:   [4, S-1]
```

例如四条轨迹长度为 `[7, 5, 8, 6]`：

```
seq_lens = [7, 5, 8, 6]
max_len = 8

A1: 有效 token × 7 + PAD × 1
A2: 有效 token × 5 + PAD × 3
B1: 有效 token × 8
B2: 有效 token × 6 + PAD × 2
```

具体补齐规则：

张量	补什么	例子中的形状
`input_ids`	PAD token 编号	`[4, 8]`
`full_response_masks`	0，PAD 不算动作	`[4, 8]`
`old_per_token_logps`	0.0，概率占位值	`[4, 7]`
`prompt_lens`	每条轨迹的首个动作位置，无需序列补齐	`[4]`

```
full_mask = (input_ids != tokenizer.pad_token_id).long()
```

这个 mask 只区分输入是否为 PAD：

```
内容:                问题  模型调用  工具观察  模型答案  PAD
full_mask:             1      1        1        1       0
full_response_masks:   0      1        0        1       0
```

打包完成后调用 `calculate_rewards(...)`，根据工具调用、标准结果、完成状态等为四条轨迹各给一个分数，得到 `rewards [4]`。

8.3.9 前向计算与目标 mask 对齐

```
res = model_unwrapped(input_ids, attention_mask=full_mask)
logits = res.logits[:, :-1, :]
per_token_logps = F.log_softmax(logits, dim=-1).gather(
    2, input_ids[:, 1:].unsqueeze(-1)
).squeeze(-1)

completion_mask = full_response_masks[:, 1:]
```

```
当前模型 logits:          [4, S-1, V]
实际下一个 token 的 logp: [4, S-1]
采样时的旧 logp:          [4, S-1]
参考模型 logp:            [4, S-1]
completion_mask:          [4, S-1]
```

参考模型在 `no_grad()` 中计算概率。mask 向右错开一位，与目标 token 对齐；后续还检查动作位置上的 EOS，并排除其后位置。

当前模型前向保留梯度。`model_unwrapped` 是从 DDP 包装中取出的模型；`autocast_ctx` 控制混合精度；`aux_loss` 在使用 MoE 时取模型辅助损失，否则为零。

沿用上例，mask 的错位过程是：

```
原序列:      P1  P2  C1  C2  O1  A1  A2
原 mask:      0   0   1   1   0   1   1
                ↓ 去掉第一个位置
预测目标:       P2  C1  C2  O1  A1  A2
completion_mask: 0   1   1   0   1   1
```

因此 `completion_mask = full_response_masks[:, 1:]` 与 `input_ids[:, 1:]` 一一对应。第一维仍为轨迹数，第二维从 S 变成 $S-1$。

三份 log 概率也需要区分：

变量	来源	在本次反向传播中是否作为固定数据
`old_per_token_logps`	rollout 时记录的采样概率	是
`per_token_logps`	当前模型重新评价已采样轨迹	否，梯度通过这里传播
`ref_per_token_logps`	冻结参考模型评价相同轨迹	是

EOS 部分可以按以下顺序理解：

```
is_eos：找出“目标是 EOS 且属于动作”的位置
has_eos：逐条判断是否存在这样的位置
eos_idx：存在时取第一个 EOS 索引，否则取最后位置
pos <= eos_idx：仅保留到该位置，包括 EOS 本身
最后乘回原 completion_mask，继续屏蔽工具观察等位置
```

前面的 rollout 已过滤采样结果中的 EOS，因此这段检查是否实际生效，取决于最终打包的 token 和动作 mask。

```
token_counts = completion_mask.sum(dim=1)
valid_rows = token_counts > 0
```

只有包含有效动作 token 的轨迹参与策略损失的轨迹平均。若整批没有有效动作，代码构造零策略项，MoE 辅助项是否有贡献则取决于模型配置。

```
token_counts = [4, 2, 6, 0]
valid_rows   = [True, True, True, False]
```

这里的计数是动作 token 数，不是整条序列长度。中间 `debug_mode` 分支只打印上下文、token 解码结果和奖励，不参与损失计算。

8.3.10 组内优势与更新

```
grouped_rewards = rewards.view(-1, args.num_generations)
mean_r = grouped_rewards.mean(dim=1).repeat_interleave(args.num_generations)
std_r = grouped_rewards.std(dim=1,
unbiased=False).repeat_interleave(args.num_generations)
advantages = (rewards - mean_r) / (std_r + 1e-4)
```

形状变化与普通 GRPO 完全对应：

```
轨迹奖励 [4] → 分组 [2, 2] → 均值/标准差各 [2]
                                ↓ repeat_interleave(2)
                              展开各 [4]
                                ↓ 标准化
                              轨迹优势 [4]
                                ↓ unsqueeze(1) + 广播
                              作用于 [4, S-1]
```

再计算概率比、参考 KL 与选择的 GRPO/CISPO 损失，按 `completion_mask` 只汇总模型动作部分：

```
loss = (policy_loss + aux_loss) / args.accumulation_steps
loss.backward()
```

源码里较长的 `policy_loss` 表达式，可以等价拆成：

```
if valid_rows.any():
    # 1. 对每条轨迹，仅加总动作 token 的损失
    loss_sum = (per_token_loss * completion_mask).sum(dim=1)

    # 2. 排除没有动作 token 的轨迹
    valid_loss_sum = loss_sum[valid_rows]
    valid_counts = token_counts[valid_rows]

    # 3. 每条轨迹内部平均，再在有效轨迹之间平均
    policy_loss = (valid_loss_sum / valid_counts.clamp(min=1)).mean()
else:
    # 在逐 token 损失有限时，得到连接计算图的零策略项
    policy_loss = per_token_loss.sum() * 0.0
```

例如三条有效轨迹的动作损失总和分别为 `2、3、6`，动作 token 数为 `4、2、6`：

```
每条轨迹平均：[2/4, 3/2, 6/6] = [0.5, 1.5, 1.0]
轨迹间平均：   (0.5 + 1.5 + 1.0) / 3 = 1.0
```

这里是“每条轨迹先平均”，不是把所有 token 混在一起除以总 token 数。零策略项不是通用的 NaN 防护：非有限值乘零仍可能是 NaN。

用户所贴的片段到 `loss.backward()` 为止，它只完成梯度计算与累积，尚未执行参数更新；实际更新发生在后面的 `optimizer.step()`。

达到累积间隔后进行梯度裁剪、优化器更新、学习率调度和梯度清空。工具执行与奖励检查不需要可微；它们提供固定分数，训练梯度通过当前模型对已采样动作的 log 概率传播。

```
问题 + 工具 + gt
      ↓ rollout，不建立训练计算图
四条交互轨迹：token、动作 mask、旧概率
      ↓ 拼接、截断、补齐
input_ids [4, S]，目标概率和动作 mask [4, S-1]
      ├─轨迹评分 [4] → 组内优势 [4]
      └─当前模型和参考模型重新计算 token log 概率
                          ↓
                概率比、参考 KL、GRPO/CISPO 损失
                          ↓
                mask 排除问题、工具观察、PAD
                          ↓
                轨迹内部平均，再在轨迹间平均
```

```
                          ↓
                加 aux_loss，缩放，backward
```

8.3.11 主程序、日志与运行细节

主程序负责加载策略模型、冻结参考模型、加载奖励模型、选择 rollout 引擎、创建 `AgentRLDataset`，然后配置优化器、学习率调度、断点恢复与分布式包装。每个 batch 包含 `messages`、`tools` 和 `gt`。

日志和保存流程与普通 GRPO 类似。日志中的 `KL` 是参考与当前 log 概率之差的采样平均，不是训练使用的非负 `exp(d)-d-1`；总损失日志也可能包含 `aux_loss`。采样策略在保存间隔或最后一步同步，因此它可能落后于当前训练策略。

epoch 尾部会更新不足一个累积周期的梯度，但循环内最后保存和策略同步发生在这次尾部更新之前，需要留意实际保存的参数版本。

当前 `execute_tool` 通过 `signal.SIGALRM` 和 `signal.alarm` 实现超时。原生 Windows 不支持这套机制；相关异常在当前函数中会被捕获并返回 `None`，可能导致工具无法执行。这是脚本的运行平台限制，不是 Agent 训练算法本身的要求。

```
普通 GRPO                              本文件 Agent 训练
一条回答                        →      一条多轮交互轨迹
评价回答文本                    →      评价调用与最终答案
回答区域大多是模型生成          →      模型动作与环境观察交替
对有效回答 token 计算损失       →      仅对保留的模型动作 token 计算损失

共同核心：组内奖励标准化 → 概率比与参考 KL → 策略损失 → 参数更新
```